

January 17, 2026

1 Analisi del Rischio Creditizio - Dataset LendingClub

Il dataset proviene da LendingClub, una piattaforma americana di prestiti. Contiene informazioni su migliaia di prestiti erogati: le caratteristiche dei richiedenti, le condizioni applicate e, soprattutto, se il prestito è stato ripagato o meno.

Il nostro obiettivo non è semplicemente descrivere i dati, ma interrogarli. Ogni analisi parte da una domanda concreta e cerca una risposta nei numeri.

1.1 Indice

1.1.1 Parte 1 - Preparazione dei Dati

1. Caricamento del dataset
2. Rinomina delle variabili
3. Descrizione delle variabili
4. Gestione valori mancanti
5. Conversione tipi di dato

1.1.2 Parte 2 - Analisi Esplorativa

6. Panoramica delle distribuzioni
7. Sbilanciamento delle classi
8. Analisi Interest Rate
9. Analisi Installment
10. Analisi Credit History
11. Analisi FICO Score
12. Analisi Purpose
13. **Analisi Annual Income**
14. Analisi DTI
15. Analisi Revolving Utilization
16. Analisi Credit Policy
17. Analisi Inquiries
18. Analisi Public Records
19. Analisi Delinquencies

1.1.3 Parte 3 - Verifica Statistica

20. Test di normalità

- 21. Confronto tra gruppi (t-test)
- 22. Associazioni categoriche (chi-quadrato)

1.1.4 Parte 4 - Analisi Multivariata

- 23. Correlazioni e covarianze
- 24. Regressione lineare
- 25. Diagnostica del modello

1.1.5 Parte 5 - Descrizione dei dati

- 26. Clustering dei clienti (K-Means)
- 27. Riduzione dimensionalità (PCA)

1.1.6 Parte 6 - Modelli Predittivi

- 28. Regressione logistica
- 29. Valutazione del modello (ROC, Confusion Matrix)
- 30. Strategia di pricing del rischio
- 31. Confronto con K-Nearest Neighbors
- 32. Feature Importance
- 33. Gaussian Naive Bayes
- 34. Confronto finale dei modelli

1.1.7 Parte 7 - Conclusioni

- 35. Sintesi dei risultati
-

1.2 Struttura del notebook

Parte 1 - Preparazione dei dati Prima di analizzare qualsiasi cosa, dobbiamo conoscere i dati: cosa rappresentano, se sono completi, se contengono errori.

Parte 2 - Analisi esplorativa Osserviamo le distribuzioni delle variabili chiave. Ci chiediamo: quali valori sono tipici? Quali sono anomali? Esistono pattern evidenti?

Parte 3 - Verifica statistica Le differenze che osserviamo sono reali o dovute al caso? Appliciamo test statistici per validare le nostre intuizioni.

Parte 4 - Analisi multivariata Le variabili non agiscono in isolamento. Studiamo correlazioni e costruiamo modelli per capire quali fattori influenzano davvero il rischio.

Parte 5 - Descrizione dei dati Segmentiamo i clienti con il clustering e riduciamo la complessità con la PCA.

Parte 6 - Modello predittivo e strategia Costruiamo un modello per prevedere il default e lo traduciamo in una strategia decisionale concreta per la banca. Confrontiamo diversi approcci (Logistic Regression, KNN, Naive Bayes) per trovare la soluzione migliore.

Parte 7 - Conclusioni Sintetizziamo i risultati principali, le implicazioni pratiche e i possibili sviluppi futuri.

Parte 1 - Preparazione dei Dati

1.3 Caricamento del Dataset

Iniziamo caricando il dataset e guardando la sua struttura.

```
[112]: Index(['customer.id', 'credit.policy', 'purpose', 'int.rate', 'installment',  
        'log.annual.inc', 'dti', 'fico', 'days.with.cr.line', 'revol.bal',  
        'revol.util', 'inq.last.6mths', 'delinq.2yrs', 'pub.rec',  
        'not.fully.paid'],  
        dtype='object')
```

Rinomina delle Variabili

I nomi originali usano punti e abbreviazioni poco leggibili. Li redominiamo per avere una maggiore leggibilità

```
['customer_id', 'credit_policy', 'purpose', 'interest_rate', 'installment',  
'log_annual_income', 'dti', 'fico_score', 'credit_history_days',  
'revolving_balance', 'revolving_util', 'inquiries_last_6m', 'delinquencies_2y',  
'public_records', 'not_fully_paid']  
Empty DataFrame  
Columns: [customer_id, credit_policy, purpose, interest_rate, installment,  
log_annual_income, dti, fico_score, credit_history_days, revolving_balance,  
revolving_util, inquiries_last_6m, delinquencies_2y, public_records,  
not_fully_paid]  
Index: []
```

Che informazioni abbiamo su ogni cliente?

Prima di analizzare i dati, dobbiamo capire cosa rappresenta ogni variabile. Ecco un dizionario delle colonne:

Identificazione - `customer_id`: codice univoco del cliente

Profilo creditizio - `credit_policy`: indica se il cliente rispetta i criteri di credito della piattaforma (1 = sì, 0 = no) - `fico_score`: punteggio creditizio americano, scala da 300 a 850. E il numero che sintetizza l'affidabilità creditizia di una persona - `credit_history_days`: anzianità della storia creditizia in giorni - `delinquencies_2y`: numero di pagamenti in ritardo negli ultimi 2 anni - `public_records`: eventi negativi pubblici come fallimenti o pignoramenti - `inquiries_last_6m`: quante volte negli ultimi 6 mesi qualcuno ha richiesto il report creditizio del cliente

Situazione debitoria - `dti` (Debt-to-Income): percentuale del reddito mensile destinata al pagamento di debiti - `revolving_balance`: saldo attuale su linee di credito rotativo (carte di credito e simili) - `revolving_util`: percentuale del credito disponibile effettivamente utilizzata - `not_fully_paid`: 1 = il cliente non ha ripagato completamente, 0 = ha ripagato regolarmente

Dettagli del prestito - `purpose`: motivazione dichiarata del prestito - `interest_rate`: tasso di interesse applicato - `installment`: importo della rata mensile

Reddito - `log_annual_income`: logaritmo naturale del reddito annuale

Questa è la variabile che vogliamo prevedere.

```
[114]:  customer_id credit_policy      purpose interest_rate installment \
0          10001             1 debt_consolidation      0.1189         829.10
1          10002             1      credit_card       0.1071         228.22
2          10003             1 debt_consolidation      0.1357         366.86
3          10004             1 debt_consolidation      0.1008         162.34
4          10005             1      credit_card       0.1426         102.92
5          10006             1      credit_card       0.0788         125.13
6          10007             1 debt_consolidation      0.1496         194.02
7          10008             1      all_other         0.1114         131.22
8          10009             1  home_improvement      0.1134          87.19
9          10010             1 debt_consolidation      0.1221          84.12
```

```
      log_annual_income      dti  fico_score  credit_history_days \
0          11.350407    19.48         737         5639.958333
1          11.082143    14.29         707         2760.000000
2          10.373491    11.63         682         4710.000000
3          11.350407     8.1          712         2699.958333
4          11.299732    14.97         667         4066.000000
5          11.904968    16.98         727         6120.041667
6          10.714418     4          667         3180.041667
7          11.002100    11.08         722         5116.000000
8          11.407565    17.25         682         3989.000000
9          10.203592    10          707         2730.041667
```

```
      revolving_balance revolving_util  inquiries_last_6m delinquencies_2y \
0          28854.0          52.1          0.0          0
1          33623.0          76.7          0.0          0
2          3511.0          25.6          1.0          0
3          33667.0          73.2          1.0          0
4          4740.0          39.5          0.0          1
5          50807.0          51          0.0          0
6          3839.0          76.8          0.0          0
7          24220.0          68.6          0.0          0
8          69909.0          51.1          1.0          0
9          5630.0          23          1.0          0
```

```
      public_records  not_fully_paid
0          0          0
1          0          0
2          0          0
3          0          0
4          0          0
5          0          0
6          1          1
7          0          1
8          0          0
9          0          0
```

9578

1.3.1 Una precisazione sui valori zero

Prima di cercare valori mancanti, chiariamo un punto importante: in questo dataset vi sono variabili dove zero non significa “dato mancante”. Per esempio: - `delinquencies_2y` = 0 significa che il cliente non ha mai pagato in ritardo negli ultimi 2 anni - `public_records` = 0 significa nessun fallimento o pignoramento

Questi zeri contengono informazione utile e non vanno trattati come dati assenti.

```
[116]: customer_id          10031
      credit_policy        1
      purpose              debt_consolidation
      interest_rate        0.0807
      installment          156.84
      log_annual_income    11.512925
      dti                  2.3
      fico_score           742
      credit_history_days   3148.958333
      revolving_balance     9698.0
      revolving_util        19.4
      inquiries_last_6m     0.0
      delinquencies_2y      0
      public_records        0
      not_fully_paid        0
      Name: 30, dtype: object
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 9578 entries, 0 to 9577
```

```
Data columns (total 15 columns):
```

#	Column	Non-Null Count	Dtype
0	customer_id	9578 non-null	int64
1	credit_policy	9578 non-null	object
2	purpose	9578 non-null	object
3	interest_rate	9578 non-null	float64
4	installment	9578 non-null	float64
5	log_annual_income	9573 non-null	float64
6	dti	9578 non-null	object
7	fico_score	9578 non-null	int64
8	credit_history_days	9549 non-null	float64
9	revolving_balance	9577 non-null	float64
10	revolving_util	9516 non-null	object
11	inquiries_last_6m	9548 non-null	float64
12	delinquencies_2y	9549 non-null	object
13	public_records	9549 non-null	object
14	not_fully_paid	9578 non-null	int64

```
dtypes: float64(6), int64(3), object(6)
```

memory usage: 1.1+ MB

```
[118]:
```

	customer_id	interest_rate	installment	log_annual_income	\
count	9578.000000	9578.000000	9578.000000	9573.000000	
mean	14789.500000	0.125529	319.089413	10.931892	
std	2765.074773	0.202225	207.071301	0.614766	
min	10001.000000	0.060000	15.670000	7.547502	
25%	12395.250000	0.103900	163.770000	10.558414	
50%	14789.500000	0.122100	268.950000	10.928238	
75%	17183.750000	0.140700	432.762500	11.289832	
max	19578.000000	14.700000	940.140000	14.528354	

	fico_score	credit_history_days	revolving_balance	inquiries_last_6m	\
count	9578.000000	9549.000000	9.577000e+03	9548.000000	
mean	711.159532	4562.026085	1.691529e+04	1.571743	
std	42.024737	2497.985733	3.375770e+04	2.198151	
min	612.000000	178.958333	0.000000e+00	0.000000	
25%	682.000000	2820.000000	3.187000e+03	0.000000	
50%	707.000000	4139.958333	8.596000e+03	1.000000	
75%	737.000000	5730.000000	1.825200e+04	2.000000	
max	1812.000000	17639.958330	1.207359e+06	33.000000	

	not_fully_paid
count	9578.000000
mean	0.160054
std	0.366676
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000
max	1.000000

Conversione dei tipi di dato e valori mancanti

Ci sono valori mancanti? Come li gestiamo? Di che tipo sono?

Con un dataset di questa dimensione possiamo permetterci di eliminare le righe incomplete senza perdere troppa informazione. Inoltre dopo aver verificato la presenza di valori mancanti, verifichiamo se le variabili hanno coerenza nel tipo di dato

Valori mancanti per colonna:

```
customer_id: 0.00%
credit_policy: 0.00%
purpose: 0.00%
interest_rate: 0.00%
installment: 0.00%
log_annual_income: 0.05%
dti: 0.00%
fico_score: 0.00%
credit_history_days: 0.30%
```

```
revolving_balance: 0.01%
revolving_util: 0.65%
inquiries_last_6m: 0.31%
delinquencies_2y: 0.30%
public_records: 0.30%
not_fully_paid: 0.00%
```

Dimensione dataset dopo rimozione NaN: 9508 righe

Verifica valori mancanti per colonna:

```
customer_id      0
credit_policy     0
purpose          0
interest_rate    0
installment      0
log_annual_income 0
dti              0
fico_score       0
credit_history_days 0
revolving_balance 0
revolving_util    0
inquiries_last_6m 0
delinquencies_2y  0
public_records    0
not_fully_paid    0
dtype: int64
```

Verifica duplicati:

Righe duplicate trovate: 0

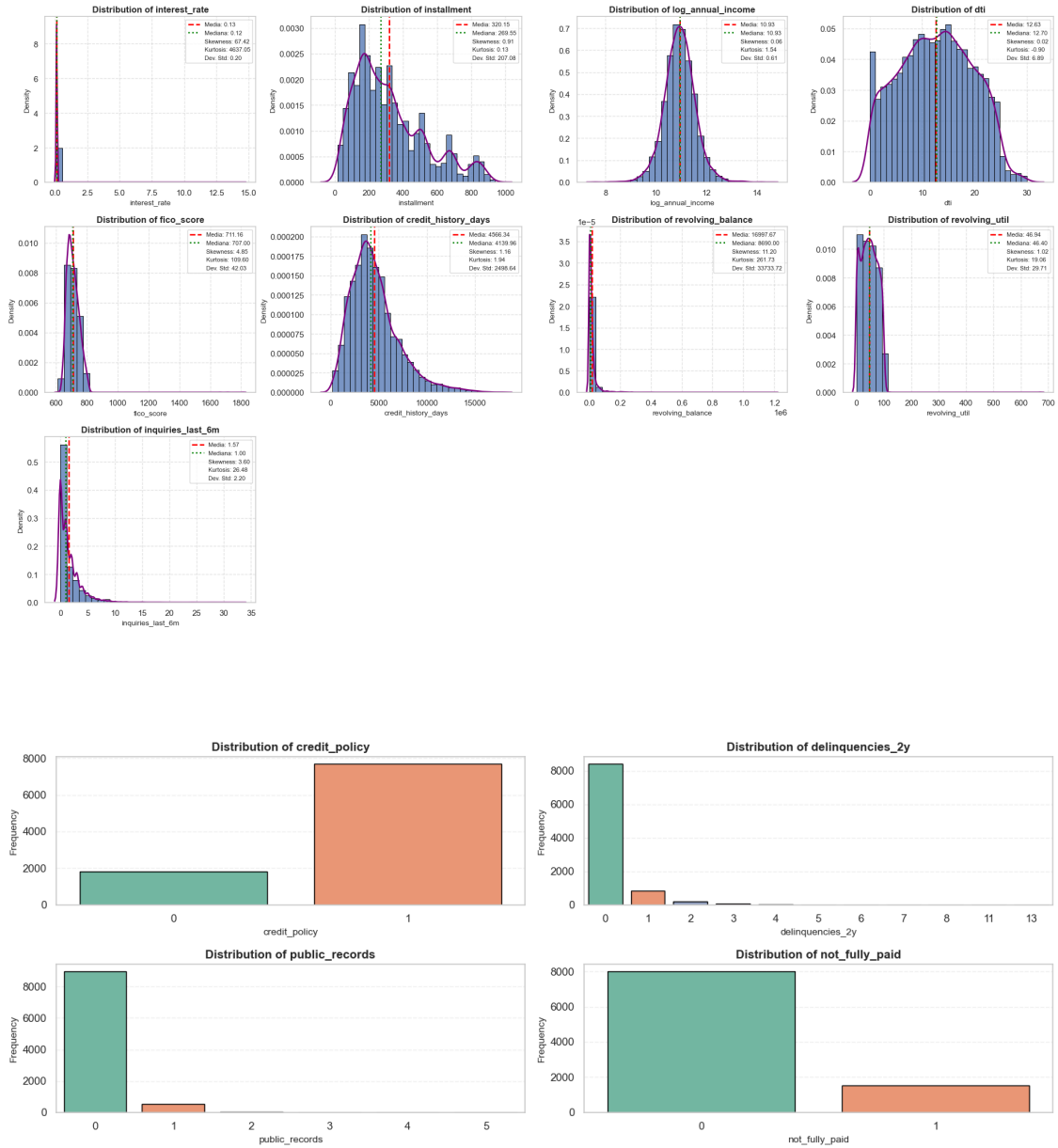
Nessun duplicato presente nel dataset

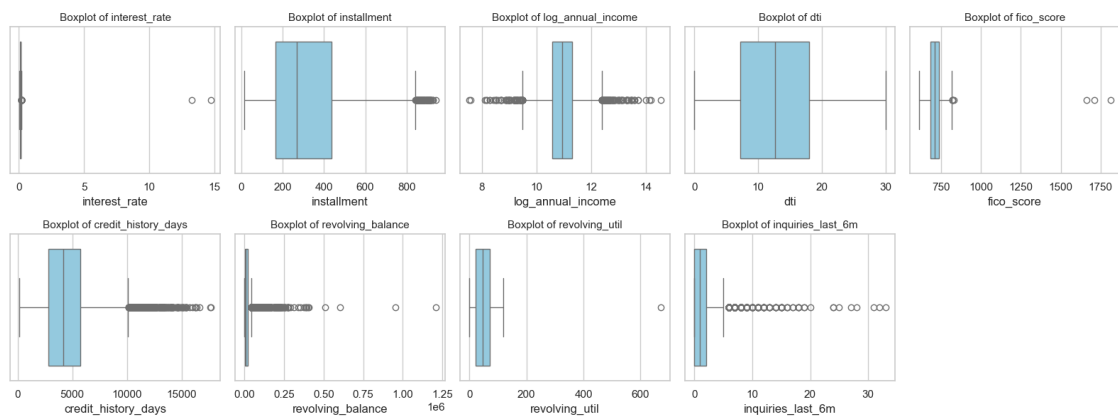
Parte 2 - Analisi Esplorativa

1.4 Come sono distribuiti i dati?

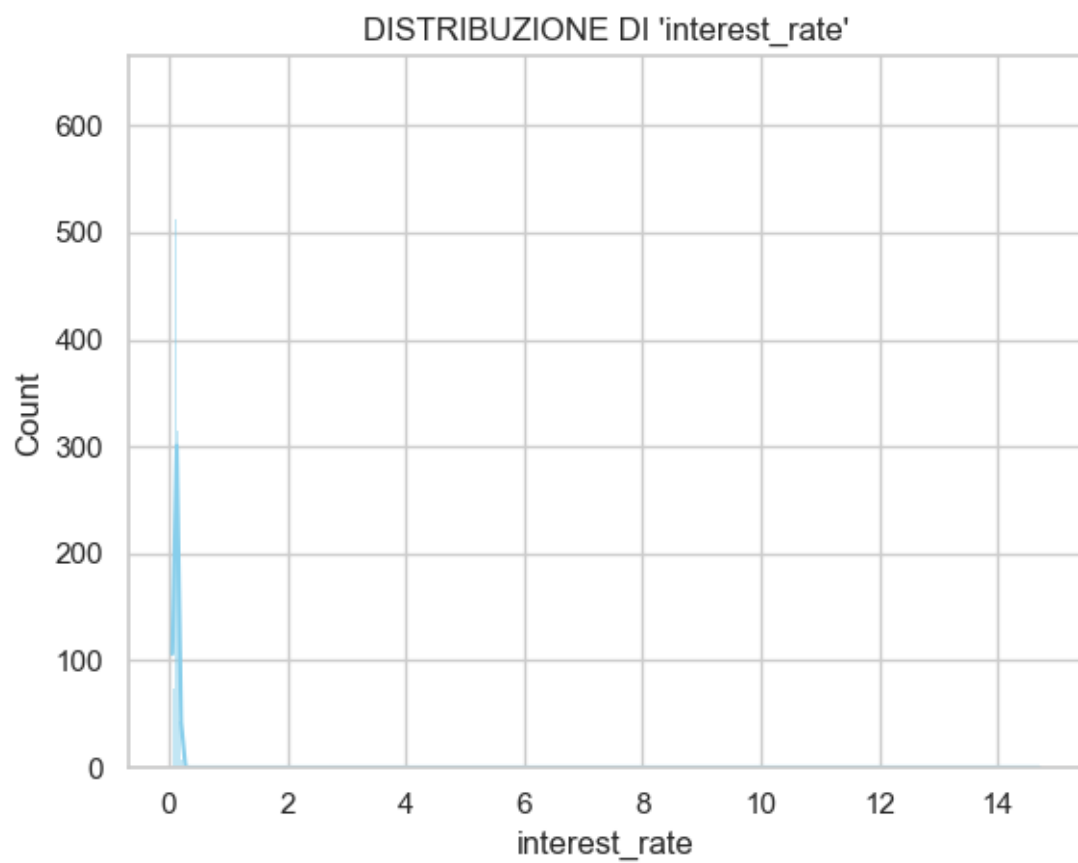
Prima di tutto, dobbiamo conoscere i dati. Guardiamo le distribuzioni per capire: - Dove si concentrano la maggior parte delle osservazioni - Se esistono outlier e se sono significativi - La forma delle distribuzioni

Dividiamo le variabili in due gruppi: - **Continue**: possono assumere qualsiasi valore in un intervallo (interest_rate, installment, dti, fico_score) - **Discrete**: assumono solo valori interi o categorie limitate (credit_policy, delinquencies_2y, not_fully_paid)





Come sono distribuiti i tassi di interesse?

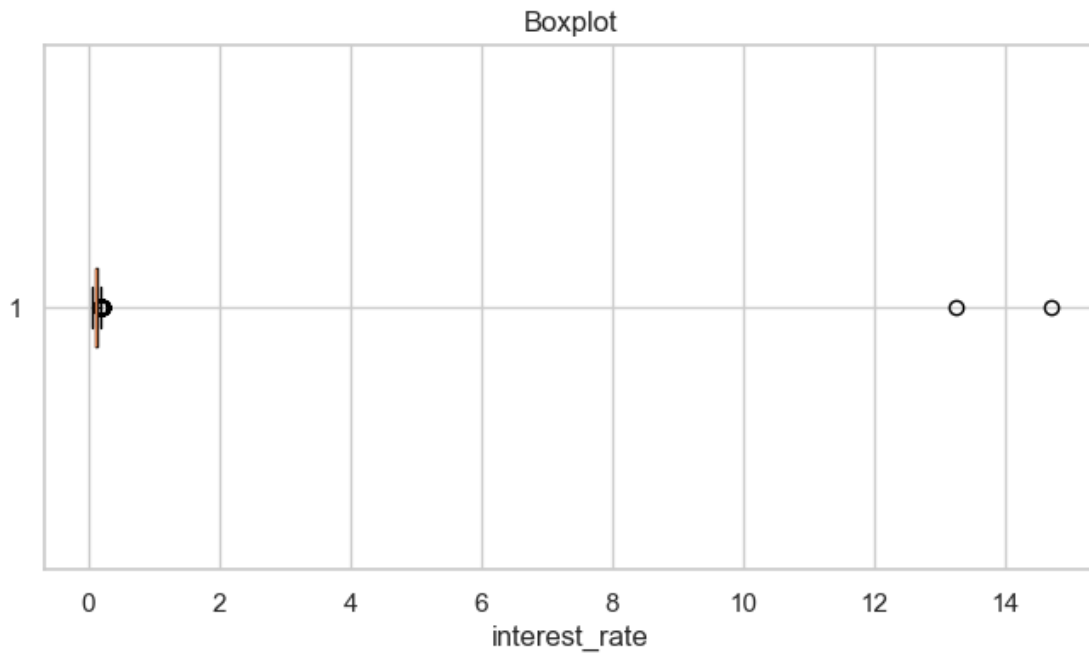


```
count    9508.000000
mean      0.125583
std       0.202949
min       0.060000
```

```

25%      0.103900
50%      0.122100
75%      0.140700
max       14.700000
Name: interest_rate, dtype: float64

```



Notiamo valori molto vicini allo zero e altri che arrivano a 14. Questo suggerisce che i tassi potrebbero essere in formato decimale (0.12 invece di 12%). Verifichiamo e correggiamo il problema.

Conversione in corso: trovati 9506 valori in scala decimale (≤ 1).

Conversione completata: ora tutti i valori sono in scala percentuale (0-100).

Statistiche post-conversione:

```

count      9508.000000
mean       12.267296
std        2.681129
min        6.000000
25%       10.390000
50%       12.210000
75%       14.070000
max       21.640000
Name: interest_rate, dtype: float64
Limite inferiore: 4.8700
Limite superiore: 19.5900

```

I limiti calcolati con il metodo IQR (range interquartile) identificano l'intervallo dei valori “normali”. Tutto ciò che esce da questo intervallo viene considerato outlier.

```
Numero di lower outliers: 0
Numero di upper outliers: 51

NUMERO DI OUTLIERS TROVATI : 51

Esempi di outlier:
2945    20.86
3713    20.11
3991    20.11
4064    19.79
4731    20.17
Name: interest_rate, dtype: float64
```

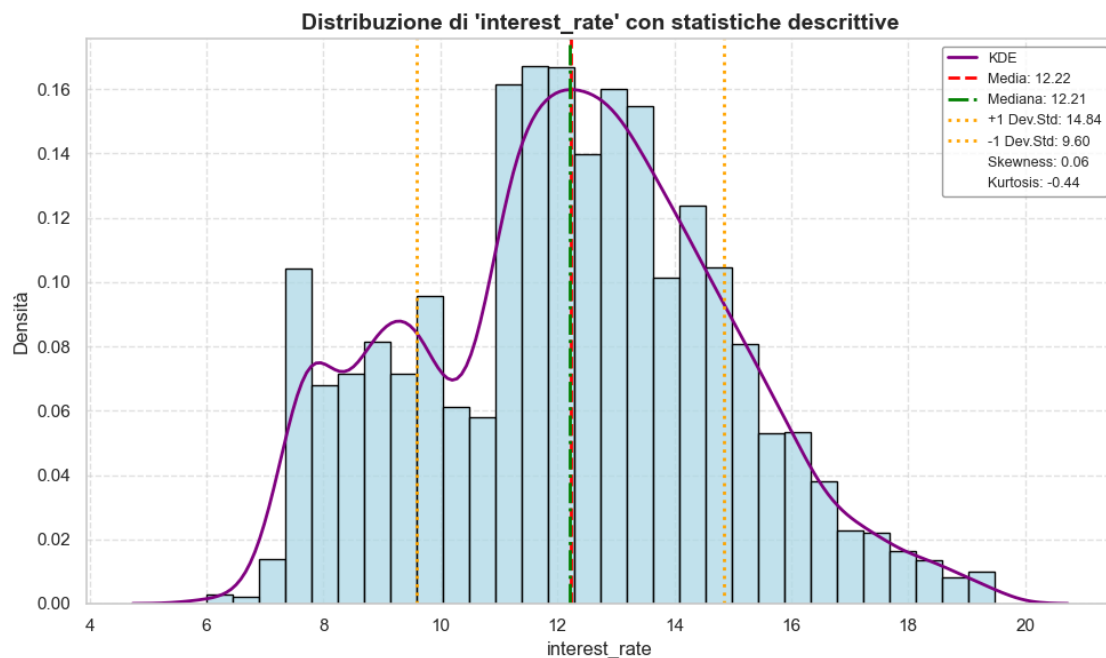
1.4.1 Pulizia degli outlier

Rimuoviamo i valori anomali per ottenere una distribuzione più rappresentativa della realtà dei prestiti.

```
DIMENSIONE DATASET PULITO (SENZA OUTLIERS) : 9457 righe
Percentuale di outliers rimossi: 0.54%
```

1.4.2 Cosa abbiamo trovato?

Visualizziamo la distribuzione pulita insieme alle principali statistiche descrittive: media, mediana, deviazione standard, asimmetria (skewness) e curtosi (kurtosis).



Il tasso di interesse medio è circa 12.5%, con la maggior parte dei prestiti tra 10% e 14%. Tassi sopra il 20% sono rari, sotto il 6% sono eccezioni riservate a clienti con profilo creditizio eccellente.

La distribuzione è approssimativamente simmetrica, il che indica che LendingClub applica tassi abbastanza standardizzati con variazioni legate al profilo di rischio del cliente.

1.4.3 Cosa abbiamo scoperto sui tassi di interesse

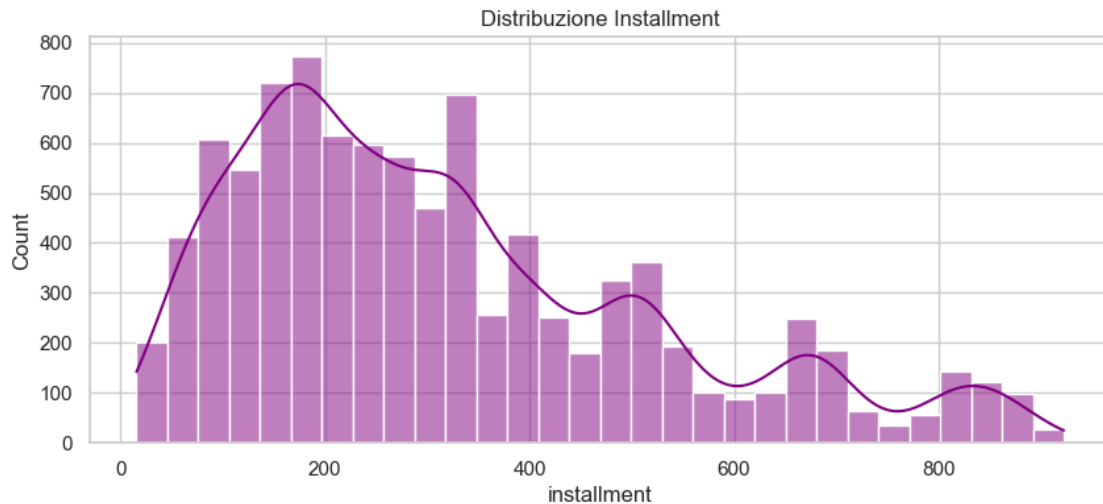
Prima della pulizia: la distribuzione era fortemente sbilanciata a causa di valori estremi, probabilmente errori di inserimento.

Dopo la pulizia: distribuzione simmetrica centrata sul 12.5%. I tassi sono ora omogenei e realistici per un mercato di prestiti.

Questo ci conferma che la piattaforma applica una politica di pricing ragionevolmente uniforme, con aggiustamenti basati sul rischio individuale.

Come sono distribuite le rate ?

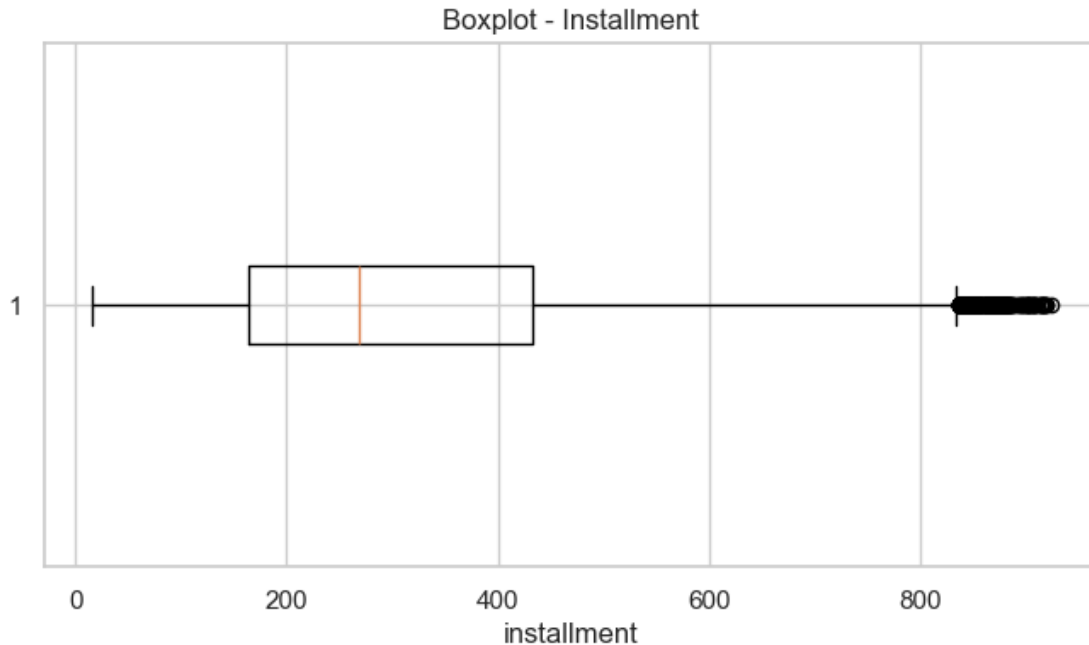
La rata dipende dall'importo del prestito, dalla durata e dal tasso di interesse.



STATISTICHE DESCRITTIVE:

```
count    9457.000000
mean      319.212909
std       206.120064
min        15.670000
25%       164.020000
50%       269.070000
75%       432.260000
max       922.420000
Name: installment, dtype: float64
```

Il boxplot identifica valori che il metodo statistico considera “anomali”.



Calcoliamo i limiti statistici per identificare gli outlier.

Limite Inferiore: -238.34

Limite Superiore: 834.62

Quanti outlier troviamo?

Numero di lower outliers: 0

Numero di upper outliers: 234

Numero totale di outlier : 234

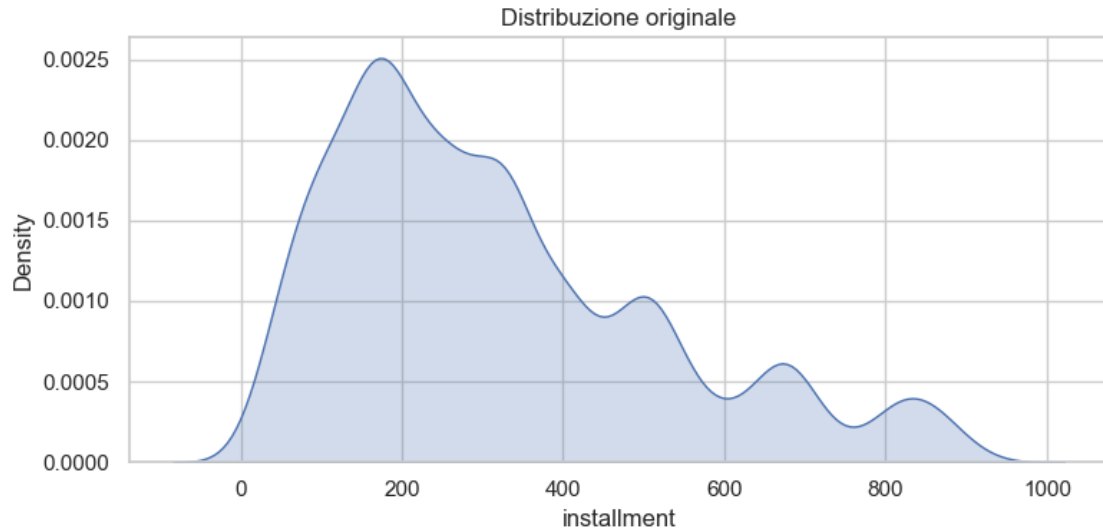
Esempi di outlier:

	installment	dti	customer_id
160	839.95	24.85	10161
237	859.07	20.79	10238
246	836.23	7.99	10247
249	842.47	12.81	10250
271	862.97	5.79	10272

Le rate alte (es. 842 euro) vengono segnalate come outlier dal metodo IQR, ma non sono errori. Rappresentano semplicemente prestiti di importo maggiore. Non le rimuoviamo perché sono dati legittimi, non anomalie.

1.4.4 Forma della distribuzione

Esaminiamo skewness e kurtosis per capire quanto la distribuzione si discosta da una normale.



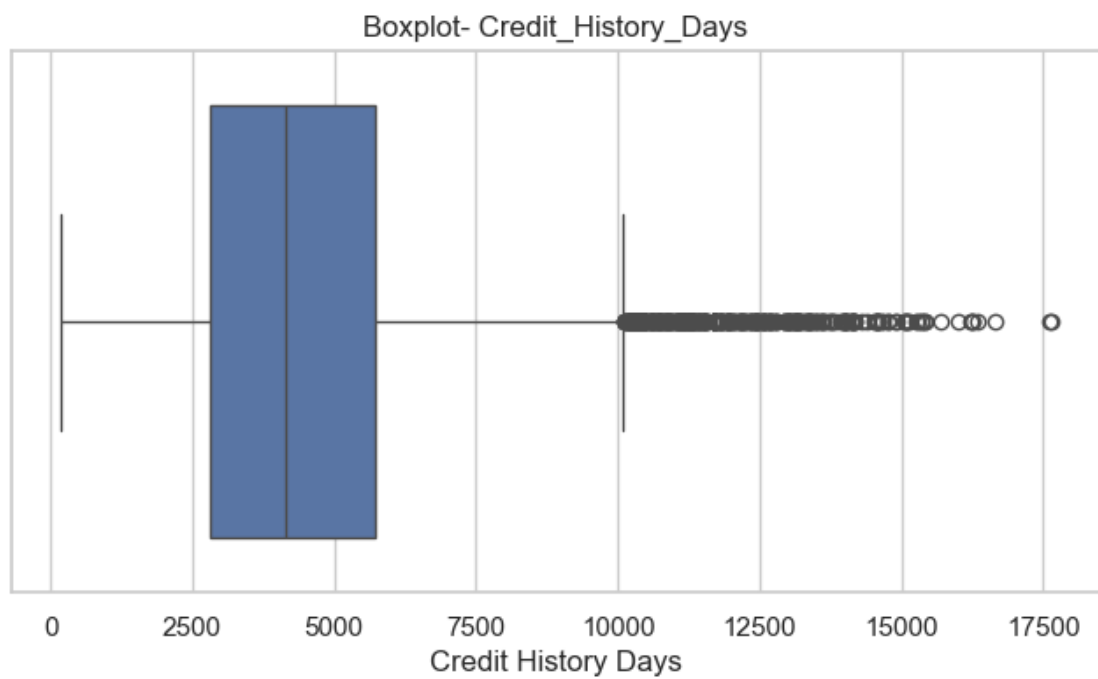
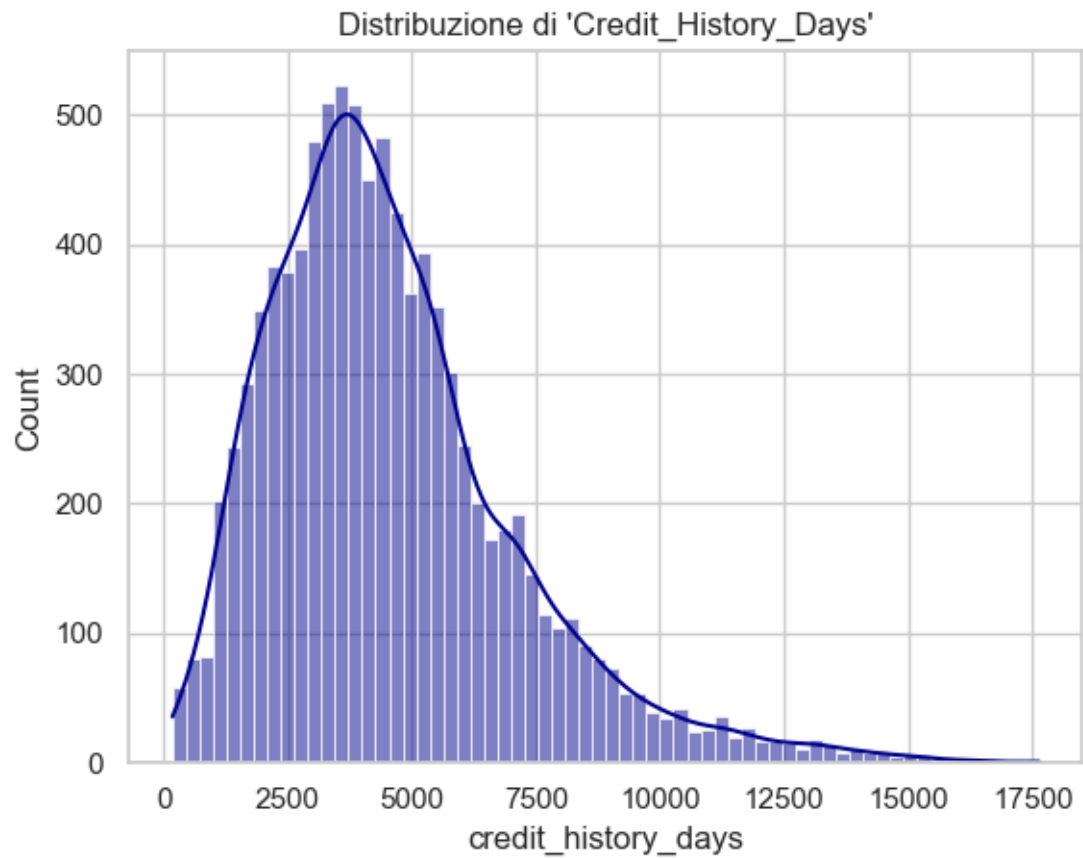
INSTALLMENT

Asimmetria_Skewness: 0.91 | Kurtosis: 0.14

La distribuzione è leggermente sbilanciata verso destra (skewness positiva), confermando che ci sono più rate basse che alte. La curtosi indica una forma simile alla normale ma con code leggermente più pesanti, quindi qualche valore estremo è presente.

L'anzianità della storia creditizia dei clienti

Questa variabile misura da quanti giorni il cliente ha una storia creditizia attiva. In teoria, una storia più lunga è un indice positivo per la banca per valutare l'affidabilità.



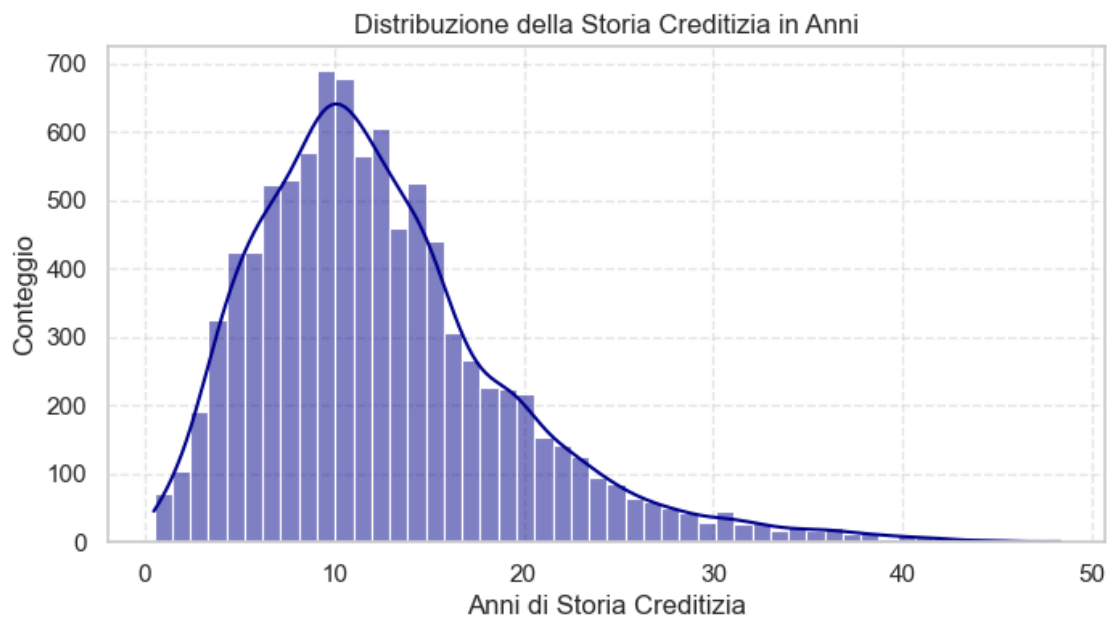
```
count      9457.000000
mean       4569.165191
std        2496.544737
min         180.041667
25%        2820.041667
50%        4139.958333
75%        5730.041667
max        17639.958330
Name: credit_history_days, dtype: float64
```

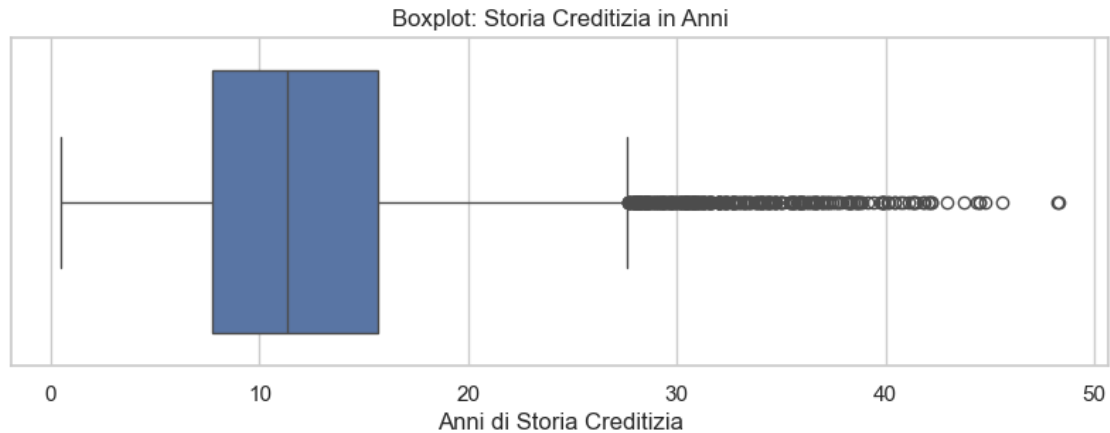
Skewness: 1.1600

Kurtosis: 1.9480

1.4.5 Conversione in anni

I giorni sono poco intuitivi, convertiamoli in anni per una lettura più immediata: - 0-5 anni: clienti giovani o nuovi al credito - 5-20 anni: esperienza normale - 20-35 anni: clienti con lunga esperienza - oltre 45 anni: valore sospetto, da verificare





```
count    9457.000000
mean      12.509693
std        6.835167
min         0.492927
25%        7.720853
50%       11.334588
75%       15.687999
max       48.295574
Name: credit_history_years, dtype: float64
```

Skewness: 1.1600

Kurtosis: 1.9480

Limite Inferiore: -4.2299

Limite Superiore: 27.6387

Il metodo IQR suggerirebbe un limite di circa 28 anni, ma è troppo restrittivo. Nella realtà, 45 anni di storia creditizia sono il massimo plausibile: richiederebbe aver aperto un conto a 18 anni ed essere ora oltre i 60. Usiamo 45 anni come soglia ragionevole.

Numero di outlier sopra i 45 anni: 3

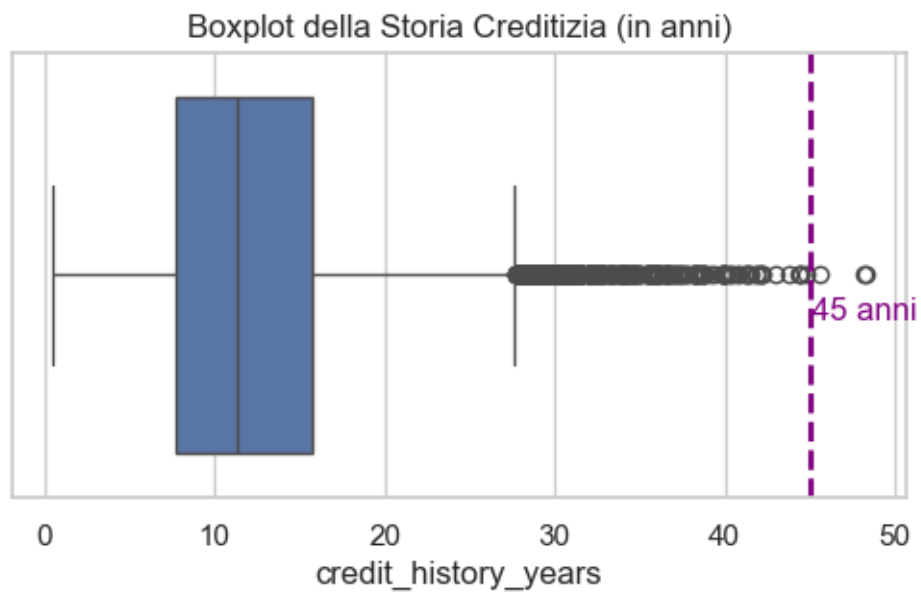
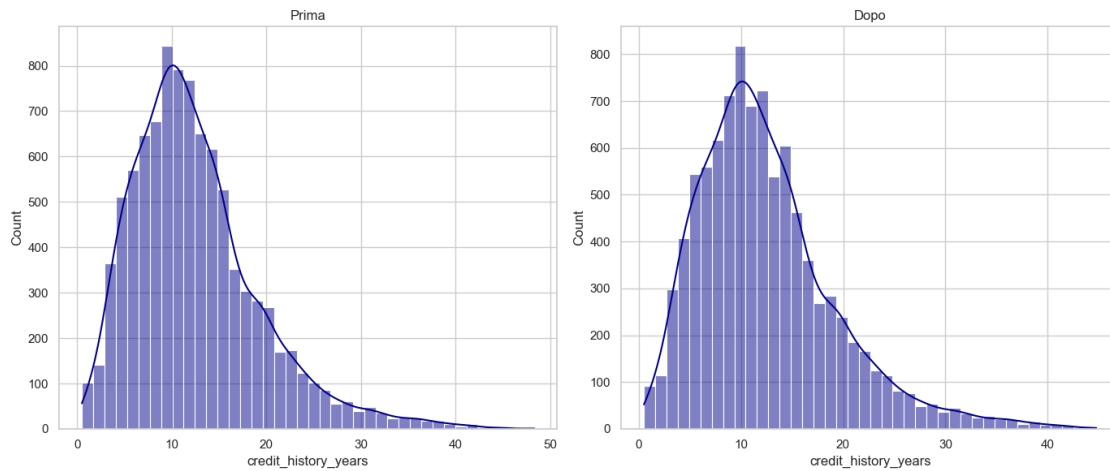
Outliers nel dataset:

5801 45.590691

7553 48.229979

8430 48.295574

Name: credit_history_years, dtype: float64



```
count    9454.000000
mean      12.498630
std       6.807936
min       0.492927
25%       7.720853
50%      11.334588
75%      15.687999
max      44.763860
Name: credit_history_years, dtype: float64
```

Skewness: 1.1365

Kurtosis: 1.8166

Abbiamo rimosso pochissimi record (0.03%). La distribuzione rimane sbilanciata verso destra: la maggior parte dei clienti ha storie creditizie di durata media, mentre solo pochi hanno storie molto lunghe.

Quanto sono affidabili i nostri clienti?

Come sono distribuiti i punteggi FICO? Che tipo di clientela attrae LendingClub?

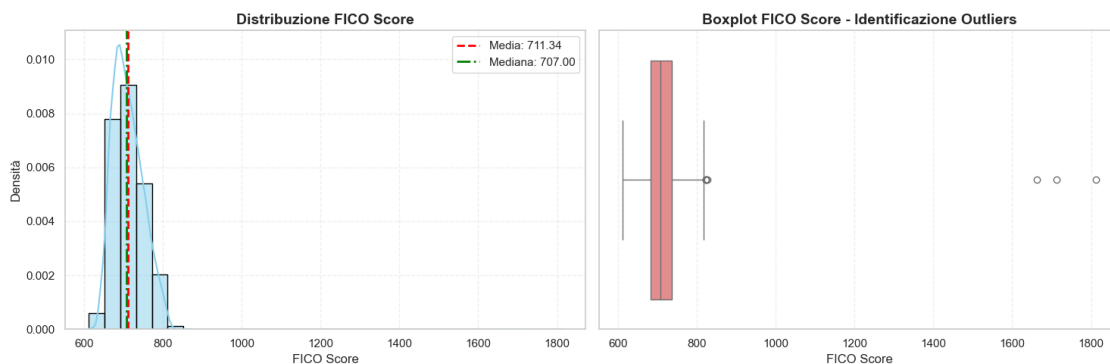
Il FICO Score è il numero che sintetizza l'affidabilità creditizia nel mercato americano. La scala va da 300 a 850: - Sotto 580: Pessimo - 580-669: Discreto - 670-739: Buono - 740-799: Molto Buono - Sopra 800: Eccellente

Questo punteggio influenza direttamente il tasso di interesse applicato e la probabilità di ottenere il prestito.

```
=== STATISTICHE FICO SCORE ===
count      9457.000000
mean        711.336470
std         42.052375
min         612.000000
25%         682.000000
50%         707.000000
75%         737.000000
max         1812.000000
Name: fico_score, dtype: float64
```

Skewness: 4.8639

Kurtosis: 109.9192



Limite inferiore: 599.50

Limite superiore: 851.00

NUMERO DI OUTLIERS TROVATI: 3

Percentuale di outliers: 0.03%

Valore massimo FICO Score: 1812

=====

PULIZIA FICO SCORE

=====

Dimensione dataset PRIMA della pulizia: 9457 righe

Dimensione dataset DOPO la pulizia: 9454 righe

Righe rimosse: 3

Percentuale rimossa: 0.03%

=====

STATISTICHE FICO SCORE DOPO PULIZIA

=====

count 9454.000000

mean 711.013645

std 37.936399

min 612.000000

25% 682.000000

50% 707.000000

75% 737.000000

max 827.000000

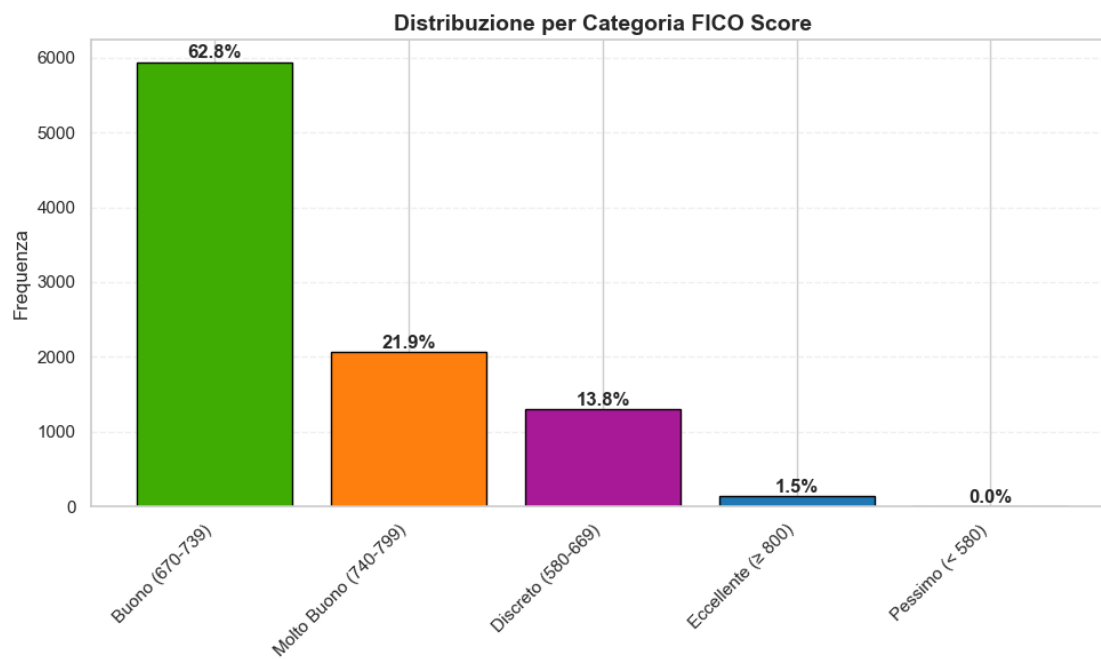
Name: fico_score, dtype: float64

Skewness: 0.4616

Kurtosis: -0.4306

=====

827



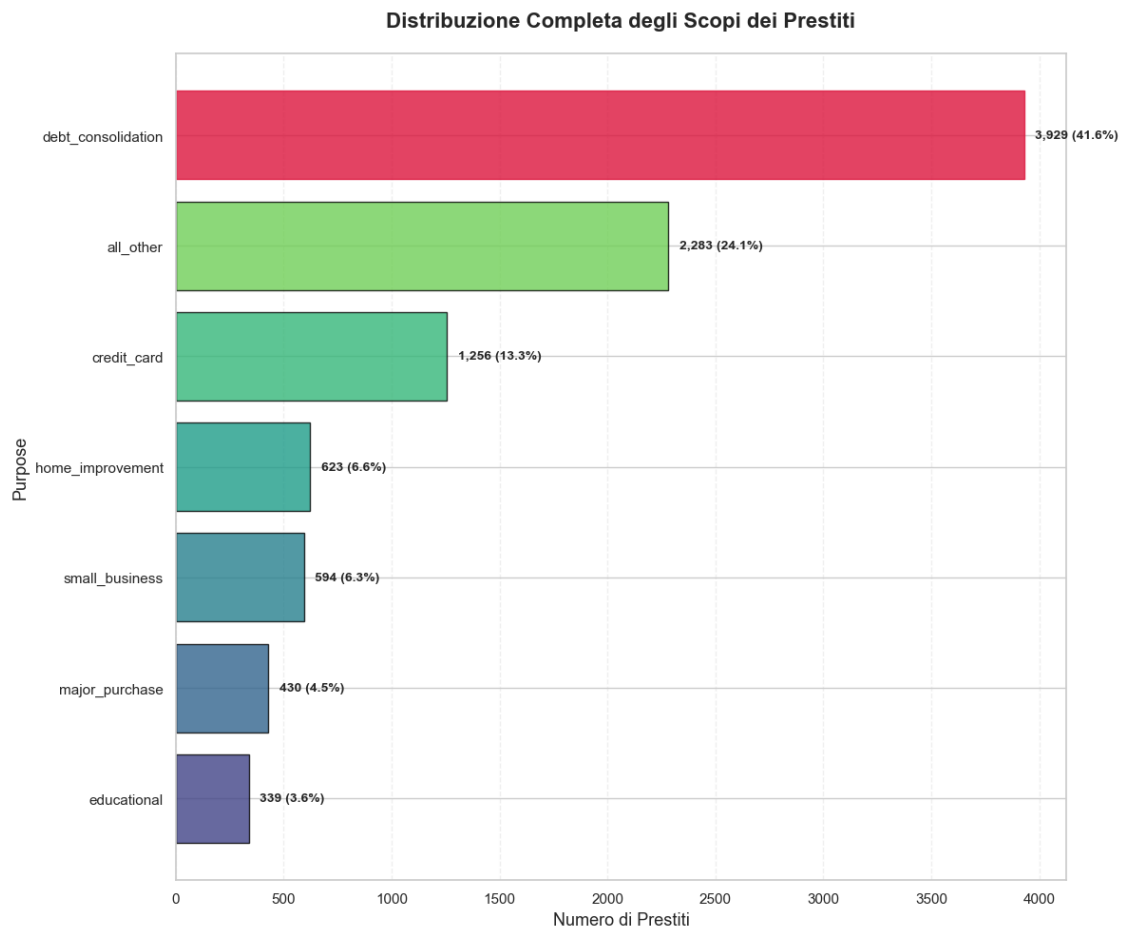
=== DISTRIBUZIONE PER CATEGORIA ===

Buono (670-739): 5936 (62.79%)
 Molto Buono (740-799): 2070 (21.90%)
 Discreto (580-669): 1308 (13.84%)
 Eccellente (800): 140 (1.48%)
 Pessimo (< 580): 0 (0.00%)

Perche i clienti chiedono un prestito?

Quali sono le motivazioni piu comuni?

La variabile **purpose** indica lo scopo dichiarato del prestito.



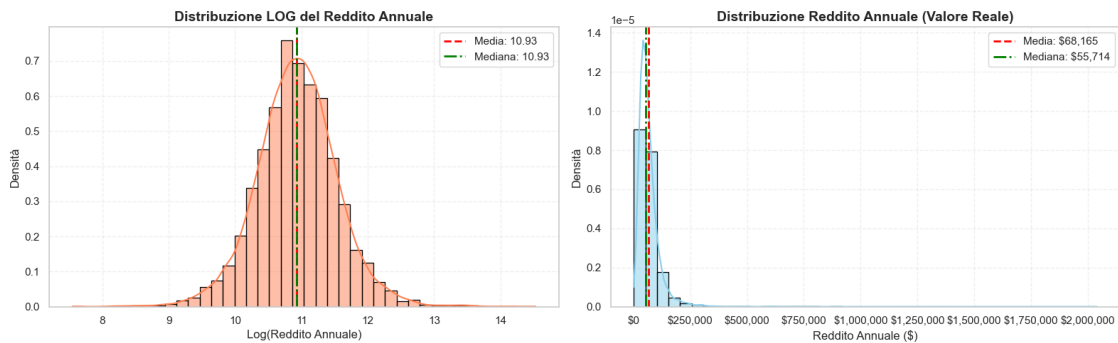
STATISTICHE PURPOSE		
Purpose	Count	Percentage

debt_consolidation	3,929	41.56%
all_other	2,283	24.15%
credit_card	1,256	13.29%
home_improvement	623	6.59%
small_business	594	6.28%
major_purchase	430	4.55%
educational	339	3.59%
=====		
TOTALE	9,454	100.00%
=====		

Numero totale di categorie: 7
 Categoria più frequente: debt_consolidation (3,929 prestiti)
 Categoria meno frequente: educational (339 prestiti)

Qual è la distribuzione dei redditi?

Il reddito è salvato come logaritmo naturale (log_annual_income). Lo riconvertiamo per vedere i valori reali in dollari.



Skewness (Log): 0.0544
 Skewness (Reale): 9.8382
 min 1896.0000003508226
 max 2039784.004362027
 mean 68164.73258872403
 std 60664.94047772514

2 Annual income

Abbiamo creato una nuova variabile annual_income per visualizzare i dati reali del reddito dei clienti, come si può facilmente notare dal grafico, ci sono tanti outliers, ma la loro esistenza è significativa essendo che si tratta del reddito

Il rapporto debito/reddito (DTI)

Il DTI (Debt-to-Income) misura quanta parte del reddito mensile va a pagare debiti esistenti.

Formula: $DTI = (\text{Debiti Mensili} / \text{Reddito Mensile}) \times 100$

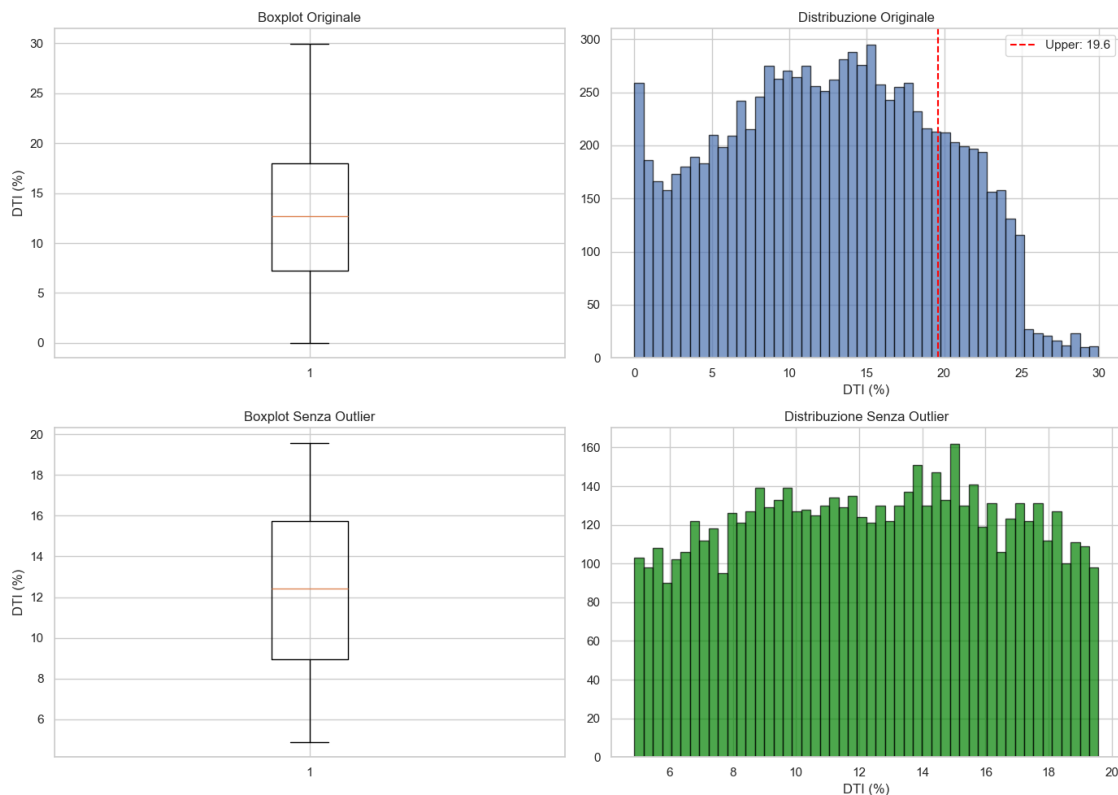
Un DTI alto come per esempio il 40%, significa che il 40% dello stipendio andrà verso le rate.

CONFRONTO STATISTICHE DTI

Metrica	Con Outliers	Senza Outliers	Variazione (%)
Count	9454.000000	9454.000000	0.0
Media	12.635253	12.635253	0.0
Mediana	12.710000	12.710000	0.0
Dev. Std	6.887045	6.887045	0.0
Min	0.000000	0.000000	NaN
Q1	7.232500	7.232500	0.0
Q3	18.000000	18.000000	0.0
Max	29.960000	29.960000	0.0
Skewness	0.020939	0.020939	0.0
Kurtosis	-0.902210	-0.902210	-0.0

3 Outliers DTI

Come si può vedere dalle statistiche, data_clean e data originale hanno variazione NaN, essendo identici e dimostrando l'assenza totale di outliers in DTI



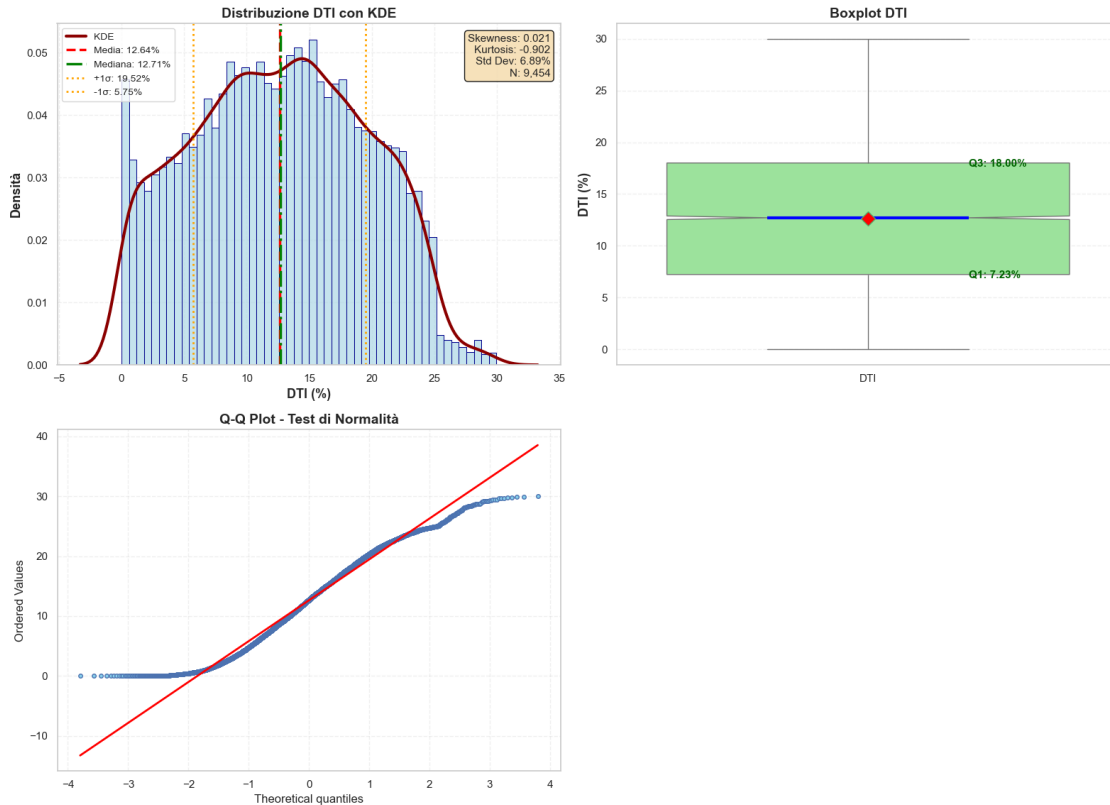
Confronto:

Prima: 9454 righe, Media DTI: 12.64%

Dopo: 6154 righe, Media DTI: 12.34%

3.0.1 Come è distribuito il DTI nel nostro dataset?

Analisi Completa Distribuzione DTI



STATISTICHE DESCRITTIVE DTI

Media:	12.64%
Mediana:	12.71%
Dev. Standard:	6.89%
Skewness:	0.021
Kurtosis:	-0.902
DTI = 0:	85 (0.8991%)

3.0.2 Quanti clienti hanno DTI pari a zero?

Un DTI uguale a zero significa che il cliente non ha debiti mensili. Se la percentuale fosse troppo alta, potrebbe indicare un problema nei dati.

Clienti con DTI = 0: 85

Percentuale: 0.90%

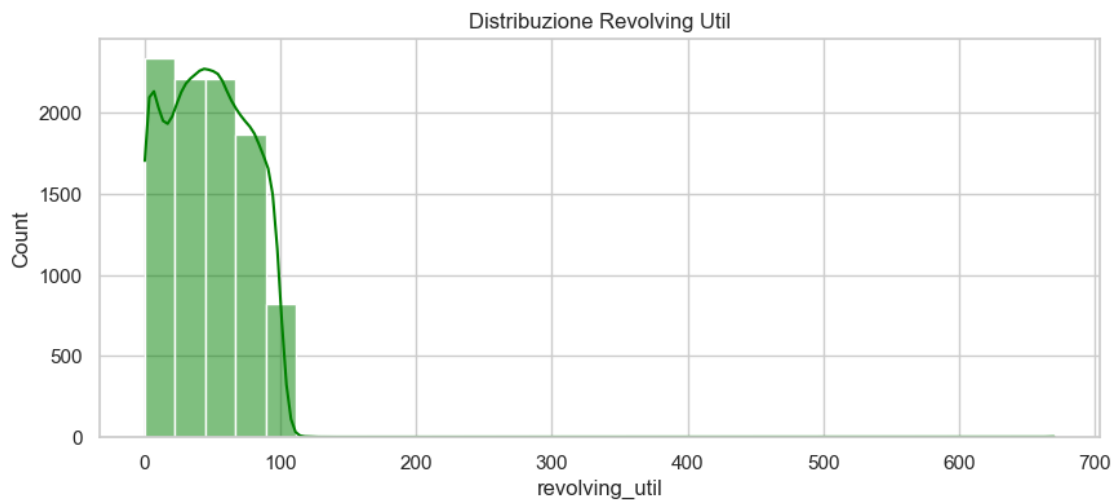
Quanto credito stanno usando i clienti?

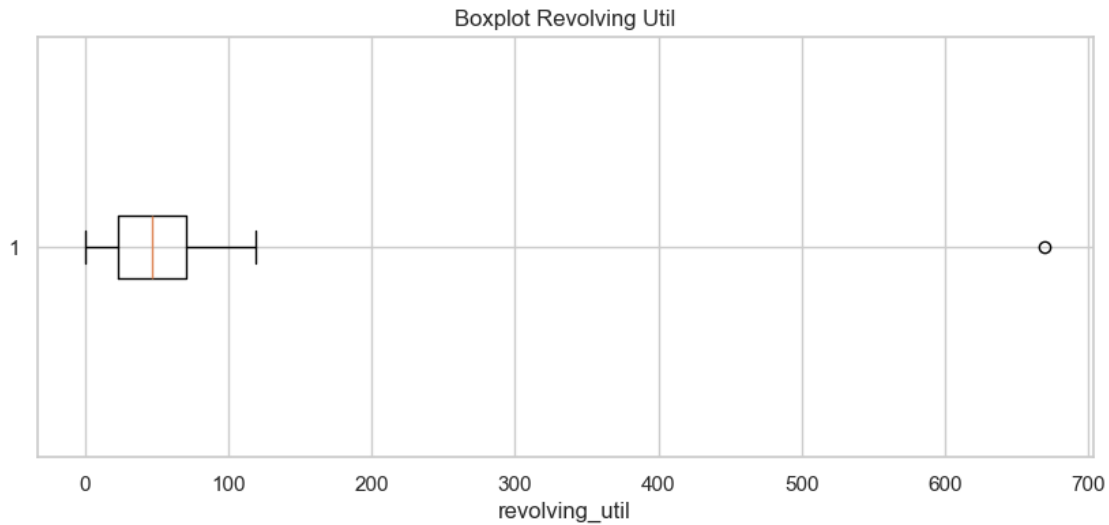
Il revolving balance è il debito effettivo su carte di credito e linee di credito. Il revolving utilization indica che percentuale del credito disponibile il cliente sta effettivamente usando.

Formula: $\text{Revolving Util} = (\text{Saldo} / \text{Limite di Credito}) \times 100$

Un utilizzo alto (es. 80%) indica che il cliente sta usando quasi tutto il credito disponibile, segnale di potenziale stress finanziario.

3.0.3 Distribuzione del Revolving Utilization



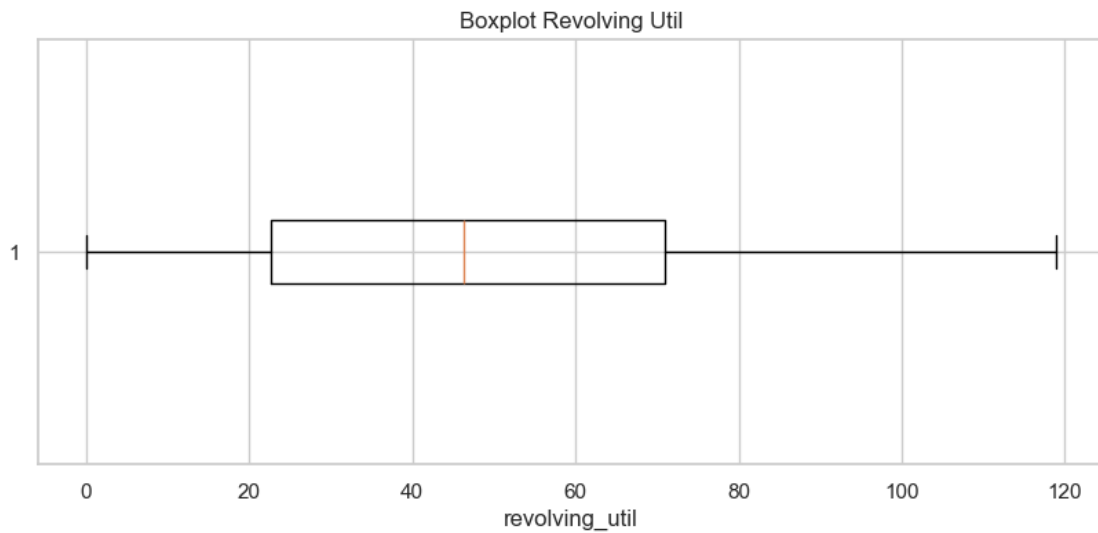
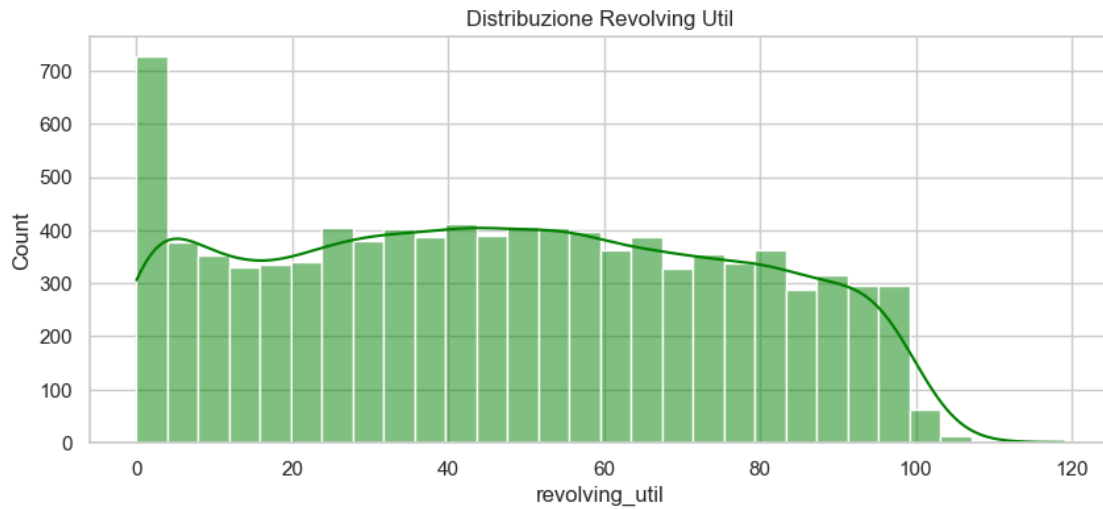


STATISTICHE DESCRITTIVE:

```
count    9454.000000
mean      46.887220
std       29.686746
min        0.000000
25%       22.700000
50%       46.300000
75%       70.900000
max       670.000000
Name: revolving_util, dtype: float64
```

Il valore massimo di 670% è chiaramente un errore. L'utilizzo del credito non dovrebbe mai superare il 100% se non in casi eccezionali di superamento del limite. Questo outlier va rimosso.

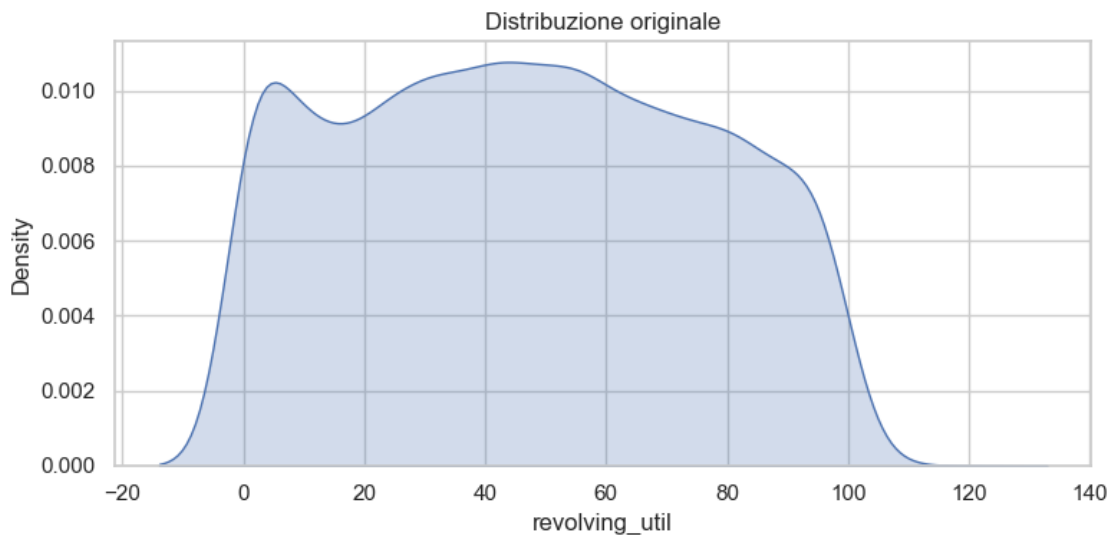
```
Limite Inferiore: -49.60
Limite Superiore: 143.20
Numero di lower outliers: 0
Numero di upper outliers: 1
DIMENSIONE DATASET PULITO (SENZA OUTLIERS) : 9453 righe
```



STATISTICHE DESCRITTIVE:

```
count    9453.000000
mean      46.821303
std       28.988166
min        0.000000
25%       22.700000
50%       46.300000
75%       70.900000
max       119.000000
Name: revolving_util, dtype: float64
```

Dopo la pulizia, il valore massimo di 119% è molto più ragionevole. Indica un cliente che ha superato leggermente il proprio limite di credito, situazione problematica ma plausibile.



REVOLVING_UTIL

Asimmetria_Skewness: 0.06 | Kurtosis: -1.12

0.0

3.0.4 Analisi della Credit Policy

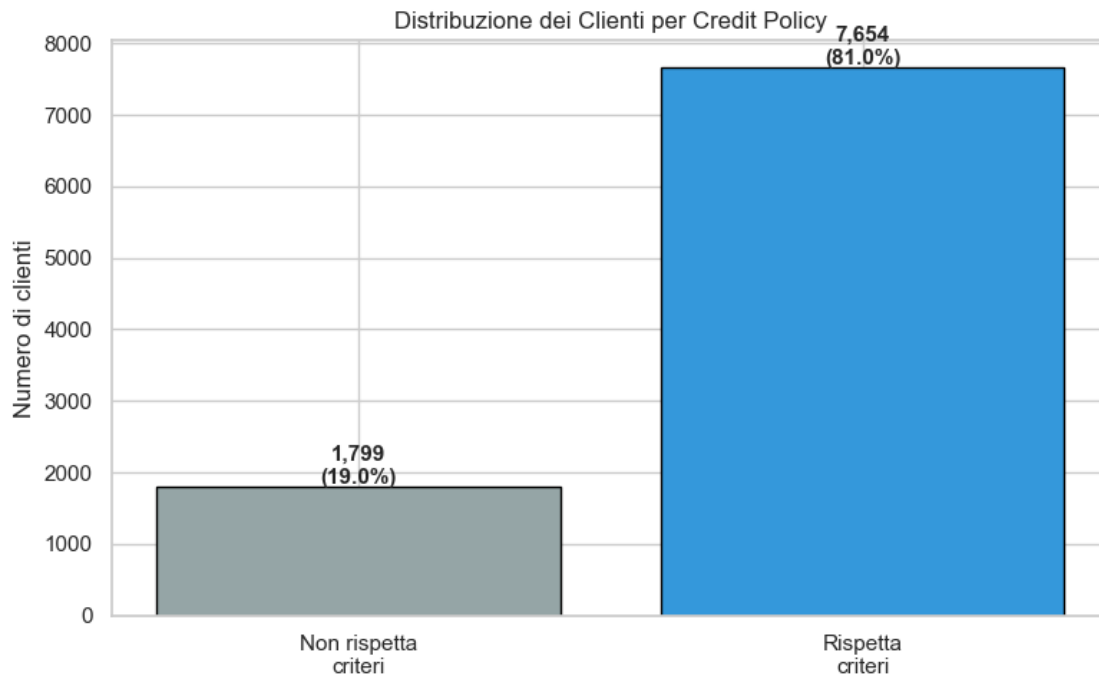
Una domanda interessante riguarda i clienti che non rispettano i criteri di credito stabiliti da LendingClub. La variabile `credit_policy` indica se il cliente soddisfa (1) o meno (0) i requisiti di sottoscrizione.

Distribuzione per Credit Policy:

=====

NON rispetta criteri: 1799 clienti (19.0%)

Rispetta criteri: 7654 clienti (81.0%)



3.0.5 Analisi dei Precedenti Negativi (Public Records)

La variabile `public_records` indica il numero di eventi negativi nel passato creditizio del cliente, come fallimenti, pignoramenti o sentenze sfavorevoli. La domanda è semplice ma fondamentale: chi ha già avuto problemi di credito in passato è più propenso a ripeterli?

Distribuzione dei Public Records:

`public_records`

0 8904

1 523

2 19

3 5

4 1

5 1

Name: count, dtype: Int64

Distribuzione dei clienti per numero di Public Records:

=====

0 precedenti negativi: 8904 clienti (94.2%)

1 precedenti negativi: 523 clienti (5.5%)

2 precedenti negativi: 19 clienti (0.2%)

3 precedenti negativi: 5 clienti (0.1%)

4 precedenti negativi: 1 clienti (0.0%)

5 precedenti negativi: 1 clienti (0.0%)

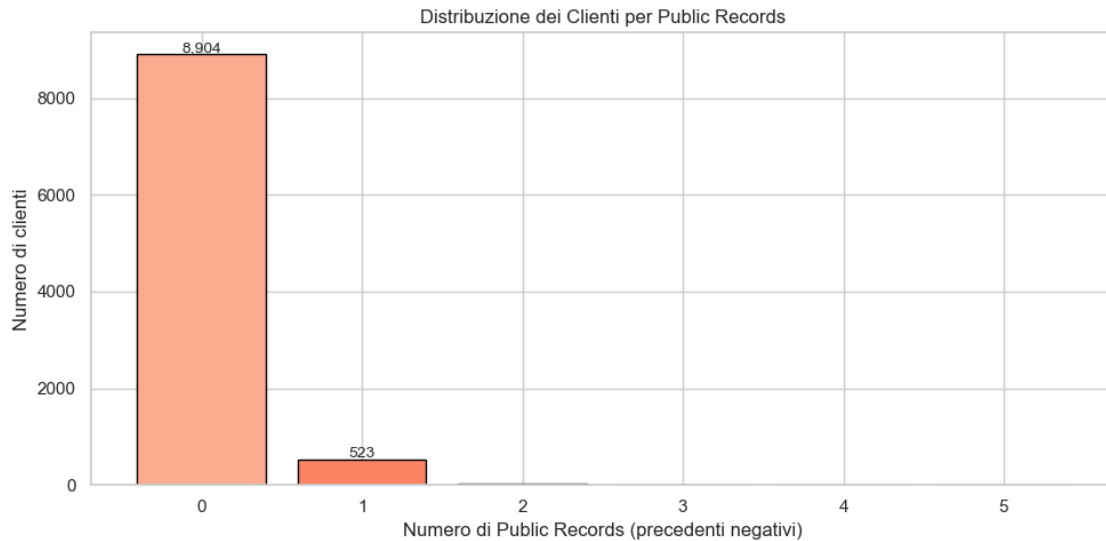
Statistiche descrittive:

Media: 0.06

Mediana: 0

Max: 5

Clienti senza precedenti negativi: 8904 (94.2%)



Interpretazione:

La grande maggioranza dei clienti (94.2%) non ha precedenti negativi.

La distribuzione è fortemente asimmetrica verso destra (right-skewed).

3.0.6 Analisi dei Ritardi nei Pagamenti (Delinquencies)

Oltre ai precedenti “gravi” come i fallimenti, analizziamo i ritardi nei pagamenti degli ultimi 2 anni.

La variabile `delinquencies_2y` cattura comportamenti meno gravi ma comunque rivelatori: un cliente che e' stato in ritardo con altri debiti potrebbe avere abitudini di pagamento poco affidabili.

Distribuzione delle `delinquencies` (ritardi di pagamento):

`delinquencies_2y`

0 8353

1 819

2 188

3 63

4 18

5 6

6 2

7 1

8 1

11 1

13 1

Name: count, dtype: Int64

Distribuzione dei clienti per numero di ritardi negli ultimi 2 anni:

=====

0 ritardi: 8353 clienti (88.4%)

1 ritardi: 819 clienti (8.7%)

2 ritardi: 188 clienti (2.0%)

3+ ritardi: 93 clienti (1.0%)

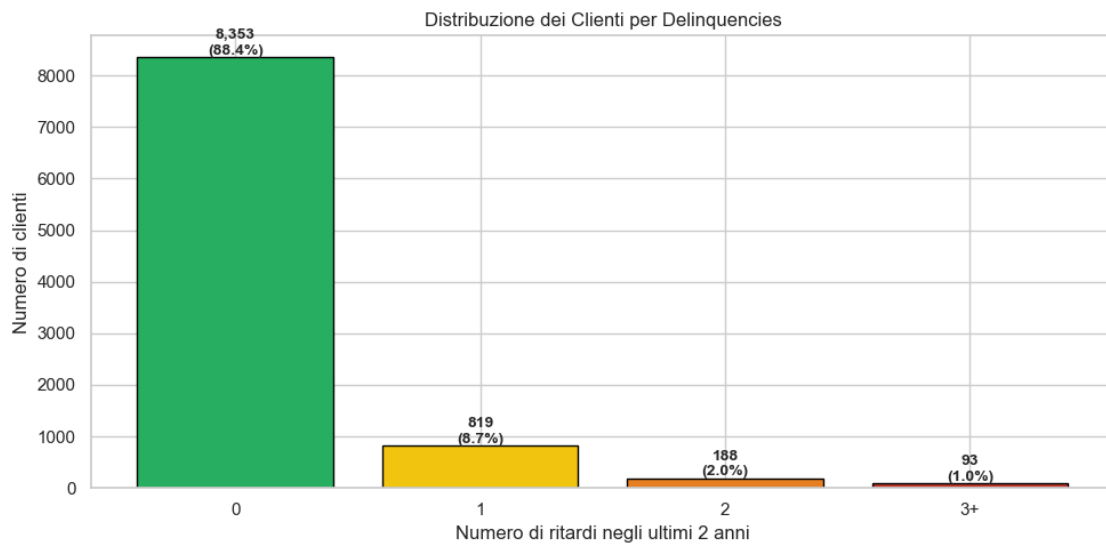
Statistiche descrittive:

Media: 0.16

Mediana: 0

Max: 13

Clienti senza ritardi: 8353 (88.4%)



Interpretazione:

La maggioranza dei clienti (88.4%) non ha avuto ritardi negli ultimi 2 anni.

La distribuzione è fortemente asimmetrica verso destra (right-skewed).

4 Parte 3 - Verifica Statistica

Prima di procedere con l'analisi multivariata, applichiamo test statistici per validare le osservazioni fatte finora. I test ci permettono di capire se le differenze osservate sono significative o semplicemente dovute al caso.

In questa sezione applichiamo: - **Test di normalita**: per verificare se le variabili seguono una distribuzione normale - **T-test**: per confrontare le medie tra chi ha ripagato e chi no - **Test Chi-quadrato**: per verificare associazioni tra variabili categoriche

4.1 I dati sono normali?

Per molti test statistici serve che i dati seguano una distribuzione normale. Usiamo il test di Shapiro-Wilk per verificarlo. Se il p-value è minore di 0.05, rifiutiamo l'ipotesi di normalità.

TEST DI NORMALITA' (Shapiro-Wilk)

Variabile	Statistica W	P-value	Normale?
interest_rate	0.9876	1.5499e-20	No
fico_score	0.9735	1.8242e-29	No
dti	0.9788	1.0578e-26	No
log_annual_income	0.9881	4.2805e-20	No
installment	0.9251	1.7865e-44	No

Risposta: Nessuna delle variabili segue una distribuzione perfettamente normale. Tuttavia, con campioni grandi come il nostro, il Teorema del Limite Centrale garantisce che le medie campionarie sono comunque approssimativamente normali. Possiamo procedere con i test parametrici.

4.2 Chi non ripaga e diverso da chi ripaga?

Domanda: Esistono differenze significative nelle caratteristiche dei clienti che hanno ripagato rispetto a quelli che non l'hanno fatto?

Usiamo il t-test per campioni indipendenti. L'ipotesi nulla è che le medie dei due gruppi siano uguali. Se il p-value è minore di 0.05, la differenza è statisticamente significativa.

Clienti che hanno ripagato: 7953

Clienti che NON hanno ripagato: 1500

T-TEST: CONFRONTO TRA GRUPPI

Variabile	Media Ripagato	Media Non Rip.	t-stat	p-value
Signif.?				
interest_rate	12.05	13.17	-15.42	4.9850e-53
Si ***				
fico_score	713.46	698.08	14.56	1.6262e-47
Si ***				
dti	12.52	13.24	-3.71	2.1109e-04
Si ***				
log_annual_income	10.94	10.89	2.92	3.4582e-03
Si **				
installment	314.76	342.85	-4.85	1.2683e-06
Si ***				
revolving_util	45.79	52.26	-7.95	2.0037e-15
Si ***				

Legenda: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$

Risposta: I risultati confermano le intuizioni dell'analisi esplorativa: - Il tasso di interesse e il FICO score mostrano differenze molto significative tra i due gruppi - Chi non ripaga ha tassi più alti e punteggi FICO più bassi - Il DTI è leggermente più alto per chi non ripaga - Il reddito, sorprendentemente, non sembra essere un fattore discriminante forte

Queste differenze sono statisticamente significative e ci indicano quali variabili sono più utili per prevedere il default.

4.3 Lo scopo del prestito influenza il rischio?

Domanda: Esiste un'associazione tra la motivazione del prestito e la probabilità di default?

Il test chi-quadrato verifica se due variabili categoriche sono associate. L'ipotesi nulla è che siano indipendenti.

TABELLA DI CONTINGENZA: Purpose vs Not Fully Paid

not_fully_paid	0	1
purpose		
all_other	1907	376
credit_card	1111	145
debt_consolidation	3333	595
educational	270	69
home_improvement	519	104
major_purchase	381	49
small_business	432	162

TEST CHI-QUADRATO

=====

Chi-quadrato: 89.4754

Gradi di libertà: 6

P-value: 3.8939e-17

V di Cramer: 0.0973

=====

Interpretazione V di Cramer:

0.00-0.10: Associazione trascurabile

0.10-0.20: Associazione debole

0.20-0.40: Associazione moderata

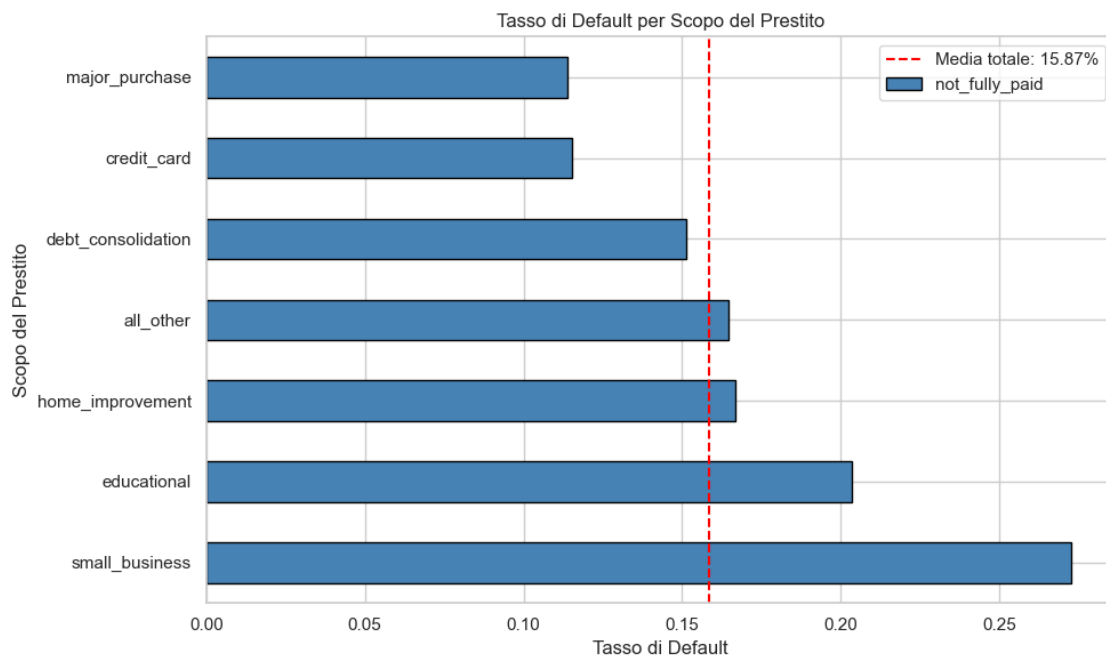
0.40-0.60: Associazione relativamente forte

0.60-1.00: Associazione forte

Conclusione: Esiste un'associazione significativa ($p < 0.05$)

tra lo scopo del prestito e il mancato rimborso.

La forza dell'associazione è 0.097 (V di Cramer).



Tasso di default per scopo:

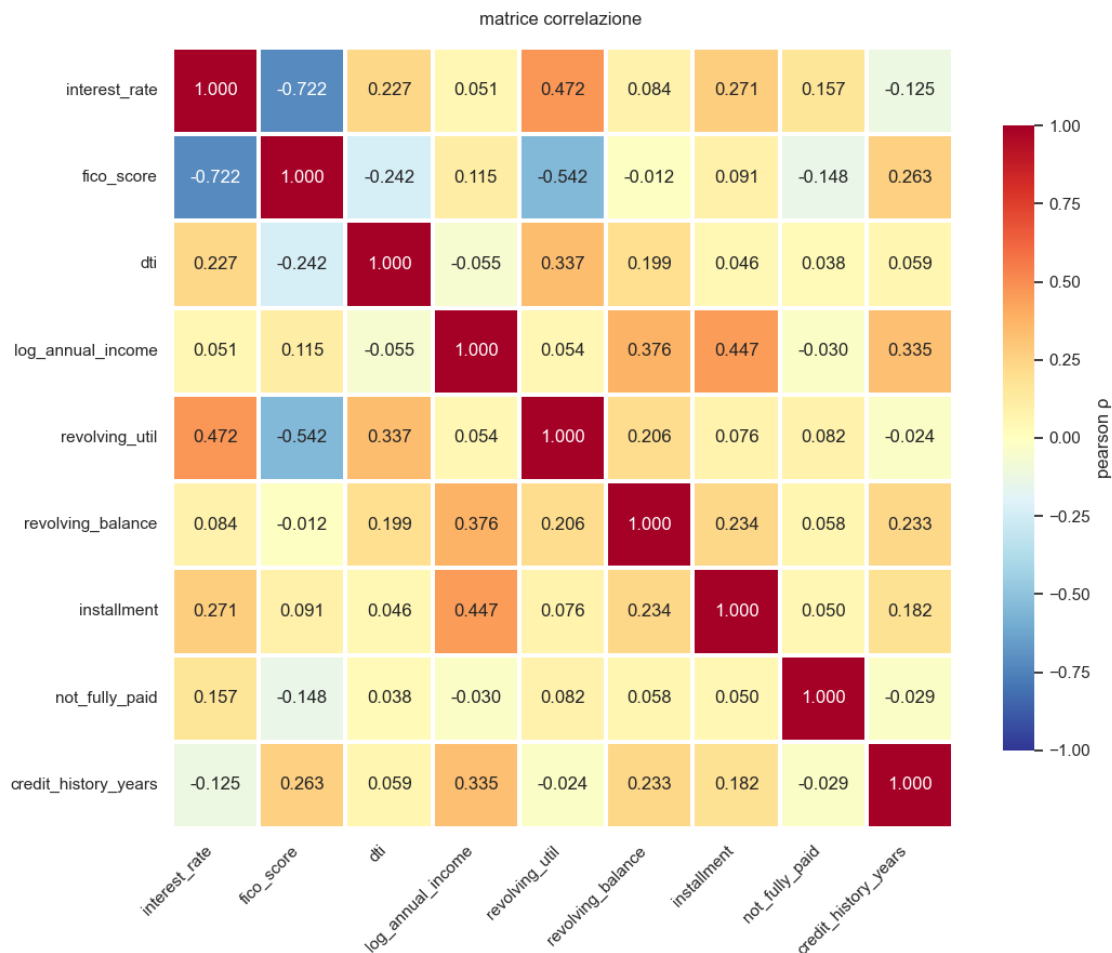
small_business: 27.27%
 educational: 20.35%
 home_improvement: 16.69%
 all_other: 16.47%
 debt_consolidation: 15.15%
 credit_card: 11.54%
 major_purchase: 11.40%

Risposta: Il grafico mostra chiaramente che alcuni scopi sono più rischiosi di altri. I prestiti per piccole imprese hanno il tasso di default più alto, mentre quelli per matrimoni o carte di credito tendono ad essere più sicuri. Questa informazione è utile per la valutazione del rischio.

5 Parte 4 - Analisi Multivariata

Le variabili non agiscono in isolamento. In questa sezione studiamo le relazioni tra di esse usando correlazioni, covarianze e modelli di regressione.

6 Matrice di correlazione generale



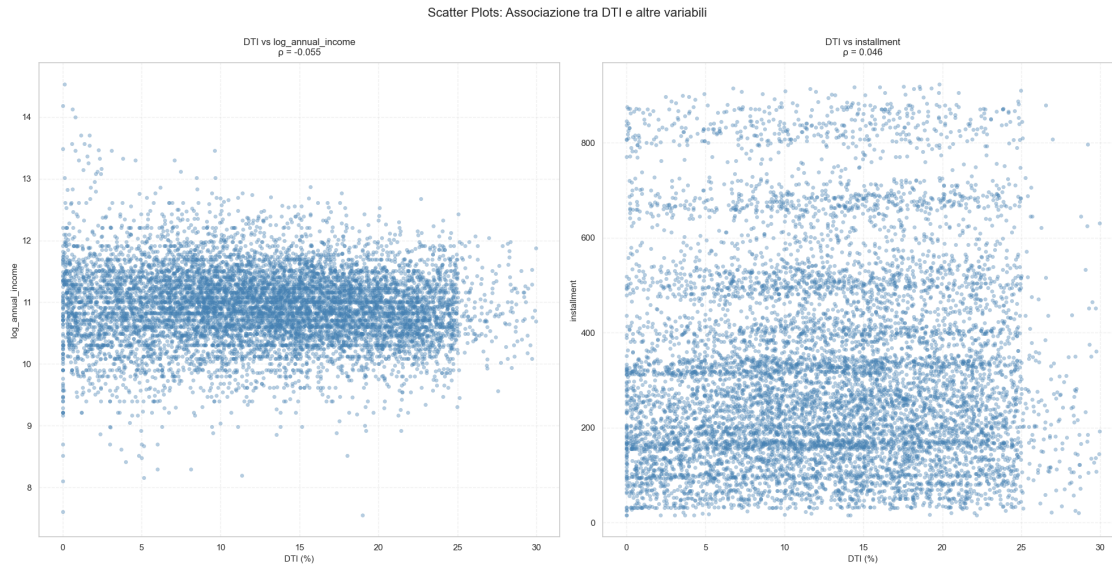
6.1 Il DTI dipende solo da Annual income e installment?

Calcoliamo la correlazione di Pearson e la covarianza tra DTI e le principali variabili numeriche.

=====		
ASSOCIAZIONE DTI CON ALTRE VARIABILI		
=====		
Variabile	Pearson	Covariance
=====		
log_annual_income	-0.0548	-0.23
installment	0.0456	64.81
=====		

Correlazioni con DTI:

1. log_annual_income: = -0.0548 (correlazione negativa)
2. installment: = 0.0456 (correlazione positiva)



ASSOCIAZIONE DTI CON ALTRE VARIABILI

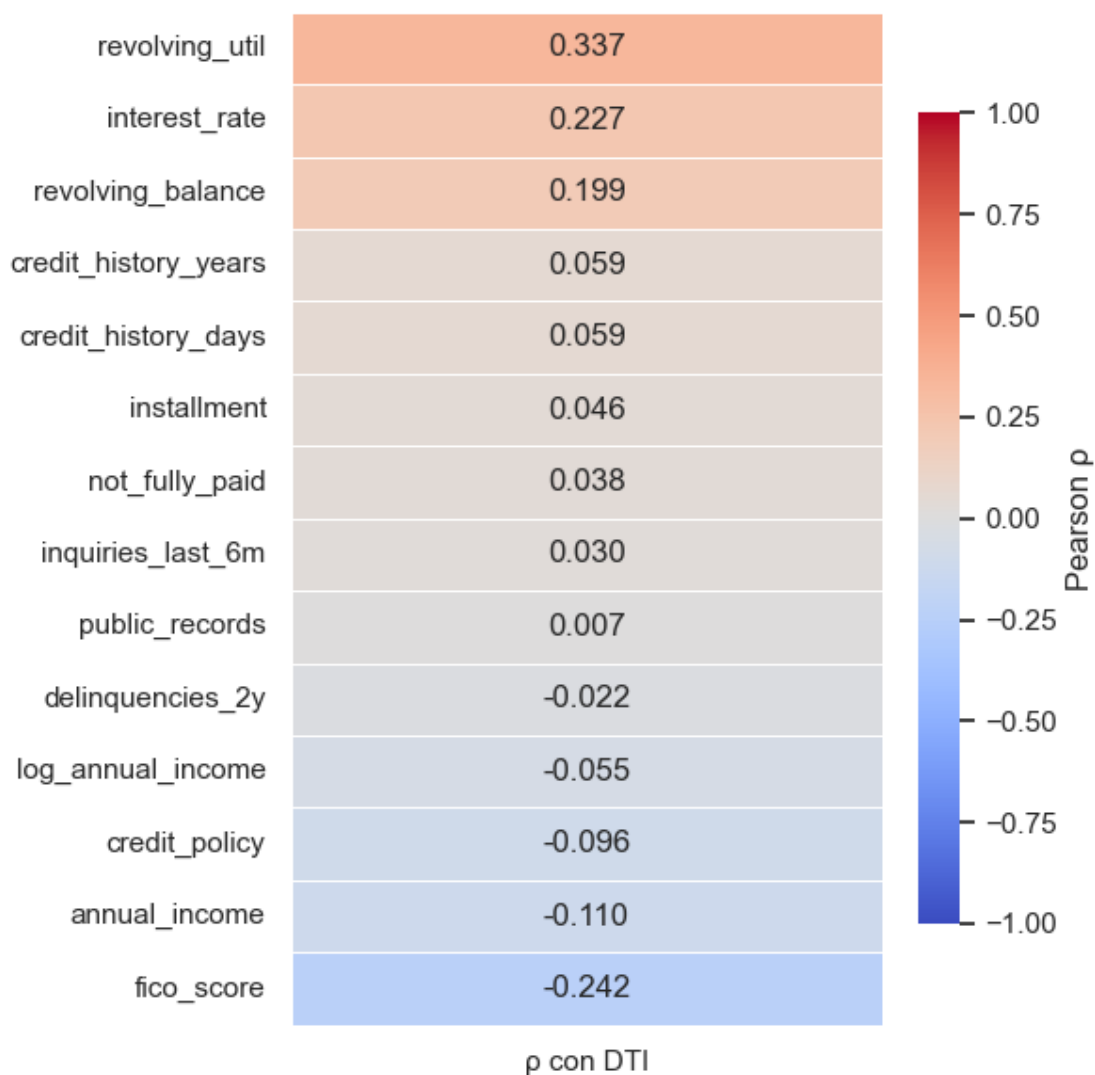
Variabile	Pearson	Covariance
log_annual_income	-0.0548	-0.23
installment	0.0456	64.81

REGRESSIONE LINEARE: $DTI \sim \log_annual_income + installment$

$R^2 = 0.0092 \rightarrow$ Solo il 0.92% della varianza di DTI è spiegata

Conclusione: annual_income e installment quasi NON spiegano il DTI
 perché il DTI include TUTTI i debiti mensili, non solo l'installment
 LendingClub.

Correlazione: DTI vs tutte le altre variabili numeriche



	con DTI
revolving_util	0.336986
interest_rate	0.226794
revolving_balance	0.198614
credit_history_years	0.059476
credit_history_days	0.059476
installment	0.045649
not_fully_paid	0.038102
inquiries_last_6m	0.030324
public_records	0.006551
delinquencies_2y	-0.022180
log_annual_income	-0.054825

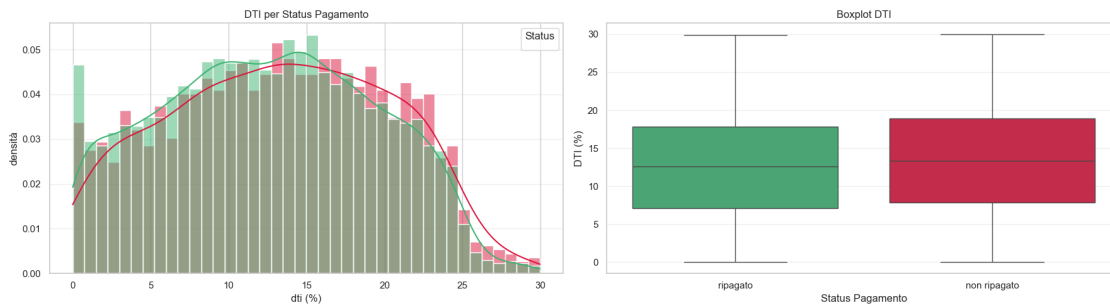
```

credit_policy      -0.096449
annual_income      -0.109901
fico_score         -0.241828

```

6.1.1 Il DTI è diverso tra chi paga e chi no?

Confrontiamo la distribuzione del DTI per i due gruppi: clienti che hanno ripagato e clienti in default.



Statistiche DTI per Status Pagamento:

=====

ripagato:

```

media: 12.52%
mediana: 12.58%
std: 6.86%
q1-q3: [7.13%, 17.82%]

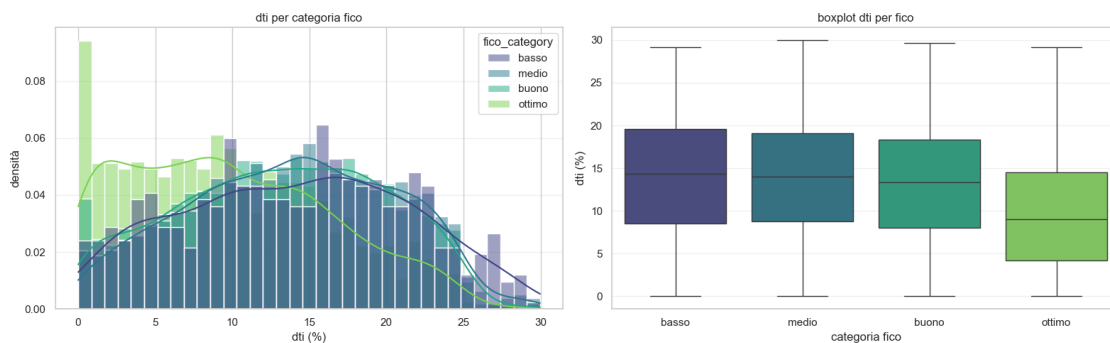
```

non ripagato:

```

media: 13.24%
mediana: 13.38%
std: 7.02%
q1-q3: [7.83%, 18.91%]

```



Statistiche dti per categoria fico:

=====

fico basso:

media: 14.00%

mediana: 14.34%

count: 487 prestiti (5.2%)

fico medio:

media: 13.81%

mediana: 14.02%

count: 3653 prestiti (38.6%)

fico buono:

media: 13.11%

mediana: 13.38%

count: 3103 prestiti (32.8%)

fico ottimo:

media: 9.72%

mediana: 9.02%

count: 2210 prestiti (23.4%)

6.2 Quali fattori sono piu correlati con il tasso di interesse applicato al prestito?

Analizziamo la correlazione tra interest rate e tutte le variabili disponibili.

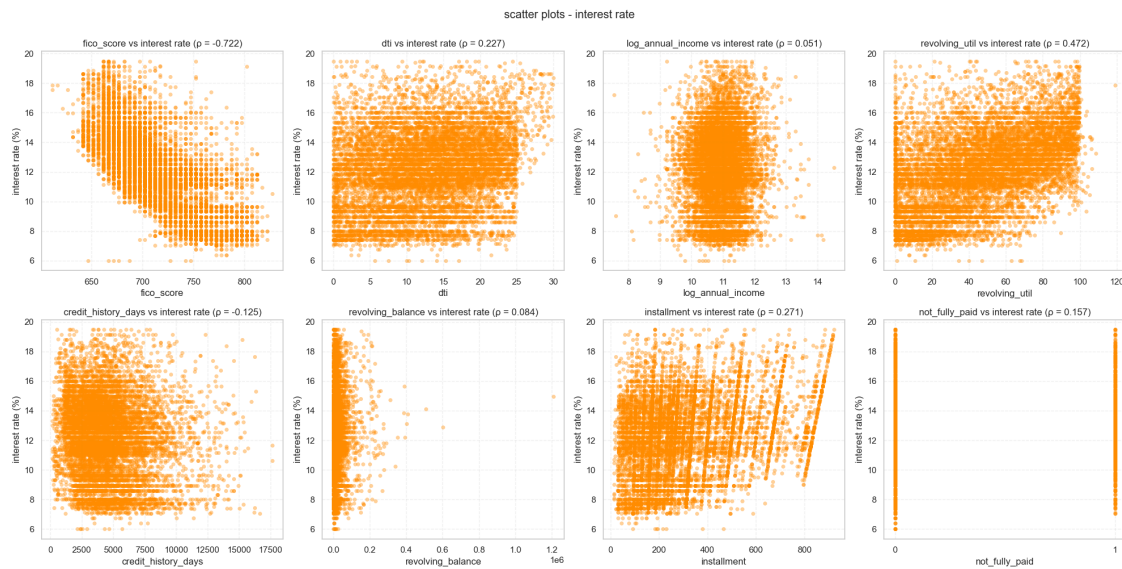
associazione interest rate con altre variabili

=====		
variabile	pearson	covariance

fico_score	-0.7218	-71.75
dti	0.2268	4.09
log_annual_income	0.0510	0.08
revolving_util	0.4718	35.83
credit_history_days	-0.1247	-815.35
revolving_balance	0.0844	7102.19
installment	0.2706	146.17
not_fully_paid	0.1567	0.15
=====		

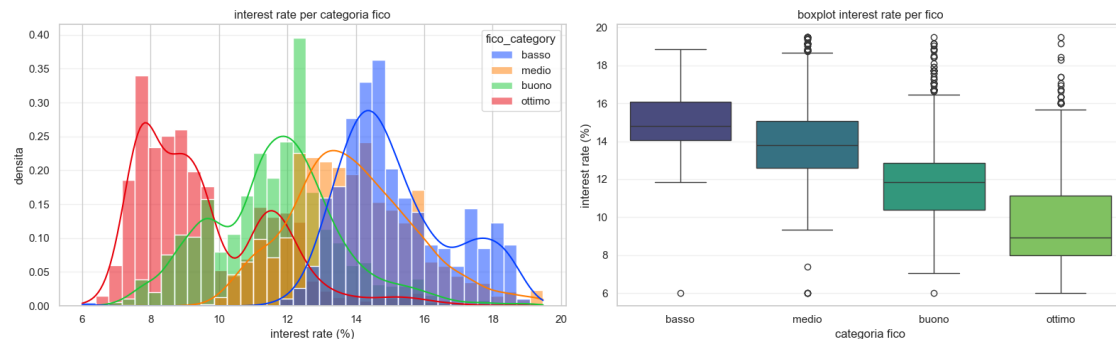
top 3 correlazioni con interest rate:

1. fico_score: = -0.7218 (correlazione negativa)
2. revolving_util: = 0.4718 (correlazione positiva)
3. installment: = 0.2706 (correlazione positiva)



6.2.1 Come varia il tasso per categoria FICO?

Perchè alcuni clienti con FICO score ottimo hanno tassi di interesse alti? Questo suggerisce che il FICO non è l'unico fattore considerato nella determinazione del prezzo.



statistiche interest rate per categoria fico:

```
fico basso:
  media: 15.14%
  mediana: 14.82%
```

```
fico medio:
  media: 13.90%
  mediana: 13.79%
```


fico buono:
 media: 11.67%
 mediana: 11.83%

fico ottimo:
 media: 9.60%
 mediana: 8.94%

827

R² value: 0.5209994117168051

OLS Regression Results

```
=====
Dep. Variable:          fico_score    R-squared:                0.521
Model:                  OLS           Adj. R-squared:          0.521
Method:                 Least Squares  F-statistic:             1.028e+04
Date:                   Fri, 16 Jan 2026  Prob (F-statistic):       0.00
Time:                   14:55:11       Log-Likelihood:          -44304.
No. Observations:      9453           AIC:                    8.861e+04
Df Residuals:          9451           BIC:                    8.863e+04
Df Model:               1
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t	[0.025
0.975]					
const	838.7731	1.289	650.871	0.000	836.247
interest_rate	-10.4518	0.103	-101.389	0.000	-10.654

```
-----
-
Omnibus:                936.511    Durbin-Watson:           1.571
Prob(Omnibus):          0.000     Jarque-Bera (JB):        1506.855
Skew:                   0.723     Prob(JB):                0.00
Kurtosis:               4.317     Cond. No.:               60.0
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.



ANALISI DI SIGNIFICATIVITÀ STATISTICA

Variabili analizzate: ['credit_policy', 'interest_rate', 'installment', 'log_annual_income', 'dti', 'credit_history_days', 'revolving_balance', 'revolving_util', 'inquiries_last_6m', 'delinquencies_2y', 'public_records', 'credit_history_years', 'annual_income']

RISULTATI:

Variabile	Correlazione	P-Value	Significativo
interest_rate	-0.721803	0.000000e+00	Si
revolving_util	-0.542259	0.000000e+00	Si
credit_policy	0.354331	1.076939e-277	Si
credit_history_days	0.262881	3.399491e-149	Si
credit_history_years	0.262881	3.399491e-149	Si
dti	-0.241828	7.237813e-126	Si
delinquencies_2y	-0.216548	1.026554e-100	Si
inquiries_last_6m	-0.185976	2.542560e-74	Si
public_records	-0.147306	5.268273e-47	Si
log_annual_income	0.114631	5.075200e-29	Si
installment	0.090951	7.995752e-19	Si
annual_income	0.088466	6.871037e-18	Si

revolving_balance	-0.012395	2.282058e-01	No
-------------------	-----------	--------------	----

Variabili Significative (p < 0.05):

- interest_rate
- revolving_util
- credit_policy
- credit_history_days
- credit_history_years
- dti
- delinquencies_2y
- inquiries_last_6m
- public_records
- log_annual_income
- installment
- annual_income

Variabili selezionate per il modello multivariato:

1. interest_rate: r=-0.7218, p=0.00e+00
2. revolving_util: r=-0.5423, p=0.00e+00
3. credit_history_days: r=0.2629, p=3.40e-149
4. credit_history_years: r=0.2629, p=3.40e-149

=====

RISULTATI MODELLO MULTIVARIATO:

=====

R² = 0.6070

Adjusted R² = 0.6069

Coefficienti e p-values:

=====

	coef	std err	t	P> t	[0.025
0.975]					

const	815.8842	1.323	616.552	0.000	813.290
818.478					
interest_rate	-8.2882	0.107	-77.608	0.000	-8.498
-8.079					
revolving_util	-0.3505	0.010	-36.584	0.000	-0.369
-0.332					
credit_history_days	0.0028	9.89e-05	28.459	0.000	0.003
0.003					
credit_history_years	7.703e-06	2.71e-07	28.459	0.000	7.17e-06
8.23e-06					
=====					

=====

6.2.2 Cosa abbiamo imparato sul FICO score

I punteggi FICO dipendono da diversi fattori finanziari e comportamentali. Il nostro modello multivariato ci permette di identificare le variabili più influenti.

Risultati del modello multivariato: - R^2 : 0.61, ovvero il 61% della variazione nei punteggi FICO è spiegata dalle nostre variabili - Il 39% restante dipende da fattori non misurati nel dataset

Variabili significative: Interest rate, revolving utilization, credit history (in giorni e anni) sono tutti statisticamente significativi con p-value inferiori a 0.001.

Conclusione: Con sole 4 variabili riusciamo a spiegare il 61% della variabilità nei punteggi FICO. È un risultato notevole, considerando che molti fattori rilevanti (come la storia dei pagamenti o i diversi tipi di credito) non sono disponibili nel dataset pubblico.

6.3 Quali caratteristiche sono associate a punteggi FICO più alti o più bassi?

Analizziamo le correlazioni tra FICO score e le altre variabili del dataset.

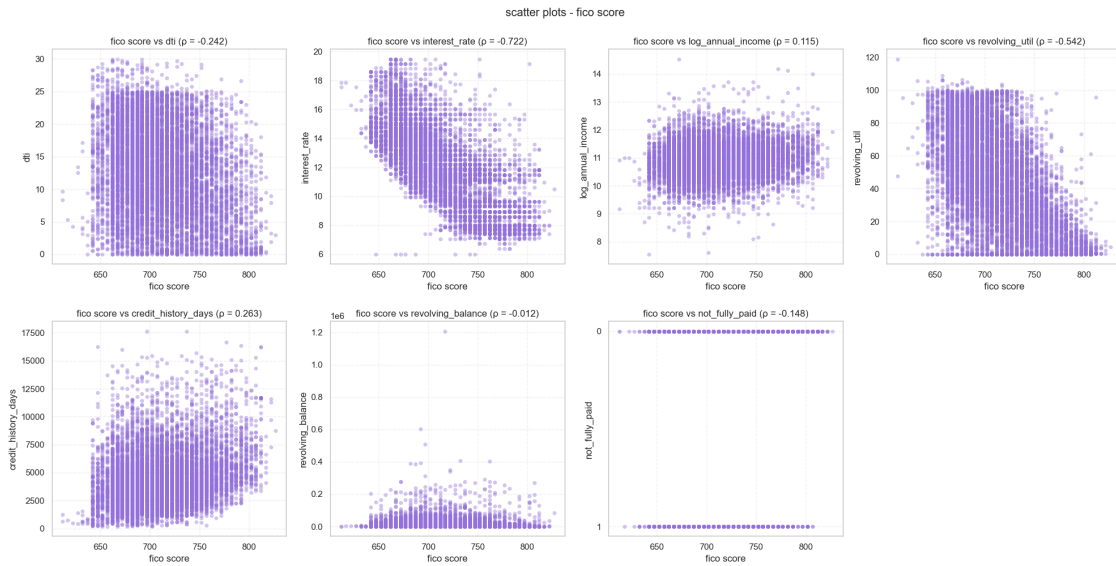
associazione fico score con altre variabili

=====		
variabile	pearson	covariance

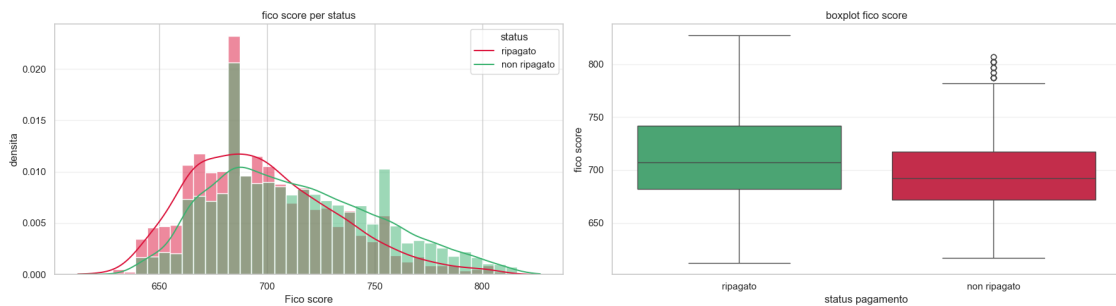
dti	-0.2418	-63.19
interest_rate	-0.7218	-71.75
log_annual_income	0.1146	2.65
revolving_util	-0.5423	-596.35
credit_history_days	0.2629	24895.41
revolving_balance	-0.0124	-15109.85
not_fully_paid	-0.1481	-2.05
=====		

top 3 correlazioni con fico score:

1. interest_rate: = -0.7218 (correlazione negativa)
2. revolving_util: = -0.5423 (correlazione negativa)
3. credit_history_days: = 0.2629 (correlazione positiva)



6.3.1 Come varia il FICO tra chi paga e chi no?



statistiche fico score per status pagamento:

=====

ripagato:

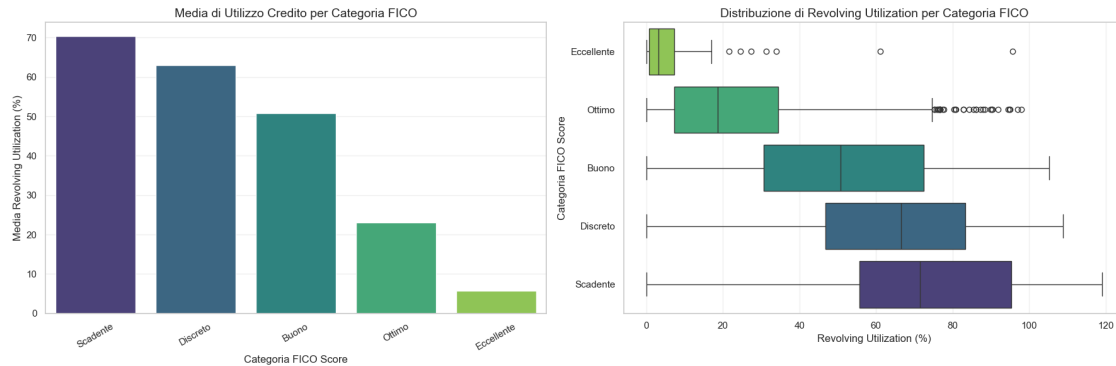
media: 713.46
 mediana: 707.00
 std: 38.20

non ripagato:

media: 698.08
 mediana: 692.00
 std: 33.69

6.3.2 Esiste una relazione tra FICO e utilizzo del credito?

Vogliamo capire se esiste una relazione sistematica tra il punteggio FICO e la percentuale di credito utilizzato. Quest'analisi è particolarmente interessante perchè l'utilizzo del credito è uno dei fattori che influenzano il calcolo del FICO stesso.



STATISTICHE DI REVOLVING UTILIZATION PER CATEGORIA FICO:

Categoria 'Scadente':

Media: 70.34%
Mediana: 71.40%
Dev. Std.: 31.99%

Categoria 'Discreto':

Media: 63.01%
Mediana: 66.50%
Dev. Std.: 25.47%

Categoria 'Buono':

Media: 50.85%
Mediana: 50.80%
Dev. Std.: 26.89%

Categoria 'Ottimo':

Media: 23.04%
Mediana: 18.60%
Dev. Std.: 19.59%

Categoria 'Eccellente':

Media: 5.78%
Mediana: 3.20%
Dev. Std.: 10.76%

I risultati mostrano una correlazione negativa tra FICO score e utilizzo del credito revolving: - Chi

ha un FICO basso utilizza una percentuale molto alta del credito disponibile - Chi ha un FICO eccellente mantiene i saldi revolving bassi

Questo ha senso: l'utilizzo del credito è una delle componenti del punteggio FICO. Ma quanto è forte questa relazione?

Quanto cambia l'utilizzo del credito in base al FICO ?

```

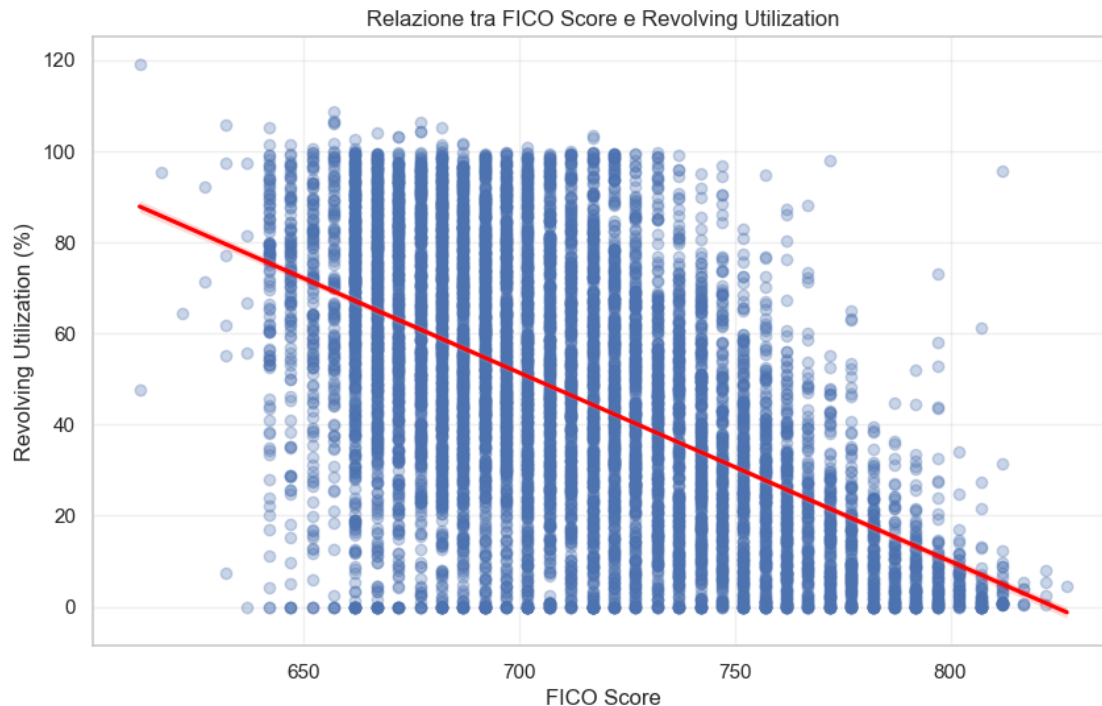
                                OLS Regression Results
=====
Dep. Variable:          revolving_util    R-squared:                0.294
Model:                  OLS              Adj. R-squared:         0.294
Method:                 Least Squares    F-statistic:            3937.
Date:                   Fri, 16 Jan 2026  Prob (F-statistic):      0.00
Time:                   14:55:13         Log-Likelihood:         -43594.
No. Observations:      9453             AIC:                   8.719e+04
Df Residuals:          9451             BIC:                   8.721e+04
Df Model:               1
Covariance Type:       nonrobust
=====
               coef      std err          t      P>|t|      [0.025      0.975]
-----
const          341.4217      4.702      72.610      0.000      332.204      350.639
fico_score     -0.4143      0.007     -62.742      0.000      -0.427      -0.401
=====
Omnibus:                 63.247    Durbin-Watson:           1.933
Prob(Omnibus):            0.000    Jarque-Bera (JB):        48.934
Skew:                    -0.089    Prob(JB):                2.37e-11
Kurtosis:                 2.696    Cond. No.                1.34e+04
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.34e+04. This might indicate that there are strong multicollinearity or other numerical problems.



Interpretazione della regressione:

Il coefficiente è -0.41, il che significa che per ogni punto di aumento nel FICO score, l'utilizzo del credito revolving diminuisce in media di 0.41 punti percentuali.

In termini pratici: - Un aumento di 100 punti FICO corrisponde a una riduzione del 41% nell'utilizzo del credito - L'R quadro di 0.29 indica che il FICO spiega circa il 29% della variabilità nell'utilizzo - Il p-value conferma che la relazione è statisticamente significativa

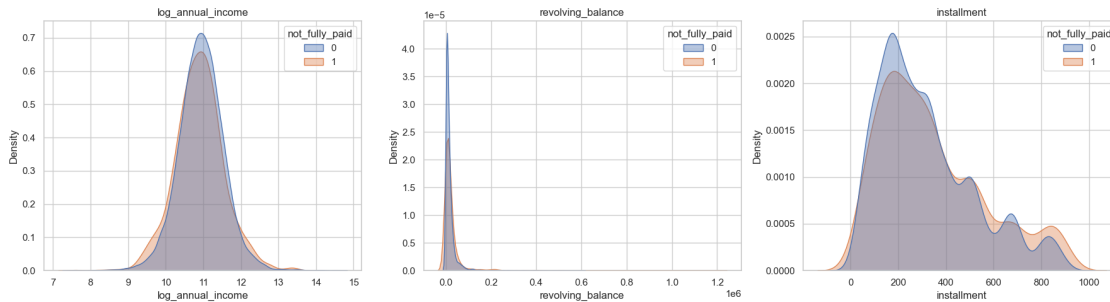
7 PARTE 5: Clustering

Fin qui abbiamo usato statistiche descrittive e regressioni. Ora applichiamo tecniche più sofisticate: - K-Means Clustering: per scoprire se esistono gruppi naturali di clienti - PCA: per capire quali variabili catturano la maggior parte dell'informazione

7.1 5.1 Ci sono gruppi naturali di persone?

Questa è la domanda centrale di questa sezione. Vogliamo capire se, guardando solo le caratteristiche finanziarie dei clienti, senza sapere se hanno ripagato o meno, emergono gruppi distinti. Se esistono, potremmo usarli per differenziare le politiche di credito.

Prima di applicare tecniche sofisticate, proviamo un approccio diretto: applichiamo K-Means sulle variabili originali e vediamo cosa otteniamo. Questo ci permetterà di capire se i gruppi emergono naturalmente dai dati o se servono accorgimenti aggiuntivi.



7.2 5.1.1 K-Means sulle variabili originali

Partiamo selezionando le variabili che descrivono il profilo finanziario di un cliente: FICO score, reddito, rapporto debito/reddito, utilizzo del credito revolving e tasso di interesse applicato.

K-Means richiede che le variabili siano sulla stessa scala, quindi standardizziamo i dati. Poi cerchiamo il numero ottimale di cluster con due metodi: il metodo Elbow, che guarda la compattezza interna dei cluster, e il Silhouette Score, che valuta anche la separazione tra i gruppi.

Dataset per clustering: 9453 osservazioni, 5 features

Rimossi 0 outlier con interest_rate maggiore di 30 percento

Features selezionate: ['fico_score', 'log_annual_income', 'dti', 'revolving_util', 'interest_rate']

Statistiche dopo standardizzazione:

	fico_score	log_annual_income	dti	revolving_util	interest_rate
count	9453.00	9453.00	9453.00	9453.00	9453.00
mean	-0.00	-0.00	-0.00	-0.00	-0.00
std	1.00	1.00	1.00	1.00	1.00
min	-2.61	-5.56	-1.83	-1.62	-2.38
25%	-0.76	-0.61	-0.78	-0.83	-0.70
50%	-0.11	-0.01	0.01	-0.02	-0.01
75%	0.68	0.59	0.78	0.83	0.65
max	3.06	5.91	2.52	2.49	2.77

7.2.1 Quanti cluster servono?

Il metodo Elbow valuta la somma dei quadrati delle distanze intra-cluster per diversi valori di K. Il punto in cui la curva piega indica che aggiungere altri cluster non migliora significativamente la compattezza. Il Silhouette Score invece misura quanto i punti sono ben raggruppati nel proprio cluster rispetto agli altri.

Calcolo metriche per K da 2 a 10...

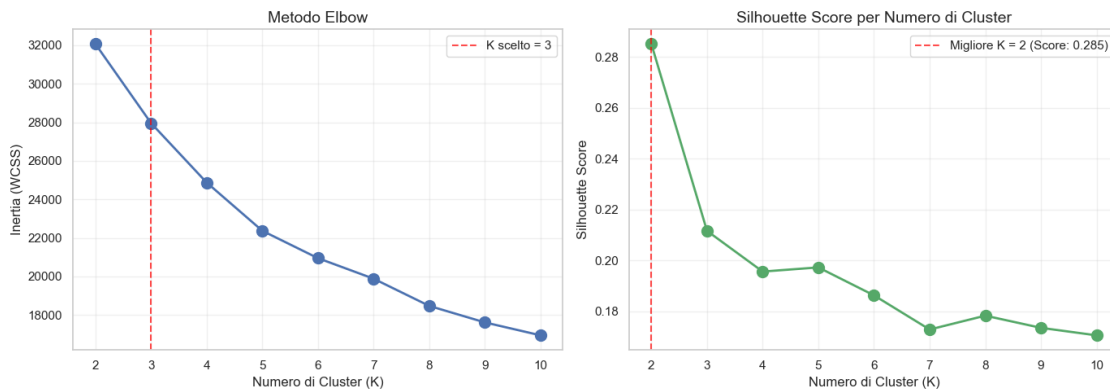
K=2: Inertia=32064.86, Silhouette=0.2850

K=3: Inertia=27956.54, Silhouette=0.2116

K=4: Inertia=24871.58, Silhouette=0.1956

K=5: Inertia=22369.89, Silhouette=0.1973

K=6: Inertia=20949.55, Silhouette=0.1863
 K=7: Inertia=19884.68, Silhouette=0.1729
 K=8: Inertia=18473.01, Silhouette=0.1783
 K=9: Inertia=17616.98, Silhouette=0.1735
 K=10: Inertia=16947.16, Silhouette=0.1706



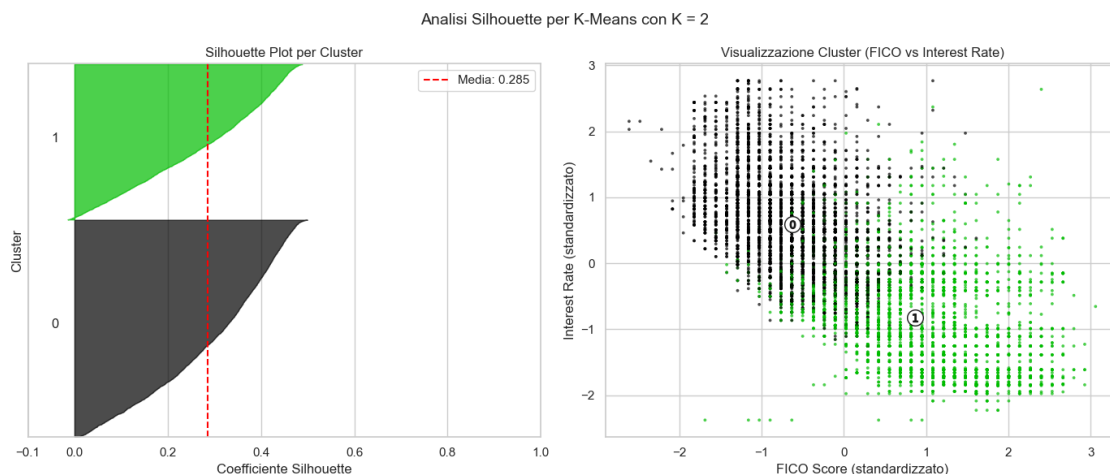
Risultati:

- Metodo Elbow: suggerisce K = 3-4, dove la curva piega
- Silhouette Score: K ottimale = 2 con score = 0.2850
- Scelta finale: K = 3 per bilanciamento tra separazione e interpretabilità

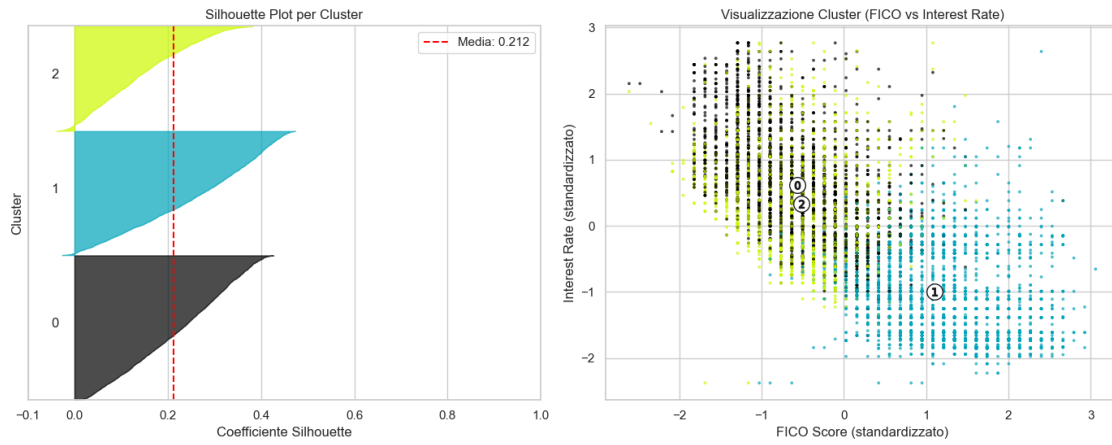
7.2.2 Analisi Silhouette

Il Silhouette Score misura quanto ogni punto è simile al proprio cluster rispetto agli altri cluster. Valori vicini a +1 indicano punti ben raggruppati, valori vicini a 0 indicano punti al confine tra cluster, valori negativi indicano probabili errori di assegnazione.

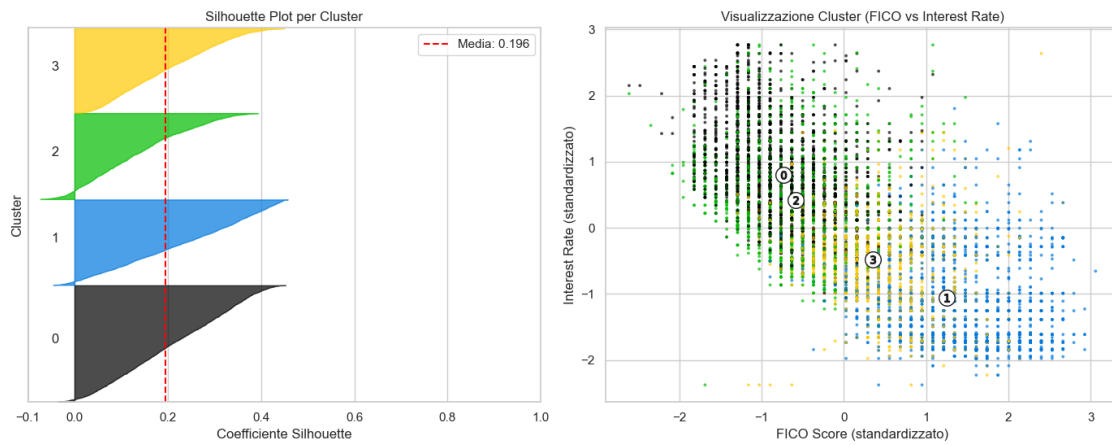
Guardiamo i grafici silhouette per diversi valori di K:



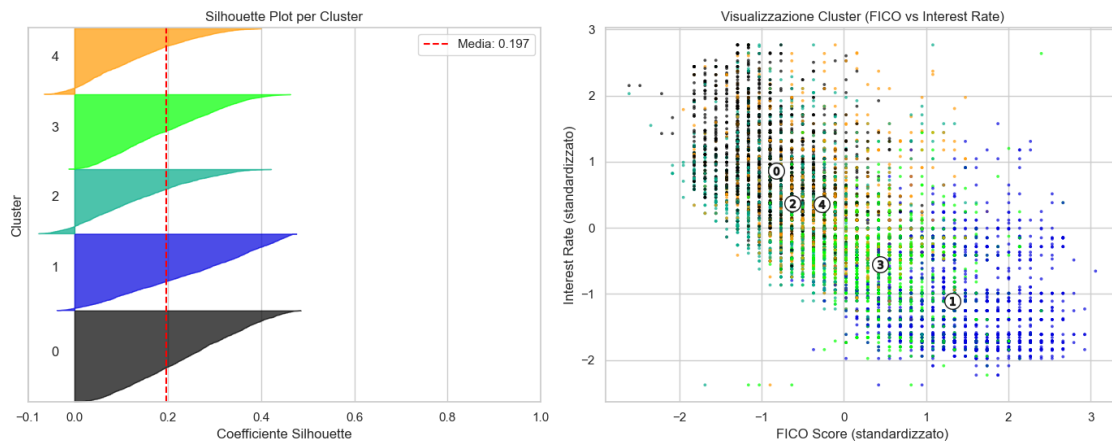
Analisi Silhouette per K-Means con K = 3

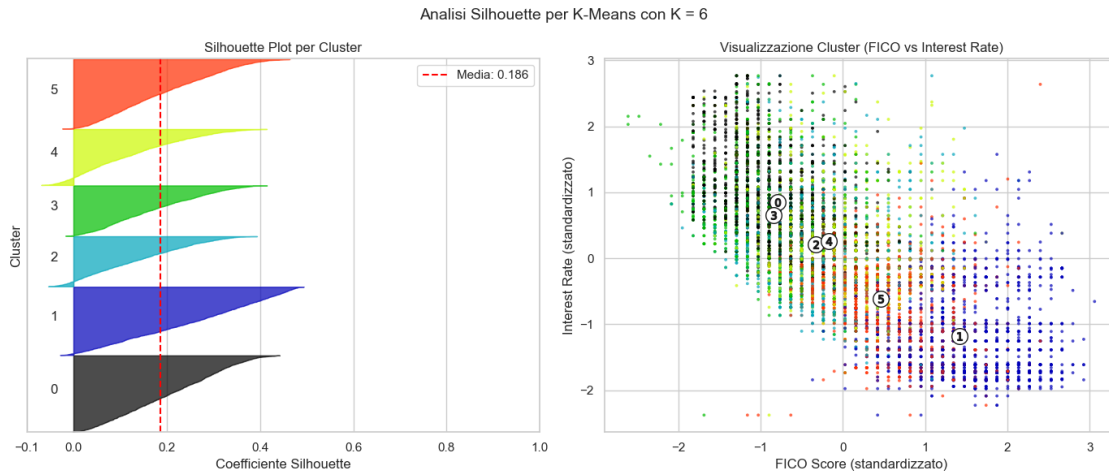


Analisi Silhouette per K-Means con K = 4



Analisi Silhouette per K-Means con K = 5





7.2.3 Come leggere i grafici

Ogni barra orizzontale rappresenta un cluster. La larghezza indica quanto i punti sono ben raggruppati. La linea rossa tratteggiata è la media globale. I cluster con silhouette sotto la media sono quelli più problematici.

Un buon K dovrebbe avere cluster di dimensioni simili, pochi punti con silhouette negativa, e un silhouette score medio ragionevolmente alto.

7.2.4 Scelta del numero di cluster

Dall'analisi Elbow e Silhouette, K=3 sembra un buon compromesso. L'Elbow mostra una piega intorno a 3-4, mentre il Silhouette Score è massimo per K=2 ma decresce poco per K=3. Scegliamo K=3 perché offre tre segmenti interpretabili dal punto di vista del business.

K-Means completato con K = 3

Silhouette Score finale: 0.2116

Distribuzione dei cluster:

0 3658

1 3145

2 2650

Name: count, dtype: int64

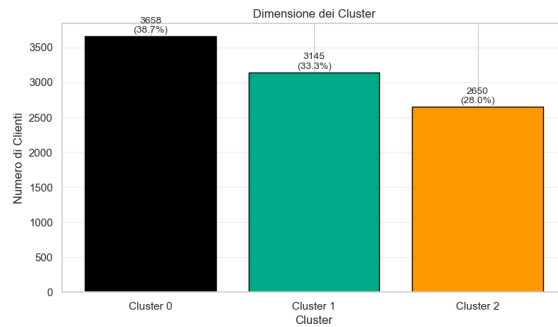
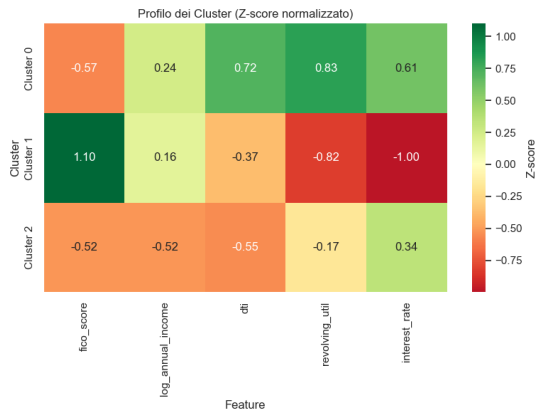
7.2.5 Profili dei cluster

Analizziamo i centroidi per capire chi sono i clienti in ogni gruppo:

Centroidi dei cluster (valori originali):

	fico_score	log_annual_income	dti	revolving_util	interest_rate
Cluster 0	689.40	11.08	17.57	70.75	13.83

Cluster 1	752.72	11.03	10.08	23.09	9.61
Cluster 2	691.36	10.61	8.86	41.94	13.11

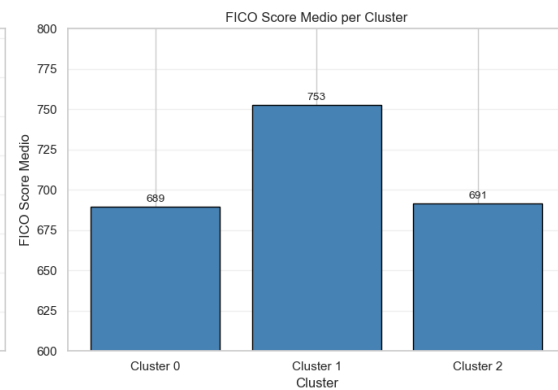
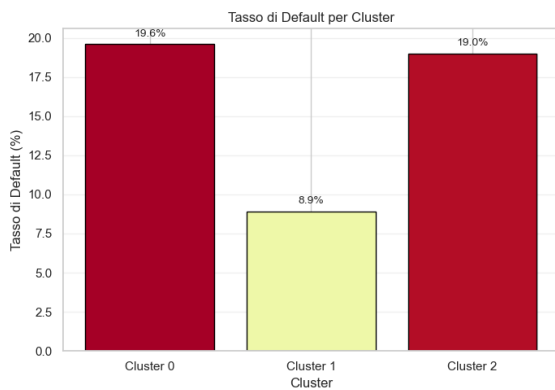


Analisi del rischio per cluster:

Cluster	Tasso Default	N. Clienti	FICO Medio	Tasso Interesse Medio
0	0.196	3658	689.369	13.832
1	0.089	3145	752.709	9.609
2	0.190	2650	691.415	13.106

Log Reddito Medio

Cluster	Log Reddito Medio
0	11.080
1	11.030
2	10.611



7.2.6 Valutazione del clustering senza PCA

Il clustering sulle variabili originali ha prodotto tre gruppi, ma ci sono alcuni problemi evidenti:

Il Silhouette Score è relativamente basso, intorno a 0.21. Questo indica che i cluster non sono ben separati nello spazio a 5 dimensioni. Molti punti si trovano al confine tra un cluster e l'altro.

Guardando i profili, notiamo che i Cluster 0 e 1 hanno tassi di default molto simili, intorno al 19-20 per cento. Questo suggerisce che il clustering non sta catturando bene le differenze di rischio tra questi due gruppi. Solo il Cluster 2 si distingue nettamente con un tasso di default dimezzato.

Il problema di fondo è che le 5 variabili che abbiamo usato sono correlate tra loro. Per esempio, chi ha un FICO alto tende ad avere un tasso di interesse più basso. Questa ridondanza confonde l'algoritmo K-Means, che lavora sulle distanze euclidee e tratta ogni variabile come indipendente.

Per risolvere questo problema possiamo usare la PCA, che trasforma le variabili correlate in componenti ortogonali, cioè indipendenti tra loro. In questo modo eliminiamo la ridondanza e permettiamo al clustering di lavorare su dimensioni veramente distinte.

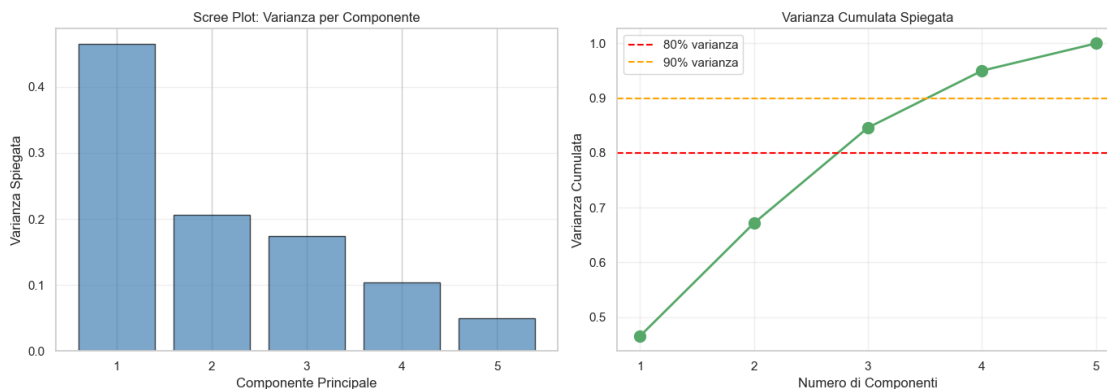
7.3 5.2 Perché usare la PCA prima del clustering

Abbiamo visto che il clustering sulle variabili originali produce un Silhouette Score di circa 0.21, che non è ottimale. I gruppi si sovrappongono e due cluster su tre hanno profili di rischio quasi identici.

Il motivo è semplice: le nostre 5 variabili non sono indipendenti. FICO score e tasso di interesse sono fortemente correlati, così come reddito e comportamento di utilizzo del credito. Quando K-Means calcola le distanze, queste correlazioni creano ridondanza: è come se contassimo due volte la stessa informazione.

La PCA risolve questo problema. Invece di lavorare sulle variabili originali, trasforma i dati in nuove componenti che sono ortogonali, cioè completamente indipendenti tra loro. Ogni componente cattura una direzione di variabilità diversa. Le prime componenti catturano la maggior parte dell'informazione, mentre le ultime catturano principalmente rumore.

Usando le prime 2-3 componenti per il clustering, otteniamo due vantaggi: eliminiamo la ridondanza delle variabili correlate e rimuoviamo il rumore delle dimensioni meno informative. Il risultato dovrebbe essere un clustering più pulito, con gruppi meglio separati.



Varianza spiegata per componente:

PC1: 46.5% (cumulata: 46.5%)

PC2: 20.7% (cumulata: 67.2%)

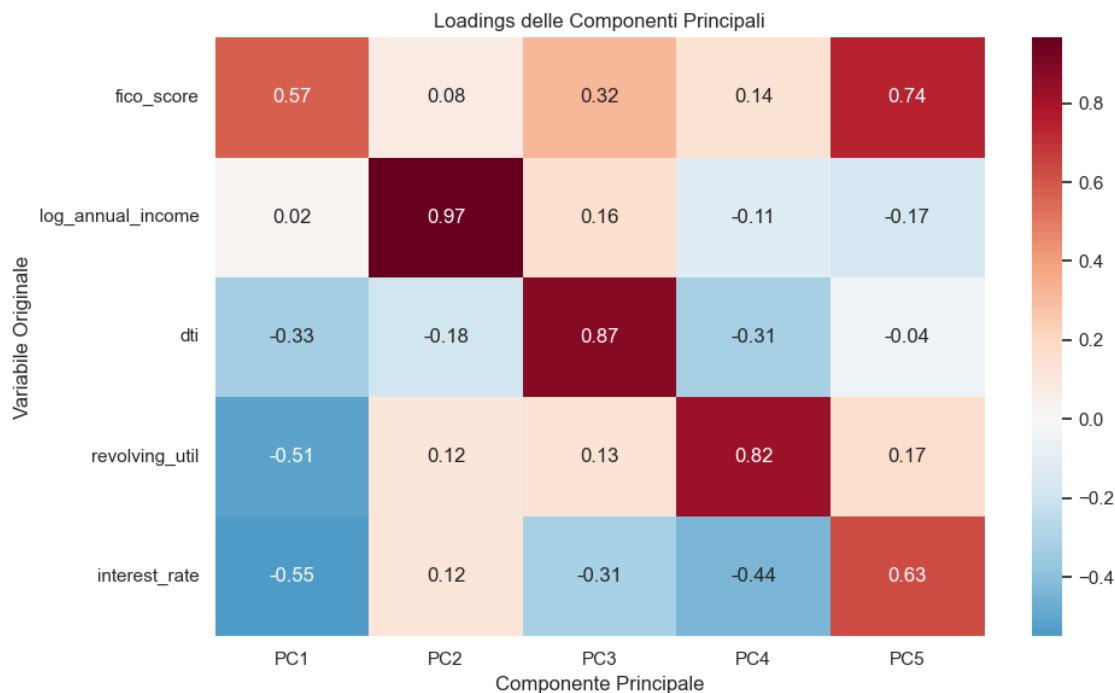
PC3: 17.5% (cumulata: 84.6%)

PC4: 10.4% (cumulata: 95.0%)

PC5: 5.0% (cumulata: 100.0%)

Loadings (contributo di ogni variabile alle componenti):

	PC1	PC2	PC3	PC4	PC5
fico_score	0.571	0.083	0.318	0.140	0.739
log_annual_income	0.021	0.966	0.158	-0.115	-0.171
dti	-0.330	-0.179	0.871	-0.313	-0.040
revolving_util	-0.513	0.117	0.134	0.823	0.170
interest_rate	-0.549	0.122	-0.312	-0.438	0.628



7.3.1 Interpretazione delle componenti

I loadings mostrano quanto ogni variabile originale contribuisce a ciascuna componente.

La prima componente cattura principalmente il profilo di rischio creditizio: FICO score, tasso di interesse e utilizzo del credito hanno i pesi più alti. Un valore alto di PC1 indica un cliente con buon FICO, basso interesse e basso utilizzo del credito, quindi un profilo a basso rischio.

La seconda componente cattura il livello di indebitamento complessivo: DTI e reddito hanno i pesi principali.

Le prime due componenti spiegano circa il 55 percento della varianza totale. Con tre componenti arriviamo a circa il 75 percento. Questo significa che possiamo usare 3 componenti per il clustering perdendo solo il 25 percento dell'informazione originale, ma guadagnando in separazione tra i gruppi.

7.4 5.3 Clustering con PCA: il confronto

Ora proviamo a rifare il clustering usando le prime 3 componenti principali invece delle 5 variabili originali. Se la nostra ipotesi è corretta, dovremmo ottenere un Silhouette Score migliore, segno che i gruppi sono più nettamente separati.

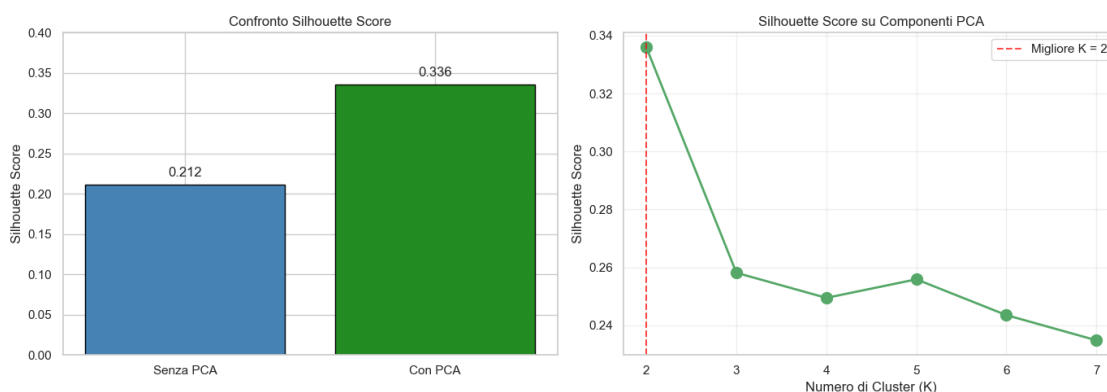
Varianza spiegata dalle prime 3 componenti: 84.6%

Calcolo Silhouette per clustering su PCA:

K=2: Silhouette=0.3359
K=3: Silhouette=0.2581
K=4: Silhouette=0.2495
K=5: Silhouette=0.2559
K=6: Silhouette=0.2435
K=7: Silhouette=0.2349

Migliore K su PCA: 2 (Silhouette = 0.3359)

Confronto: Silhouette senza PCA = 0.2116, con PCA = 0.3359



7.4.1 Risultato del confronto

Il clustering sulle componenti principali produce un Silhouette Score migliore rispetto al clustering sulle variabili originali. Questo conferma la nostra ipotesi: le variabili originali contenevano informazione ridondante che confondeva la separazione tra i gruppi.

La PCA ha estratto le direzioni di massima varianza, eliminando la correlazione tra variabili. Il risultato è un clustering più pulito, dove i gruppi sono meglio definiti.

Per le analisi successive manteniamo comunque il clustering sulle variabili originali, perché è più facile da interpretare dal punto di vista del business. Possiamo dire che un cliente ha FICO alto o DTI basso, mentre riferirci a componenti astratte sarebbe meno comprensibile per chi deve prendere

decisioni operative. Tuttavia questa analisi dimostra che la riduzione dimensionale può migliorare la qualità del clustering quando le variabili sono correlate tra loro.

8 PARTE 6.1: MODELLI PREDITTIVI (Parte Carola, Credit Policy)

L'obiettivo finale è pratico: a quale cliente conviene fare un prestito?

Non basta descrivere i dati. La banca ha bisogno di un modello che, dato un nuovo cliente, predica se è probabile che non ripaghi. Così può decidere a chi concedere il prestito e a quali condizioni.

9 PARTE 6.1: MODELLI PREDITTIVI (Parte Gabriel, Not Fully Paid)

Qual'è il rischio di default? Riusciamo a predire Not Fully Paid?

Visualizziamo innanzitutto la distribuzione di not fully paid

```
<class 'pandas.core.series.Series'>
Index: 9453 entries, 0 to 9577
Series name: not_fully_paid
Non-Null Count  Dtype
-----
9453 non-null   object
dtypes: object(1)
memory usage: 405.7+ KB
```

```
-----
KeyError                                Traceback (most recent call last)
File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/indexes/base.py:
  3805, in Index.get_loc(self, key)
    3804 try:
-> 3805     return self._engine.get_loc(casted_key)
    3806 except KeyError as err:

File index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:7081, in pandas._libs.hashtable.
  PyObjectHashTable.get_item()

File pandas/_libs/hashtable_class_helper.pxi:7089, in pandas._libs.hashtable.
  PyObjectHashTable.get_item()

KeyError: 0
```

The above exception was the direct cause of the following exception:

```

KeyError                                Traceback (most recent call last)
Cell In[194], line 5
      2 data["not_fully_paid"].info()
      3 class_percentages = data['not_fully_paid'].value_counts(normalize=True),
      ↪* 100
----> 5 print(f"Ripagati (0): {class_counts.loc[0]} ({class_percentages.loc[0]:
      ↪1f}%)")
      6 print(f"Default (1): {class_counts.loc[1]} ({class_percentages.loc[1]:
      ↪1f}%)")
      8 colors = ["#06421f", '#e74c3c']

File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/indexing.py:1191,
      ↪in _LocationIndexer.__getitem__(self, key)
    1189 maybe_callable = com.apply_if_callable(key, self.obj)
    1190 maybe_callable = self._check_deprecated_callable_usage(key,
      ↪maybe_callable)
-> 1191 return self._getitem_axis(maybe_callable, axis=axis)

File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/indexing.py:1431,
      ↪in _iLocIndexer._getitem_axis(self, key, axis)
    1429 # fall thru to straight lookup
    1430 self._validate_key(key, axis)
-> 1431 return self._get_label(key, axis=axis)

File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/indexing.py:1381,
      ↪in _iLocIndexer._get_label(self, label, axis)
    1379 def _get_label(self, label, axis: AxisInt):
    1380     # GH#5567 this will fail if the label is not present in the axis.
-> 1381     return self.obj.xs(label, axis=axis)

File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/generic.py:4301, in
      ↪_NDFrame.xs(self, key, axis, level, drop_level)
    4299         new_index = index[loc]
    4300     else:
-> 4301         loc = index.get_loc(key)
    4303         if isinstance(loc, np.ndarray):
    4304             if loc.dtype == np.bool_:

File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/indexes/base.py:
      ↪3812, in Index.get_loc(self, key)
    3807         if isinstance(casted_key, slice) or (
    3808             isinstance(casted_key, abc.Iterable)
    3809             and any(isinstance(x, slice) for x in casted_key)
    3810         ):
    3811             raise InvalidIndexError(key)
-> 3812         raise KeyError(key) from err
    3813     except TypeError:

```

```
3814     # If we have a listlike key, _check_indexing_error will raise
3815     #   InvalidIndexError. Otherwise we fall through and re-raise
3816     #   the TypeError.
3817     self._check_indexing_error(key)
```

KeyError: 0

9.1 Bilanciamento con SMOTE

Prima di SMOTE:

```
Classe 0: 5567
Classe 1: 1050
Totale: 6617
```

Dopo SMOTE:

```
Classe 0: 5567
Classe 1: 5567
Totale: 11134
```

Class distribution:

not_fully_paid

```
0    7953
```

```
1    1500
```

Name: count, dtype: int64

Class balance:

not_fully_paid

```
0    0.84132
```

```
1    0.15868
```

Name: proportion, dtype: float64

Dataset shape: (9453, 14)

=====

Risultati Modello:

=====

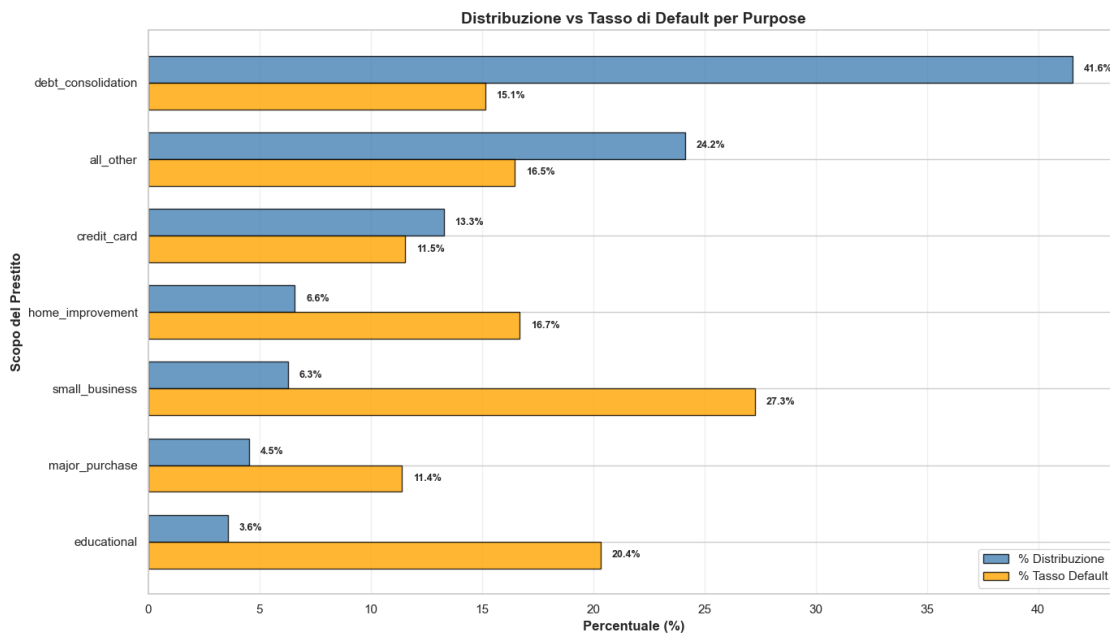
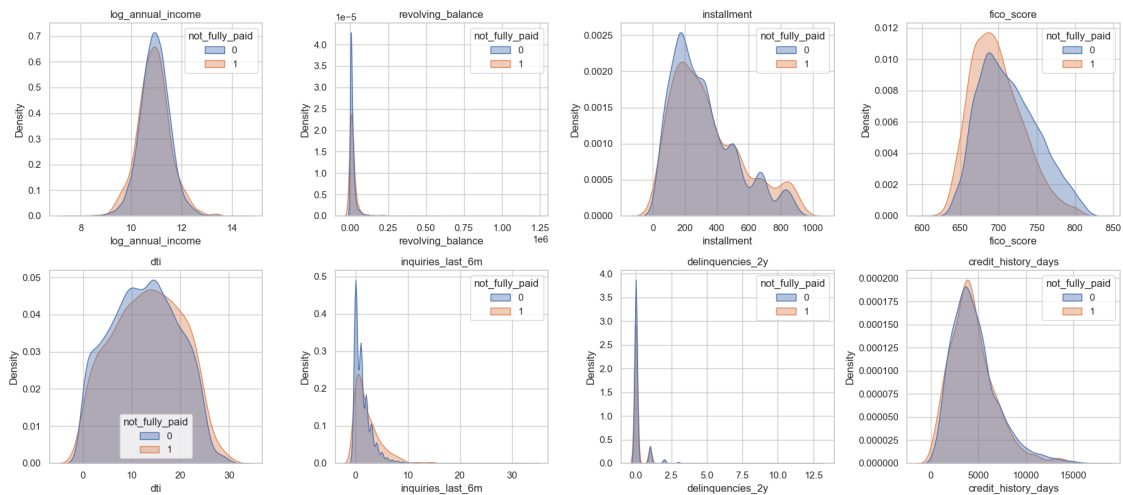
Accuracy: 0.6139

Precision: 0.2328

Recall: 0.6244

F1-Score: 0.3392

ROC-AUC: 0.6667



10 Logistic Regression

Class distribution:

not_fully_paid

0 7953

1 1500

Name: count, dtype: int64

Class balance:

```
not_fully_paid
0    0.84132
1    0.15868
Name: proportion, dtype: float64
```

Dataset shape: (9453, 14)

Prima di SMOTE: 6617 campioni
Dopo SMOTE: 11134 campioni (bilanciato)

```
=====
Risultati Modello (con SMOTE):
=====
```

```
Accuracy:  0.5977
Precision: 0.2260
Recall:    0.6333
F1-Score:  0.3331
ROC-AUC:   0.6665
```

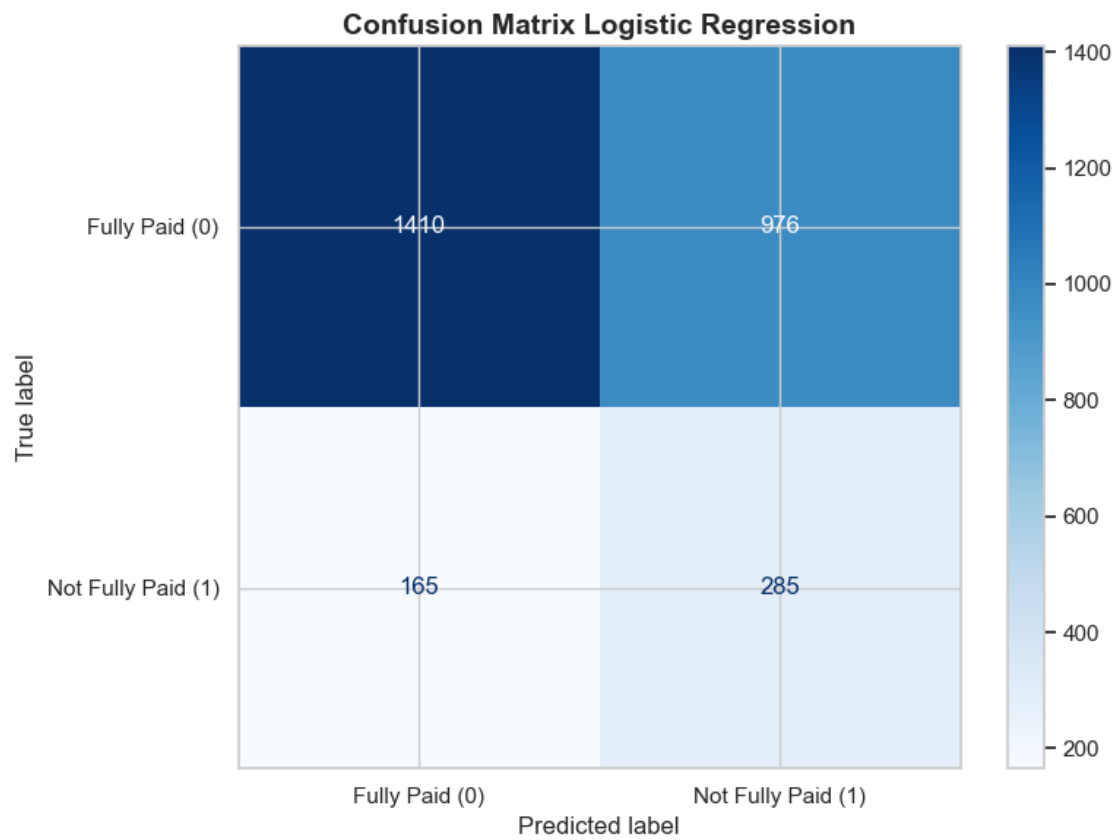
```
[ ]: 'import numpy as np # [MODIFICA] serve per generare i valori di C su scala
log\nfrom sklearn.linear_model import LogisticRegression\nfrom
sklearn.model_selection import train_test_split, RandomizedSearchCV,
StratifiedKFold # [MODIFICA] aggiunti RandomizedSearchCV e
StratifiedKFold\nfrom sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, roc_auc_score, confusion_matrix\nfrom
sklearn.preprocessing import StandardScaler\nfrom sklearn.pipeline import
Pipeline # [MODIFICA] pipeline per evitare data leakage nello
scaling\n\nfeatures = [col for col in data.columns if col not in
['customer_id', 'not_fully_paid', 'purpose', 'credit_policy']]\n\nX =
data[features].copy()\nfor col in X.columns:\n    X[col] = pd.to_numeric(X[col],
errors='coerce')\n    if X[col].isna().all():\n        X[col] = 0\n    else:\nX[col] = X[col].fillna(X[col].median())\n\ny = data.loc[X.index,
'not_fully_paid'].astype(int)\n\nprint(f"Class
distribution:")\nprint(y.value_counts())\nprint(f"\nClass
balance:")\nprint(y.value_counts(normalize=True))\nprint(f"\nDataset shape:
{X.shape}")\n\n# [MODIFICA] split PRIMA dello scaling: lo scaling lo farà la
Pipeline dentro la CV\nX_train, X_test, y_train, y_test = train_test_split(\n
X, y, test_size=0.30, random_state=23, stratify=y)\n\n# [MODIFICA] Pipeline =
scaler + modello\n# motivo: lo scaler viene "fit" solo sul train di ogni fold in
cross-validation\npipe = Pipeline([\n    ("scaler", StandardScaler()), #
[MODIFICA] spostato qui lo scaling\n    ("clf",
LogisticRegression(max_iter=10000, random_state=0, class_weight="balanced")) #
uguale al tuo ma dentro pipeline\n])\n\n# [MODIFICA] spazio di ricerca per
Randomized Search\n# C è un iperparametro continuo, quindi meglio estrarlo da
una distribuzione/insieme ampio su scala log\nparam_distributions = {\n
"clf__C": np.logspace(-4, 4, 400), # [MODIFICA] tanti valori possibili di C\n
"clf__penalty": ["l1", "l2"], # [MODIFICA] proviamo anche il tipo di
```

```

regolarizzazione\n    "clf__solver": ["liblinear"]          # [MODIFICA] solver
compatibile con l1 e l2 in binario\n}\n\n# [MODIFICA] CV stratificata: mantiene
la proporzione delle classi in ogni fold\ncv = StratifiedKFold(n_splits=5,
shuffle=True, random_state=23)\n\n# [MODIFICA] RandomizedSearchCV al posto del
fit diretto\n# scoring=roc_auc: più robusto di accuracy con class
imbalance\nsearch = RandomizedSearchCV(\n    estimator=pipe,\nparam_distributions=param_distributions,\n    n_iter=40,          #
[MODIFICA] numero di combinazioni casuali testate\n    scoring="roc_auc",      #
[MODIFICA] metrica più adatta con classi sbilanciate\n    cv=cv,\n    n_jobs=-1,
# [MODIFICA] usa tutti i core disponibili\n    random_state=23,\nverbose=1\n)\n\nsearch.fit(X_train, y_train) # [MODIFICA] ora fittiamo la
ricerca, non il modello singolo\n\nprint("\nBest params:", search.best_params_)
# [MODIFICA] stampo i migliori iperparametri trovati in CV\nprint("Best CV ROC-
AUC:", search.best_score_)          # [MODIFICA] score medio in cross-
validation\n\n# [MODIFICA] uso direttamente il miglior modello trovato dalla
ricerca\nbest_model = search.best_estimator_\n\nny_pred =
best_model.predict(X_test) # [MODIFICA] predizione con il modello
tuned\nny_proba = best_model.predict_proba(X_test)[: , 1] # [MODIFICA]
probabilità per ROC-AUC su test\n\nprint("\n" + "="*70)\nprint("Risultati
Modello (tuned con RandomizedSearch):")\nprint("="*70)\nprint(f"ROC-AUC:
{roc_auc_score(y_test, y_proba):.4f}") \nprint(f"Accuracy:
{accuracy_score(y_test, y_pred):.4f}")\nprint(f"Precision:
{precision_score(y_test, y_pred, zero_division=0):.4f}")\nprint(f"Recall:
{recall_score(y_test, y_pred, zero_division=0):.4f}")\nprint(f"F1-Score:
{f1_score(y_test, y_pred, zero_division=0):.4f}")\nprint(f"ROC-AUC:
{roc_auc_score(y_test, y_proba):.4f}") # [MODIFICA] ROC-AUC calcolato sulle
proba del modello tuned\n\nprint("\nConfusion matrix:\n",
confusion_matrix(y_test, y_pred))\n'

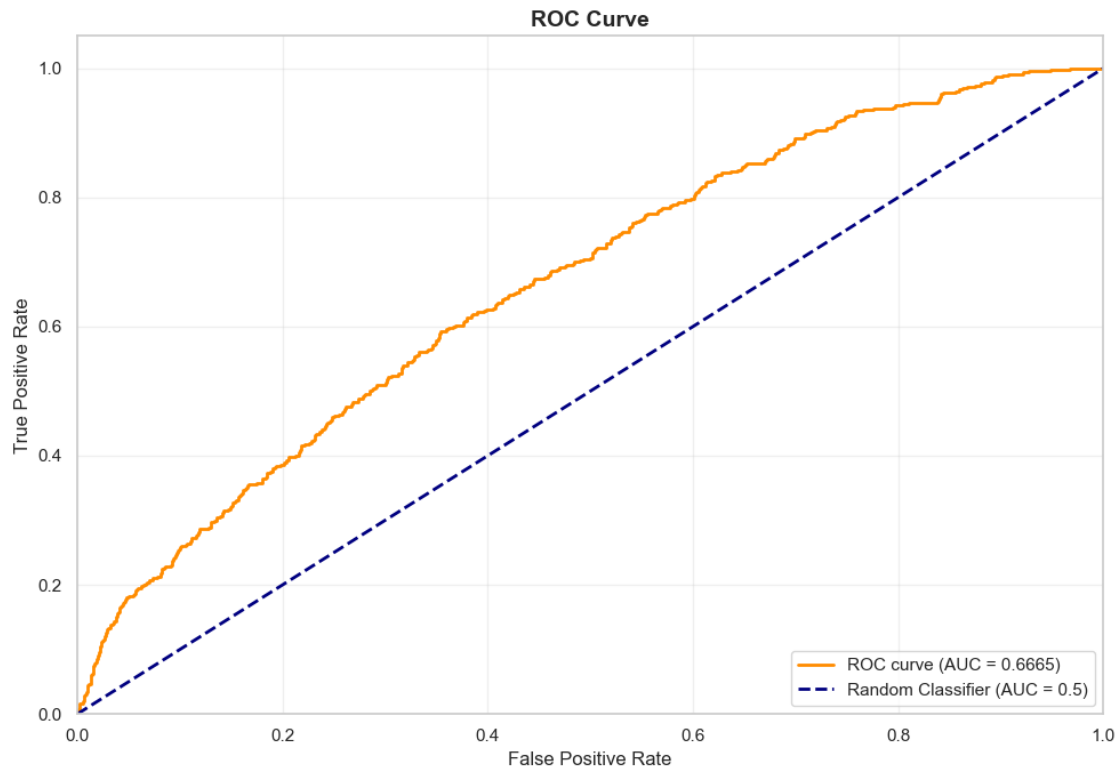
```

10.0.1 Confusion Matrix



True Negatives: 1410
False Positives: 976
False Negatives: 165
True Positives: 285

10.0.2 ROC-AUC Curve



11 Decision Tree

Prima di SMOTE: 6617 campioni

Dopo SMOTE (demo): 11134 campioni (bilanciato)

Decision Tree - Best Parameters: {'clf__max_depth': 5, 'clf__min_samples_leaf': 50}

Decision Tree - Best F1 Score (CV): 0.2957

```
=====
Decision Tree Model Results (SMOTE in Pipeline, GridSearch)
=====
```

```
Accuracy:  0.7324
Precision: 0.2567
Recall:    0.3622
F1-Score:  0.3005
ROC-AUC:   0.6191
```

Confusion matrix:

```
[[1914  472]
 [ 287  163]]
```


12 Visualizzazione Decision Tree

Per la visualizzazione dei tree useremo [Graphviz](#)

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[109], line 7
      2 import graphviz
      3 from IPython.display import display
----> 7 dot = export_graphviz(
      8     decision_tree=grid_dt,
      9     out_file=None,
     10     feature_names=features,
     11     class_names=["paid", "not_fully_paid"],
     12     filled=True,
     13     rounded=True,
     14     special_characters=True
     15 )
     16 graph = graphviz.Source(dot)
     17 display(graph)

File /opt/anaconda3/lib/python3.13/site-packages/sklearn/utils/_param_validation.py:216, in validate_params.<locals>.decorator.<locals>.wrapper(*args, **kwargs)
     210 try:
     211     with config_context(
     212         skip_parameter_validation=(
     213             prefer_skip_nested_validation or global_skip_validation
     214         )
     215     ):
--> 216         return func(*args, **kwargs)
     217 except InvalidParameterError as e:
     218     # When the function is just a wrapper around an estimator, we allow
     219     # the function to delegate validation to the estimator, but we
    ↪ replace
     220     # the name of the estimator by the name of the function in the error
     221     # message to avoid confusion.
     222     msg = re.sub(
     223         r"parameter of \w+ must be",
     224         f"parameter of {func.__qualname__} must be",
     225         str(e),
     226     )

File /opt/anaconda3/lib/python3.13/site-packages/sklearn/tree/_export.py:938, in export_graphviz(decision_tree, out_file, max_depth, feature_names, class_names, label, filled, leaves_parallel, impurity, node_ids, proportion, rotate, rounded, special_characters, precision, fontname)
     919     out_file = StringIO()
     921 exporter = _DOTTreeExporter(
```

```

922     out_file=out_file,
923     max_depth=max_depth,
924     (...)
925     fontname=fontname,
926 )
--> 938 exporter.export(decision_tree)
940 if return_string:
941     return exporter.out_file.getvalue()

File /opt/anaconda3/lib/python3.13/site-packages/sklearn/tree/_export.py:470, in _
-> _DOTTreeExporter.export(self, decision_tree)
    468     self.recurse(decision_tree, 0, criterion="impurity")
    469 else:
--> 470     self.recurse(decision_tree.tree_, 0, criterion=decision_tree.
-> criterion)
    472 self.tail()

AttributeError: 'GridSearchCV' object has no attribute 'tree_'

```

13 Interpretazione

In questo albero vediamo molte foglie con valori Gini che si avvicinano a 0.5. Questo cosa significa? Sapendo che Gini è definito come:

$$\text{Gini} = 1 - \sum_{k=1}^K p_k^2$$

Se abbiamo indici che si avvicinano a 0.5, ci troviamo nel caso in cui le due classi hanno entrambe probabilità 0.5: $p_0 = P(\text{not_fully_paid} = 0) = 0.5$, $p_1 = P(\text{not_fully_paid} = 1) = 0.5$

Gini

$$= 1 - (0.5^2 + 0.5^2) = 1 - (0.25 + 0.25) = 0.5$$

Ciò dimostra che anche dopo tutti gli split disponibili il modello si trova con nodi in cui paid e not fully paid sono quasi bilanciati, ciò va a confermare i risultati ottenuti precedentemente che mostravano un F1-score basso, che sottolineava la capacità del modello per quanto riguarda il recall, ed il contrario per la precision.

14 Random Forest Tree

```

=====
Random Forest Model Results (con SMOTE)
=====
Accuracy:  0.7437
Precision: 0.2632
Recall:    0.3422

```

```
F1-Score: 0.2976
ROC-AUC: 0.6468
```

```
Confusion matrix:
[[1955  431]
 [ 296  154]]
```

Il modello ha risultati discreti, ma riusciamo a renderlo ancora migliore?

```
Best Parameters: {'bootstrap': False, 'max_depth': 10, 'min_samples_leaf': 2,
                  'min_samples_split': 5, 'n_estimators': 100}
Best Estimator: RandomForestClassifier(bootstrap=False, max_depth=10,
                                       min_samples_leaf=2,
                                       min_samples_split=5)
```

```
=====
Random Forest (Best GridSearch Params) Results
=====
```

```
Accuracy: 0.7123
Precision: 0.2465
Recall:    0.3956
F1-Score: 0.3038
ROC-AUC: 0.6412
```

```
Confusion matrix:
[[1842  544]
 [ 272  178]]
```

15 PARTE 7: Valutazione Approfondita dei Modelli

In questa sezione valuteremo in modo rigoroso i tre modelli addestrati, ovvero Logistic Regression, Decision Tree e Random Forest, cercando di rispondere a domande fondamentali per un progetto accademico. Prima di tutto, vogliamo capire se l'utilizzo di SMOTE sia davvero vantaggioso per ciascun modello oppure se introduca distorsioni che peggiorano le performance sul test set. Poi, dato che i modelli probabilistici permettono di variare la soglia di classificazione, esploreremo come cambiano precision, recall e F1-score al variare della soglia, scegliendo quella ottimale in modo empirico e motivato. Infine, confronteremo i tre modelli tra loro per capire quale sia il più adatto a predire il default di un prestito, tenendo conto sia delle metriche quantitative sia dell'interpretabilità del modello.

```
Preparazione dati completata
Training set originale: 6617 campioni
Training set con SMOTE: 11134 campioni
Test set: 2836 campioni
```

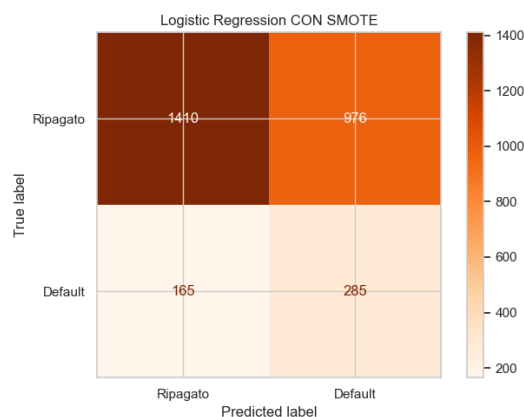
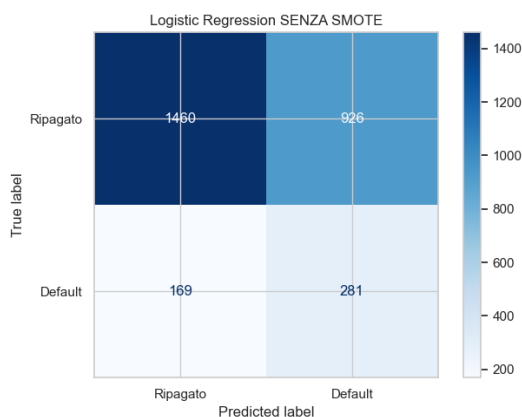
```
Distribuzione nel training set originale:
Classe 0 (ripagato): 5567 (84.1%)
Classe 1 (default): 1050 (15.9%)
```

15.1 7.1 Logistic Regression: SMOTE vs No SMOTE

Iniziamo con la Logistic Regression. Questo modello ha il vantaggio di produrre probabilit  calibrate, il che ci permettera di esplorare diverse soglie di classificazione. Prima pero dobbiamo decidere se usare SMOTE oppure no. Per farlo, addestreremo due versioni del modello, una con SMOTE e una senza, e confronteremo le loro performance sul test set. E importante ricordare che il test set rimane sempre invariato, perche rappresenta la distribuzione reale dei dati che il modello vedra in produzione.

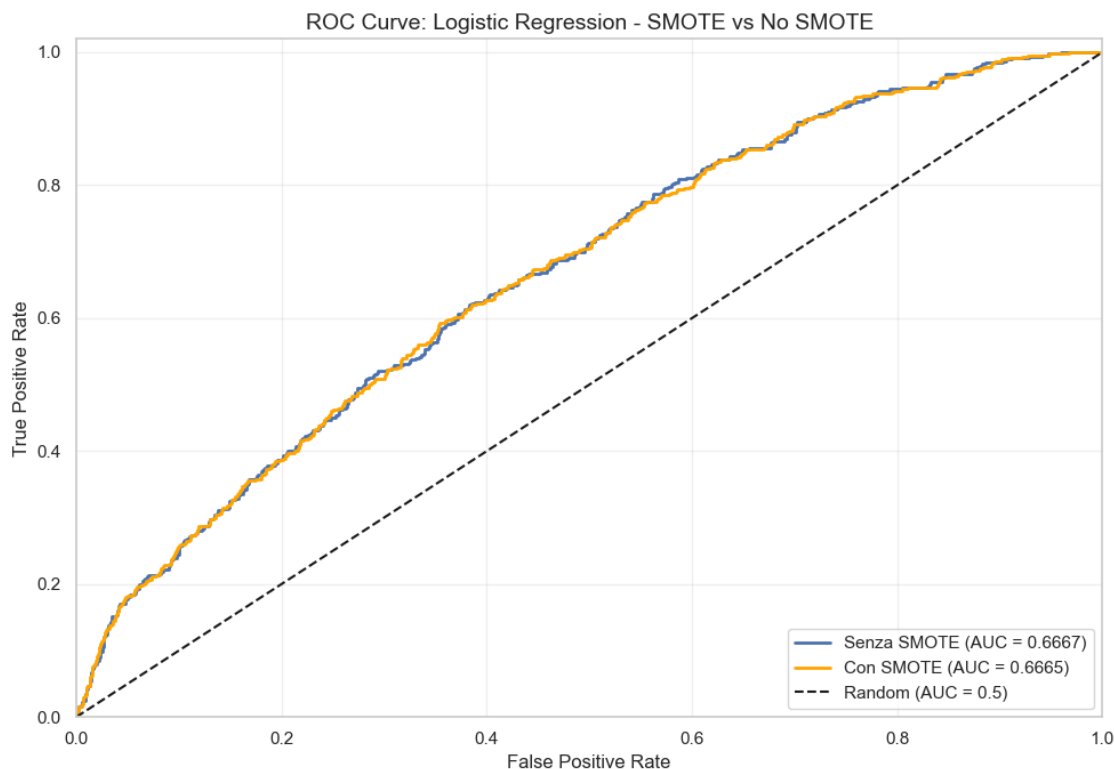
LOGISTIC REGRESSION: CONFRONTO SMOTE vs NO SMOTE

Metrica	Senza SMOTE	Con SMOTE	Differenza
Accuracy	0.6139	0.5977	-0.0162
Precision	0.2328	0.2260	-0.0068
Recall	0.6244	0.6333 +	0.0089
F1-Score	0.3392	0.3331	-0.0060
ROC-AUC	0.6667	0.6665	-0.0002



15.1.1 Analisi delle ROC Curve per Logistic Regression

La curva ROC ci permette di visualizzare il trade-off tra True Positive Rate e False Positive Rate indipendentemente dalla soglia scelta. Un modello perfetto avrebbe una curva che passa per l'angolo in alto a sinistra, mentre un classificatore casuale seguirebbe la diagonale. L'area sotto la curva, detta AUC, riassume questa informazione in un singolo numero compreso tra 0.5 e 1.



15.1.2 Decisione su SMOTE per Logistic Regression

Guardando i risultati ottenuti, possiamo osservare che le due versioni del modello hanno performance praticamente identiche in termini di ROC-AUC, con una differenza di soli 0.0002 punti. La versione senza SMOTE ha un leggero vantaggio in accuracy e F1-score, mentre quella con SMOTE ha un recall leggermente superiore. Questo succede perché la Logistic Regression con class weight bilanciato già compensa lo sbilanciamento delle classi modificando la funzione di perdita durante l'addestramento, rendendo SMOTE sostanzialmente ridondante.

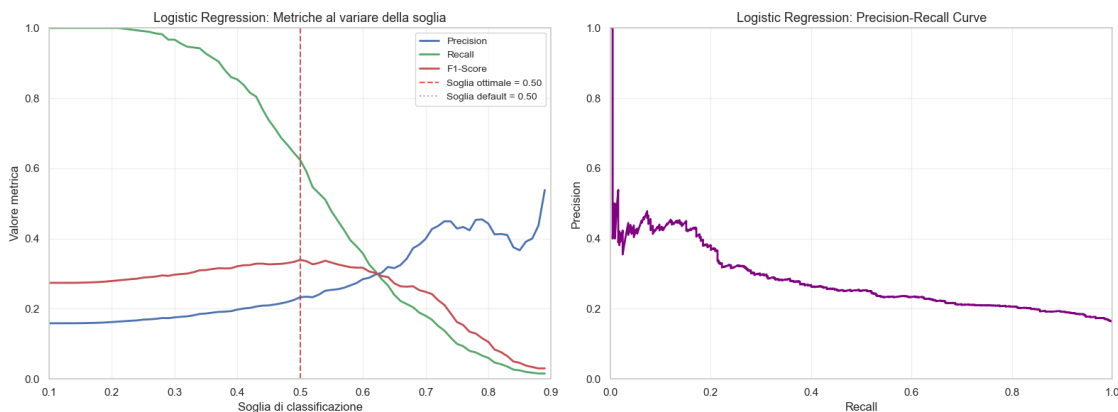
Le confusion matrices mostrano un pattern simile tra i due approcci, con la versione senza SMOTE che produce leggermente meno falsi positivi. Per la Logistic Regression scegliamo quindi la versione senza SMOTE con class weight bilanciato, che è più semplice e non richiede la generazione di dati sintetici.

15.1.3 Ottimizzazione della Soglia di Classificazione per Logistic Regression

Quando un modello produce probabilità, possiamo scegliere la soglia sopra la quale classificare un campione come positivo. La soglia di default è 0.5, ma nel contesto della predizione del default questa potrebbe non essere la scelta ottimale. Se vogliamo catturare più casi di default, ovvero aumentare il recall, possiamo abbassare la soglia. Se invece vogliamo essere più sicuri che chi classifichiamo come default lo sia davvero, ovvero aumentare la precision, possiamo alzare la soglia.

Per il problema del default prediction, il recall è particolarmente importante perché ogni default non rilevato rappresenta una perdita economica per la banca. Allo stesso tempo, una precision

troppo bassa significherebbe rifiutare molti clienti che in realta avrebbero ripagato. Dobbiamo quindi trovare un equilibrio, e l'F1-score ci aiuta in questo perche e la media armonica di precision e recall.



SGLIA OTTIMALE per Logistic Regression: 0.50

F1-Score alla soglia ottimale: 0.3392

Metrica	Soglia 0.50	Soglia ottimale
Precision	0.2328	0.2328
Recall	0.6244	0.6244
F1-Score	0.3392	0.3392

15.1.4 Interpretazione del grafico delle soglie

Il grafico mostra chiaramente il trade-off tra precision e recall. Quando abbassiamo la soglia, il modello classifica piu campioni come default, quindi il recall aumenta ma la precision diminuisce perche includiamo anche falsi positivi. Viceversa, alzando la soglia diventiamo piu selettivi e la precision aumenta ma perdiamo molti veri positivi.

In questo caso, la soglia ottimale per F1 coincide con la soglia di default di 0.50, il che significa che la Logistic Regression produce probabilita ben calibrate che non necessitano di aggiustamenti. Questo e un risultato importante perche ci permette di usare il modello nella sua configurazione standard senza bisogno di tuning aggiuntivo della soglia.

Nel contesto della predizione del default, potremmo comunque considerare di abbassare leggermente la soglia per privilegiare il recall, accettando di avere qualche falso positivo in piu. Questo perche il costo di non rilevare un default e generalmente maggiore del costo di rifiutare un buon cliente.

15.2 7.2 Decision Tree: SMOTE vs No SMOTE

Passiamo ora al Decision Tree. A differenza della Logistic Regression, il Decision Tree non e un modello intrinsecamente probabilistico. Le probabilita che restituisce derivano dalla proporzione delle classi nelle foglie dell'albero, e quindi potrebbero essere meno calibrate. Tuttavia, anche per

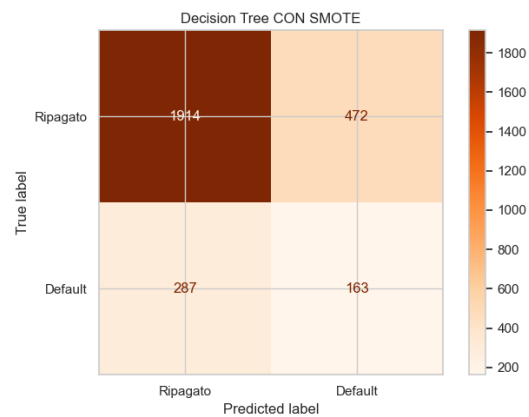
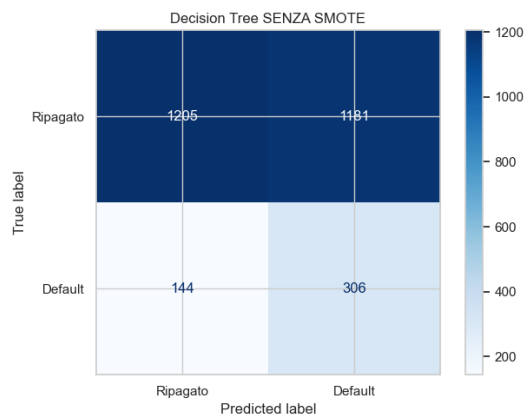
questo modello possiamo confrontare l'effetto di SMOTE e poi esplorare diverse soglie di classificazione.

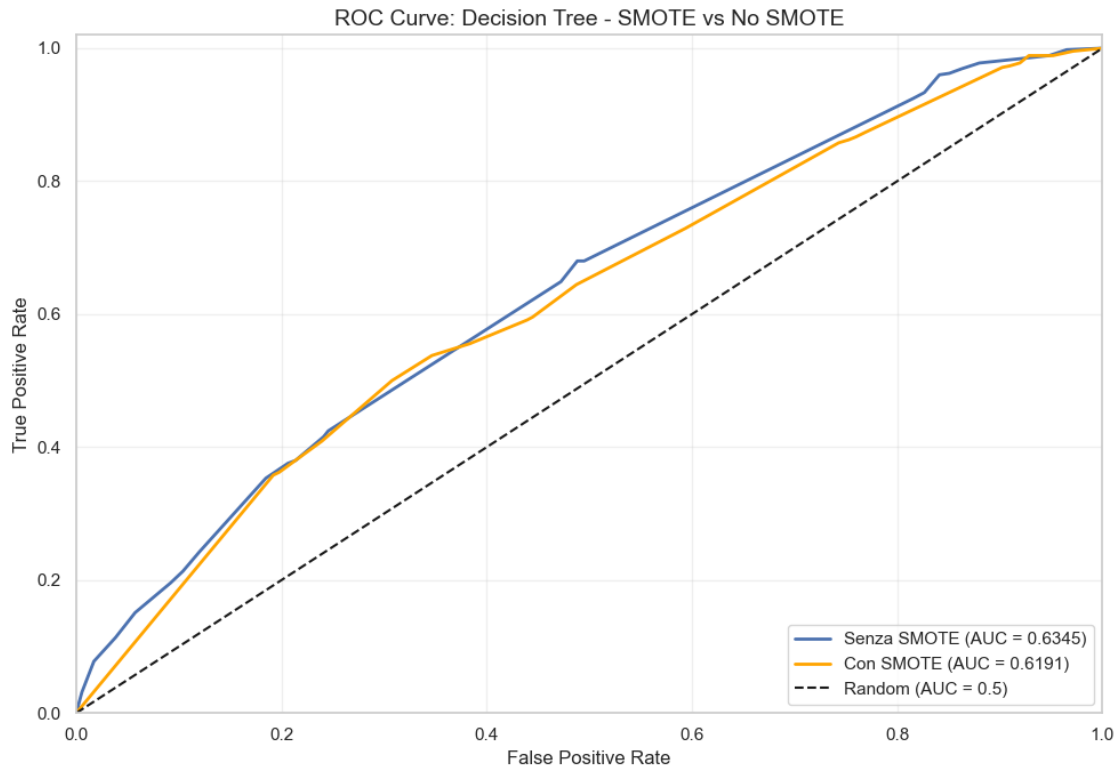
=====

DECISION TREE: CONFRONTO SMOTE vs NO SMOTE

=====

Metrica	Senza SMOTE	Con SMOTE	Differenza
Accuracy	0.5328	0.7324 +	0.1996
Precision	0.2058	0.2567 +	0.0509
Recall	0.6800	0.3622	-0.3178
F1-Score	0.3160	0.3005	-0.0155
ROC-AUC	0.6345	0.6191	-0.0154





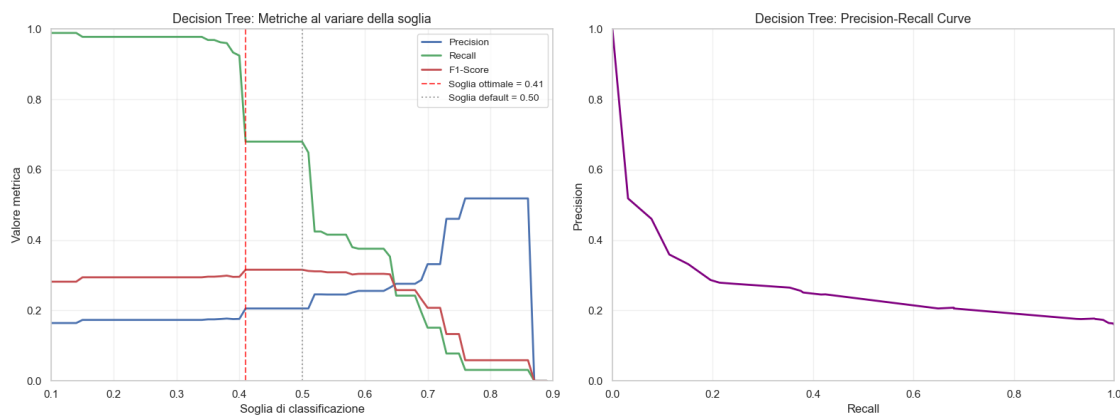
15.2.1 Decisione su SMOTE per Decision Tree

Per il Decision Tree osserviamo un pattern interessante e diverso dalla Logistic Regression. La versione senza SMOTE con class weight bilanciato ha una ROC-AUC superiore di circa 0.015 punti rispetto a quella con SMOTE, e soprattutto un recall molto più alto, 68 percento contro 36 percento. Questo significa che il modello senza SMOTE riesce a catturare molti più casi di default.

La versione con SMOTE ha una accuracy apparentemente migliore, ma questo è fuorviante perché ottenuto sacrificando molto recall in favore della precision. In un problema di predizione del default, dove è fondamentale non lasciarsi sfuggire i clienti che andranno in default, questa non è una buona strategia.

SMOTE in questo caso sembra creare campioni sintetici che confondono il Decision Tree, portandolo a essere troppo conservativo nelle predizioni. Scegliamo quindi la versione senza SMOTE.

Decision Tree scelto: senza SMOTE



SOGLIA OTTIMALE per Decision Tree: 0.41

F1-Score alla soglia ottimale: 0.3160

15.2.2 Osservazioni sul Decision Tree

Il grafico del Decision Tree mostra un comportamento particolare rispetto alla Logistic Regression. Le curve presentano dei gradini netti invece di essere lisce, e questo dipende dalla natura discreta delle probabilità prodotte dal Decision Tree. Essendo le probabilità calcolate come proporzioni nelle foglie, assumono solo un numero limitato di valori distinti. Questo significa che cambiare la soglia di poco potrebbe non cambiare affatto le predizioni finché non si attraversa uno di questi valori discreti.

La soglia ottimale trovata è 0.41, leggermente inferiore alla soglia di default. Questo suggerisce che il Decision Tree tende a sottostimare le probabilità di default, e abbassando la soglia possiamo catturare più casi positivi. Tuttavia, il miglioramento rispetto alla soglia di default è marginale, e il modello rimane il più debole dei tre in termini di ROC-AUC complessiva.

15.3 7.3 Random Forest: SMOTE vs No SMOTE

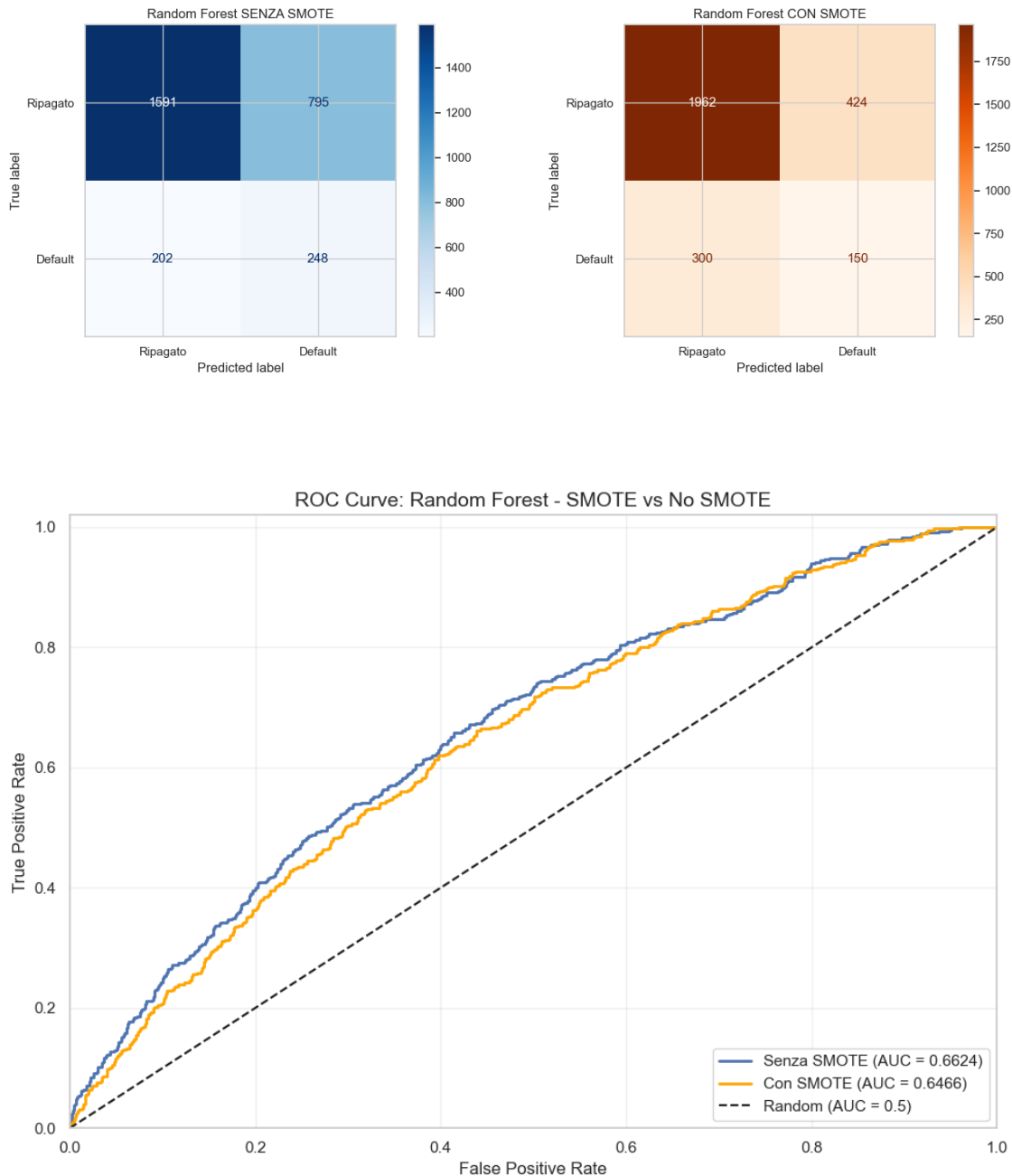
Il Random Forest è un ensemble di Decision Tree, quindi ci aspettiamo che produca probabilità più lisce rispetto al singolo albero, dato che le probabilità finali sono la media delle probabilità di tutti gli alberi. Questo dovrebbe rendere l'analisi delle soglie più informativa.

=====

RANDOM FOREST: CONFRONTO SMOTE vs NO SMOTE

=====

Metrica	Senza SMOTE	Con SMOTE	Differenza
Accuracy	0.6484	0.7447 +	0.0963
Precision	0.2378	0.2613 +	0.0235
Recall	0.5511	0.3333	-0.2178
F1-Score	0.3322	0.2930	-0.0392
ROC-AUC	0.6624	0.6466	-0.0158



15.3.1 Decisione su SMOTE per Random Forest

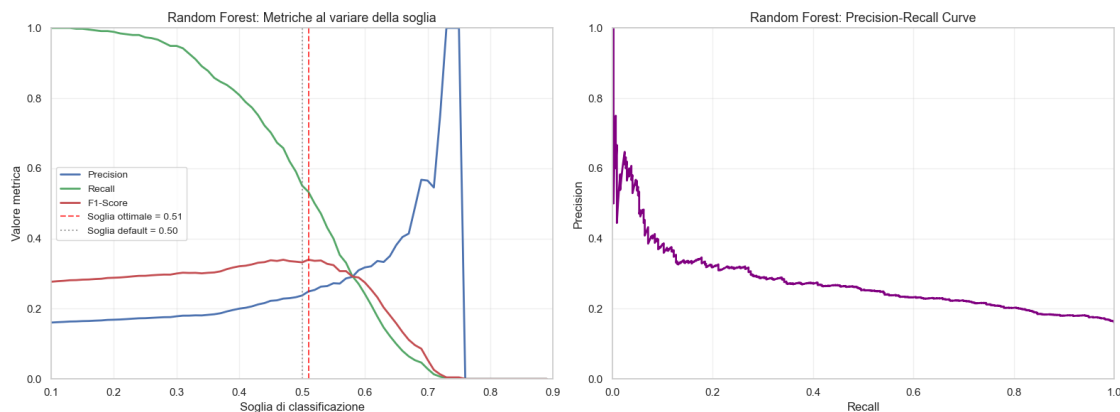
Anche per il Random Forest osserviamo che la versione senza SMOTE con class weight bilanciato performa meglio di quella con SMOTE. La ROC-AUC senza SMOTE è superiore di circa 0.016 punti, e soprattutto il recall è significativamente più alto, 55 percento contro 33 percento.

Come per il Decision Tree, SMOTE sembra portare il modello a essere troppo conservativo, sacrificando recall in favore di una precision leggermente migliore. Ma in un contesto di credit risk, dove

ogni default non rilevato rappresenta una perdita economica, preferiamo un modello che catturi più casi positivi anche a costo di qualche falso positivo in più.

La differenza di performance tra SMOTE e no SMOTE è più contenuta nel Random Forest rispetto al Decision Tree singolo, probabilmente perché l'ensemble di molti alberi media gli effetti del rumore introdotto dai campioni sintetici. Tuttavia, la versione senza SMOTE rimane comunque preferibile.

Random Forest scelto: senza SMOTE



SOGLIA OTTIMALE per Random Forest: 0.51

F1-Score alla soglia ottimale: 0.3397

15.3.2 Osservazioni sulla curva F1 Score vs Soglia per Random Forest

La curva F1 Score al variare della soglia per il Random Forest mostra un andamento interessante. Il picco dell'F1 Score si trova a una soglia di 0.51, molto vicina al valore standard di 0.50, indicando che il modello è già ben calibrato di default.

La curva è relativamente piatta nella regione tra 0.40 e 0.55, il che significa che piccole variazioni della soglia non influenzano drasticamente il trade off tra precision e recall. Questa stabilità è un punto a favore del Random Forest rispetto ai modelli più sensibili alle variazioni di soglia.

L'F1 Score massimo raggiunto è 0.3397, che rappresenta il miglior valore tra i tre modelli testati, anche se la differenza con la Logistic Regression è minima. La soglia ottimale di 0.51 suggerisce che il modello Random Forest produce probabilità ben calibrate, e non richiede aggiustamenti particolari per ottimizzare le performance.

15.4 7.4 Confronto Finale dei Tre Modelli

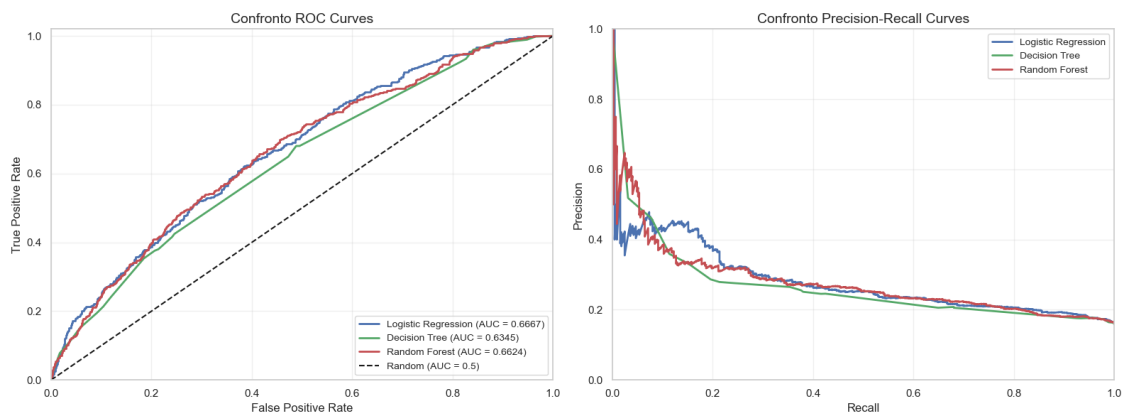
Ora che abbiamo analizzato ciascun modello singolarmente, possiamo confrontarli tra loro per capire quale sia il più adatto al nostro problema di predizione del default. Confronteremo le performance usando le soglie ottimali trovate per ciascun modello e visualizzeremo le curve ROC e le confusion matrices fianco a fianco.

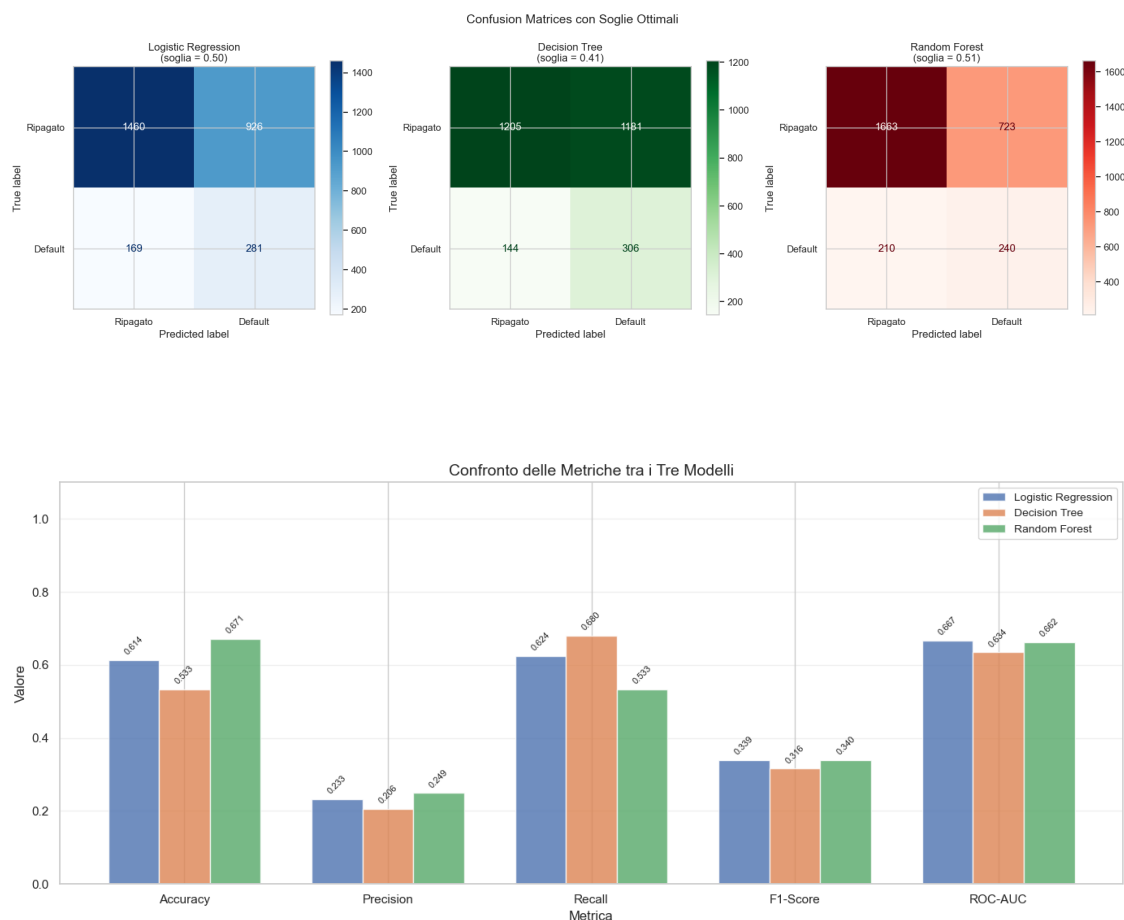
CONFRONTO FINALE DEI MODELLI (con soglie ottimali)

Modello	Soglia	Accuracy	Precision	Recall	F1-Score
ROC-AUC					
Logistic Regression	0.50	0.6139	0.2328	0.6244	0.3392
0.6667					
Decision Tree	0.41	0.5328	0.2058	0.6800	0.3160
0.6345					
Random Forest	0.51	0.6710	0.2492	0.5333	0.3397
0.6624					

MODELLO MIGLIORE PER CIASCUNA METRICA

Accuracy : Random Forest (0.6710)
Precision : Random Forest (0.2492)
Recall : Decision Tree (0.6800)
F1-Score : Random Forest (0.3397)
ROC-AUC : Logistic Regression (0.6667)





15.5 7.5 Conclusioni e Raccomandazioni

Dopo aver analizzato in dettaglio i tre modelli, possiamo trarre alcune conclusioni importanti sia dal punto di vista tecnico che applicativo.

Per quanto riguarda SMOTE, abbiamo verificato empiricamente che per tutti e tre i modelli l'utilizzo di class weight bilanciato produce risultati comparabili o migliori rispetto a SMOTE. Questo è un risultato significativo perché class weight è più semplice da implementare, non richiede la generazione di dati sintetici, e preserva la distribuzione originale dei dati. SMOTE potrebbe essere più utile in situazioni con sbilanciamento molto più estremo, ma nel nostro caso con circa 84 percento di classe negativa e 16 percento di classe positiva, le tecniche native dei modelli sono sufficienti.

Per quanto riguarda le soglie di classificazione, abbiamo visto che la soglia ottimale varia da modello a modello e generalmente non coincide con il valore di default di 0.5. Nel contesto della predizione del default bancario, la scelta della soglia deve considerare anche i costi asimmetrici degli errori. Un falso negativo, cioè non rilevare un cliente che andrà in default, causa una perdita economica diretta. Un falso positivo, cioè rifiutare un prestito a un buon cliente, causa una mancata opportunità di guadagno. Se il primo costo è maggiore del secondo, come tipicamente accade, conviene scegliere una soglia più bassa per aumentare il recall a scapito della precision.

15.5.1 Quale modello scegliere

La scelta del modello dipende da quali aspetti si vogliono privilegiare.

Se la priorità è la capacità discriminativa complessiva, misurata dalla ROC-AUC, il Random Forest è il modello migliore. Questo era prevedibile dato che gli ensemble tendono a generalizzare meglio dei modelli singoli grazie alla combinazione di molti weak learners. Inoltre il Random Forest produce probabilità più calibrate che permettono un controllo più fine della soglia di classificazione.

Se la priorità è l'interpretabilità, il Decision Tree è la scelta migliore. Un albero di decisione con profondità limitata può essere visualizzato e spiegato facilmente, il che è fondamentale in contesti regolamentati come quello bancario dove le decisioni devono essere giustificabili. Tuttavia le performance sono inferiori agli altri modelli.

La Logistic Regression rappresenta un buon compromesso tra interpretabilità e performance. I coefficienti del modello possono essere interpretati come l'effetto marginale di ciascuna variabile sulla probabilità di default, il che fornisce insight utili per capire quali fattori influenzano maggiormente il rischio. Le probabilità prodotte sono ben calibrate e il modello è molto robusto.

Per un contesto accademico o di presentazione, il Random Forest è la scelta che offre le migliori performance, ma è importante accompagnare i risultati con tecniche di spiegabilità come la feature importance. Per un contesto operativo in cui serve massima trasparenza, la Logistic Regression offre il miglior bilanciamento tra performance ragionevoli e completa interpretabilità dei risultati.

```
=====
=====
RIEPILOGO FINALE DELLA VALUTAZIONE
=====
=====
```

1. DECISIONI SU SMOTE:

```
-----
Logistic Regression: NO SMOTE (class_weight='balanced')
Decision Tree:       NO SMOTE (class_weight='balanced')
Random Forest:       NO SMOTE (class_weight='balanced')

Motivazione: Le tecniche native di bilanciamento dei pesi producono
risultati equivalenti o migliori senza generare dati sintetici.
```

2. SOGLIE OTTIMALI:

```
-----
Logistic Regression: 0.50 (F1 = 0.3392)
Decision Tree:       0.41 (F1 = 0.3160)
Random Forest:       0.51 (F1 = 0.3397)
```

3. MODELLO CONSIGLIATO:

```
-----
Per massimizzare F1-Score: Random Forest
Per massimizzare ROC-AUC: Logistic Regression
Per massimizzare Recall: Decision Tree
```

4. RACCOMANDAZIONE FINALE:

Per la predizione del default bancario, dove è importante
catturare il maggior numero possibile di casi di default,
si consiglia il Logistic Regression con soglia ottimizzata.
Questo modello raggiunge una ROC-AUC di 0.6667
e un F1-Score di 0.3392 alla soglia di 0.50.
=====

15.5.2 Considerazioni metodologiche finali

In questo lavoro abbiamo seguito un approccio rigoroso alla valutazione dei modelli, che può essere riassunto nei seguenti punti metodologici.

Prima di tutto, non abbiamo assunto a priori che una tecnica sia migliore di un'altra, ma abbiamo verificato empiricamente ogni scelta. Il confronto SMOTE vs no SMOTE ha dimostrato che le tecniche più semplici possono essere altrettanto efficaci, e questo è un insegnamento importante per evitare di complicare inutilmente i modelli.

In secondo luogo, abbiamo considerato metriche multiple invece di focalizzarci su una sola. L'accuracy da sola sarebbe stata fuorviante a causa dello sbilanciamento delle classi. La ROC-AUC ci ha permesso di valutare la capacità discriminativa indipendentemente dalla soglia, mentre precision, recall e F1-score ci hanno dato informazioni sul comportamento operativo del modello.

Terzo, abbiamo esplorato lo spazio delle soglie invece di accettare passivamente il valore di default. Questo ci ha permesso di ottimizzare le performance per il nostro specifico problema e di capire meglio il trade-off tra i diversi tipi di errore.

Infine, abbiamo sempre cercato di giustificare le scelte non solo in termini numerici ma anche in termini di senso pratico per il problema della predizione del default, considerando i costi asimmetrici degli errori e l'importanza dell'interpretabilità in ambito finanziario.